Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»

Кафедра информационных систем и программной инженерии

# ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
## к курсовому проекту по дисциплине «Распределенные программные системы» на тему

Прототип программной системы, автоматизирующей сборку операционной системы Linux для микрокомпьютеров Luckfox

Выполнил:
студент гр. ПРИ-123
Поздняков Н.Э.

Принял:
ст. пр. кафедры ИСПИ
Тимофеев А.А.

Владимир, 2026

# АННОТАЦИЯ

Разработан прототип распределённой программной системы автоматизированной сборки Linux-образов для микрокомпьютеров Luckfox Pico. Спроектированы и реализованы подсистемы конфигурирования и сборки образов: разрешение зависимостей компонентов, очередь сборок, интеграция с Luckfox SDK, хранение и выдача артефактов. Проведено тестирование, по итогам которого оптимизация повысила пропускную способность в 2,6 раза.

Пояснительная записка: 113 с., 24 рис., 38 табл., 10 листингов, 23 источника, 8 прил.

# ABSTRACT

A prototype of a distributed software system for automated building of Linux images for Luckfox Pico microcomputers was developed. The image configuration and build subsystems were designed and implemented: component dependency resolution, a build queue, Luckfox SDK integration, artifact storage and delivery. Testing was performed, and the resulting optimization increased throughput 2.6 times.

Explanatory note: 113 pages, 24 figures, 38 tables, 10 listings, 23 references, 8 appendices.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ ..... 5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ К СИСТЕМЕ ..... 7
1.1 Назначение системы, цель и задачи проекта ..... 7
1.2 Особенности создания Linux-образов для микрокомпьютеров Luckfox Pico ..... 8
1.3 Анализ существующих решений и обоснование разработки ..... 11
1.4 Словарь предметной области ..... 14
1.5 Пользователи системы и их роли ..... 15
1.6 Функциональные требования к системе ..... 16
1.7 Нефункциональные требования к системе ..... 18
2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ ..... 21
2.1 Общая архитектура системы и место подсистемы сборки ..... 21
2.2 Сценарии конфигурирования и сборки образов ..... 23
2.3 Модель образа, состав компонентов и разрешение зависимостей ..... 25
2.4 Очередь сборок и жизненный цикл образа ..... 28
2.5 Взаимодействие со сборочным инструментарием Luckfox ..... 30
2.6 Хранение, получение и обработка результатов сборки ..... 32
3 РЕАЛИЗАЦИЯ ПРОТОТИПА ПРОГРАММНОЙ СИСТЕМЫ ..... 34
3.1 Реализация управления конфигурацией образа ..... 34
3.2 Разрешение зависимостей компонентов и расчёт стоимости образа ..... 35
3.3 Асинхронная сборка и управление состояниями образа ..... 37
3.4 Интеграция с Luckfox SDK и реализация симуляционного режима ..... 39
3.5 Хранение артефактов и журналов сборки ..... 41
3.6 Реализация API заказа сборки и скачивания образов ..... 42
4 ТЕСТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ ..... 46
4.1 Тестовый стенд, методика и критерии оценки ..... 46
4.2 Модульное тестирование ..... 47

4.3 Интеграционное тестирование серверной части ..... 49
4.4 Сквозное тестирование пользовательских сценариев ..... 50
4.5 Проверка конвейера сборки и отказоустойчивости ..... 50
4.6 Нагрузочное тестирование и анализ производительности ..... 52
4.7 Анализ выявленных дефектов и оптимизация системы ..... 53
4.8 Оценка результатов и ограничения тестирования ..... 57
5 ОРГАНИЗАЦИЯ РАБОТЫ НАД ПРОЕКТОМ ..... 58
5.1 Распределение задач и зон ответственности ..... 58
5.2 Организация совместной разработки и контроль версий ..... 59
5.3 Календарный план выполнения работ ..... 61
5.4 Использование инструментов искусственного интеллекта ..... 62
ЗАКЛЮЧЕНИЕ ..... 64
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ..... 66
ПРИЛОЖЕНИЕ А РАСШИРЕННЫЕ ОПИСАНИЯ ПРЕЦЕДЕНТОВ ..... 68
ПРИЛОЖЕНИЕ Б ДИАГРАММЫ ПРЕЦЕДЕНТОВ И БИЗНЕС-ПРОЦЕССОВ ..... 75
ПРИЛОЖЕНИЕ В ДИАГРАММЫ КЛАССОВ И СОСТОЯНИЙ ..... 77
ПРИЛОЖЕНИЕ Г СХЕМЫ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТОВ И РАЗВЁРТЫВАНИЯ СИСТЕМЫ ..... 79
ПРИЛОЖЕНИЕ Д СХЕМА БАЗЫ ДАННЫХ И ОПИСАНИЕ ТАБЛИЦ ..... 81
ПРИЛОЖЕНИЕ Е ДИАГРАММА НАВИГАЦИИ И ЭКРАНЫ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА ..... 85
ПРИЛОЖЕНИЕ Ж ПРОТОКОЛЫ И СВОДНЫЕ РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ ..... 91
ПРИЛОЖЕНИЕ И ОСНОВНЫЕ ЛИСТИНГИ ПРОГРАММНОГО КОДА ..... 104

# ВВЕДЕНИЕ

Микрокомпьютеры на базе энергоэффективных систем на кристалле всё шире применяются в системах видеонаблюдения, компьютерного зрения и промышленного интернета вещей. Одним из представителей этого класса устройств является линейка плат Luckfox Pico на процессорах Rockchip RV1103 и RV1106, сочетающих малые габариты и стоимость со встроенными средствами обработки видео и изображений [4]. Штатный образ операционной системы, поставляемый производителем, ориентирован на демонстрацию возможностей платы и, как правило, не соответствует требованиям конкретного изделия: в нём отсутствуют необходимые пакеты и библиотеки, а состав системы избыточен для решаемой задачи. Поэтому разработчикам приходится формировать собственные образы, что делает актуальным вопрос упрощения и удешевления этой процедуры.

Создание собственного образа для Luckfox Pico выполняется с помощью официального SDK, построенного на основе системы сборки Buildroot [5], [6]. Для этого необходимо развернуть на локальной машине под управлением Linux среду сборки, занимающую десятки гигабайт дискового пространства, разобраться в структуре конфигурационных файлов и зависимостях между пакетами, а затем дождаться завершения кросс-компиляции, первый запуск которой занимает часы. Указанные трудоёмкость, длительность и высокие требования к ресурсам рабочей станции, а также сложность воспроизведения результата на другой машине образуют основную проблему, на решение которой направлена настоящая работа.

Цель курсового проекта заключается в разработке прототипа распределённой программной системы, позволяющей быстро сконструировать конфигурируемый Linux-образ для микрокомпьютера Luckfox Pico и получить его без запуска ресурсоёмкого процесса сборки на локальном устройстве. Система реализуется как веб-приложение с клиент-серверной архитектурой:

пользователь выбирает компоненты в визуальном редакторе, серверная часть автоматически разрешает их зависимости, помещает заказ в очередь и выполняет сборку, после чего выдаёт готовый файл образа.

Объектом исследования является процесс создания и распространения пользовательских Linux-образов для микрокомпьютеров Luckfox Pico. Предметом исследования являются методы и средства автоматизации этого процесса в распределённой программной системе: конфигурирование состава образа, разрешение зависимостей компонентов, организация очереди сборок, управление пользователями и доступом, а также проверка качества и производительности системы.

Пояснительная записка состоит из введения, пяти глав, заключения, списка использованных источников и приложений. В первой главе выполнен анализ предметной области, существующих решений и сформулированы требования к системе. Во второй главе описано проектирование программной системы, включая архитектуру, модель данных и сценарии работы. В третьей главе рассмотрена реализация прототипа. Четвёртая глава посвящена тестированию системы и оценке её производительности. В пятой главе описана организация работы над проектом. В заключении подведены итоги выполненной работы.

# 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ К СИСТЕМЕ

## 1.1 Назначение системы, цель и задачи проекта

Разрабатываемая система, получившая рабочее название Luckfox Image Builder, представляет собой распределённое веб-приложение, предназначенное для конструирования и удалённой сборки пользовательских Linux-образов для микрокомпьютеров Luckfox Pico. Система избавляет пользователя от необходимости локально разворачивать тяжёлую среду сборки, настраивать её и расходовать на сборку вычислительные ресурсы и дисковое пространство собственного компьютера.

Система предоставляет каталог готовых образов и каталог компонентов (пакетов и библиотек), визуальный редактор образов с автоматическим разрешением зависимостей, удалённую сборку с отслеживанием статуса и выдачу готового файла образа с расширением .img. Для сотрудников, отвечающих за наполнение каталога, предусмотрены административные функции: управление компонентами, изменение и удаление образов, управление учётными записями и ролями пользователей.

Практическая потребность в такой системе иллюстрируется следующим сценарием. Компания, выпускающая оборудование для видеонаблюдения и систем компьютерного зрения, переходит на использование микрокомпьютеров Luckfox для обработки видео и изображений. При переходе выясняется, что стандартный образ производителя не удовлетворяет требованиям системы, вследствие чего возникает необходимость в создании большого количества разных уникальных образов. Ручная подготовка каждого из них требует установки и освоения инструмента типа Buildroot, длительной настройки и ресурсоёмкой сборки, что и предлагается заменить работой в веб-интерфейсе.

Цель работы: разработать прототип распределённой программной системы, позволяющей удобно и быстро создать требуемый конфигурируемый

Linux-образ для микрокомпьютера Luckfox Pico без необходимости запускать долгий и требовательный процесс сборки на локальном устройстве.

Для достижения поставленной цели требуется решить следующие задачи:

– проанализировать предметную область, существующие решения и определить основные сущности, роли пользователей, функциональные и нефункциональные требования к системе;

– спроектировать архитектуру системы, модель данных и сценарии взаимодействия пользователей с системой;

– разработать средства хранения данных и организации каталогов образов и компонентов, реализовать регистрацию, аутентификацию пользователей и разграничение доступа на основе ролей;

– разработать редактор образов, позволяющий добавить в образ необходимые пакеты и библиотеки и автоматически разрешить их зависимости;

– реализовать удалённую сборку образов с последовательной обработкой заказов в очереди, отслеживанием статусов и хранением журналов сборки;

– реализовать выдачу готового образа пользователю и административные функции управления компонентами, образами и пользователями;

– разработать веб-интерфейс системы и обеспечить развёртывание всех её частей средствами контейнеризации;

– провести модульное, интеграционное, сквозное и нагрузочное тестирование системы, выявить дефекты и выполнить оптимизацию.

## 1.2 Особенности создания Linux-образов для микрокомпьютеров Luckfox Pico

Luckfox Pico – линейка компактных микрокомпьютеров на процессорах Rockchip RV1103 и RV1106. Обе микросхемы построены на архитектуре ARM

Cortex-A7 и рассчитаны на применение в устройствах видеонаблюдения и компьютерного зрения, поэтому включают аппаратные блоки обработки изображений и нейросетевого вывода [4]. Вычислительные ресурсы таких устройств ограничены: количество оперативной памяти измеряется десятками и сотнями мегабайт, а постоянным хранилищем служит флеш-память или карта microSD. Как следствие, стандартные готовые дистрибутивы общего назначения для них неприменимы, а состав системы приходится тщательно подбирать, включая в образ только действительно необходимые пакеты и библиотеки. В проекте используются платы, различающиеся моделью процессора и комплектацией: RV1103_Luckfox_Pico, RV1103_Luckfox_Pico_Plus и RV1106_Luckfox_Pico_Pro_Max.

Штатным способом получения образа для этих плат является официальный пакет разработчика (SDK) luckfox-pico, публикуемый производителем в открытом репозитории [5]. Управление сборкой осуществляется скриптом build.sh, который последовательно выполняет несколько этапов:

– выбор конфигурации платы с помощью файлов вида BoardConfig, определяющих модель процессора, тип носителя, вариант корневой файловой системы и параметры разделов;

– сборку загрузчика U-Boot;

– сборку ядра Linux с необходимыми драйверами;

– сборку корневой файловой системы (rootfs) средствами Buildroot;

– упаковку полученных частей в единый прошиваемый образ update.img.

Корневая файловая система формируется системой Buildroot – набором Makefile и патчей, который автоматизирует получение встраиваемого Linux-окружения, включая кросс-компилятор, ядро, загрузчик и пользовательские пакеты [6]. Набор пакетов задаётся конфигурацией, в которой каждому пакету соответствует параметр вида BR2_PACKAGE_имя, а зависимости между пакетами описываются в конфигурационных файлах Buildroot. По результатам

работы SDK формируются отдельные артефакты: uboot.img, boot.img, rootfs.img и итоговый update.img, предназначенный для прошивки платы.

Использование SDK предъявляет ряд требований к хост-машине. Сборка выполняется в среде Linux, поэтому при работе на других операционных системах необходима виртуальная машина или подсистема, например WSL. Требуется значительный объём свободного дискового пространства (десятки гигабайт для исходных текстов, кросс-компилятора и промежуточных файлов), выход в сеть для загрузки исходных архивов пакетов и наличие кросс-компилятора для архитектуры ARM. Первый прогон полной сборки занимает от одного до трёх часов, в дальнейшем время сокращается за счёт повторного использования результатов предыдущих этапов.

Анализ этого процесса позволяет выделить следующие проблемы, затрудняющие массовое создание собственных образов:

– сложность: для получения образа необходимо знать структуру SDK, назначение файлов конфигурации плат и правила описания зависимостей пакетов, а ошибки в конфигурации обнаруживаются лишь на поздних этапах сборки;

– длительность: полная сборка занимает часы, и каждое изменение состава образа требует повторного запуска процесса;

– ресурсоёмкость: сборка занимает десятки гигабайт диска и существенно нагружает процессор и оперативную память рабочей станции;

– воспроизводимость: результат зависит от состояния локальной среды, версий инструментов и доступности внешних источников, поэтому у разных разработчиков один и тот же состав может привести к разным образам или ошибкам;

– параллельные сборки: SDK использует единый рабочий каталог, и одновременный запуск нескольких сборок в нём приводит к взаимному повреждению файлов, что необходимо учитывать при организации централизованного сервиса.

Отдельные трудности возникают при работе в WSL. Во-первых, в переменную окружения PATH в этой среде автоматически добавляются пути Windows, содержащие пробелы (например, /mnt/c/Program Files/...), а Buildroot отказывается выполнять сборку, если в PATH присутствуют такие элементы. Во-вторых, при загрузке исходных архивов утилита wget без ограничений по времени может надолго «зависать» на недоступных зеркалах, не переходя к резервным источникам. Оба обстоятельства были выявлены при отладке реальной сборки и учтены при разработке сборочных скриптов системы (санитизация PATH и настройка тайм-аутов загрузки).

Таким образом, процесс создания образа для Luckfox Pico является сложным, длительным и ресурсоёмким, а его результат зависит от окружения. Это обосновывает вынесение сборки на выделенный сервер с готовой и проверенной средой и предоставление пользователю упрощённого интерфейса конфигурирования.

## 1.3 Анализ существующих решений и обоснование разработки

Для анализа были выбраны решения, с помощью которых на практике формируются образы для встраиваемых устройств, а также сервисы, упрощающие получение готовых образов. К ним относятся ручная сборка через SDK и Buildroot, Yocto Project, Armbian build framework, Raspberry Pi Imager и OpenWrt Firmware Selector.

Ручная сборка через SDK Luckfox и Buildroot [5], [6] является штатным способом и обеспечивает полный контроль над содержимым образа. Пользователь самостоятельно настраивает конфигурацию с помощью меню Buildroot, а зависимости пакетов разрешаются средствами самой системы сборки. Недостатками остаются описанные в подразделе 1.2 требования к локальной среде, длительность сборки и высокий порог входа.

Yocto Project [7] – универсальный набор инструментов для создания собственных Linux-дистрибутивов для встраиваемых систем. Он предоставляет

мощную систему слоёв и рецептов и подходит для промышленных проектов, однако отличается ещё более высокой сложностью освоения и большими требованиями к ресурсам. Для Luckfox Pico официальная поддержка производителя ориентирована на SDK на основе Buildroot, поэтому применение Yocto потребовало бы самостоятельной подготовки слоя поддержки платы.

Armbian build framework [19] – набор скриптов для сборки дистрибутивов Armbian для широкого круга одноплатных компьютеров на архитектуре ARM. Он позволяет выбирать плату, ядро и набор пакетов, но выполняется локально, требует подготовленной среды и ориентирован на платы общего назначения, а не на процессоры RV1103 и RV1106.

Raspberry Pi Imager [20] – настольное приложение для записи готовых образов на карты памяти. Оно имеет простой интерфейс и низкий порог входа, а также позволяет задать ряд начальных настроек (имя узла, сеть, учётную запись), однако не выполняет сборку и не позволяет выбирать состав пакетов, а список поддерживаемых устройств ограничен платами Raspberry Pi и совместимыми образами.

OpenWrt Firmware Selector [21] – веб-сервис, позволяющий выбрать устройство, добавить или исключить пакеты и получить собранную на сервере прошивку без развёртывания локальной среды. По принципу работы он является ближайшим аналогом разрабатываемой системы, но предназначен для маршрутизаторов и других устройств, работающих под управлением OpenWrt, и не поддерживает платы Luckfox Pico.

Сравнение рассмотренных решений выполнено по следующим критериям: поддержка Luckfox Pico, возможность выбора пакетов, автоматическое разрешение зависимостей, удалённая сборка без локальной среды, наличие веб-интерфейса, наличие библиотеки готовых образов и низкий порог входа. Результаты сравнения приведены в таблице 1, где «+» означает наличие возможности, «–» её отсутствие, «частично» – ограниченную или косвенную поддержку.

Таблица 1 – Сравнение существующих решений и разрабатываемой системы

| **Критерий** | **SDK Luckfox и Buildroot** | **Yocto Project** | **Armbian build** | **Raspberry Pi Imager** | **OpenWrt Firmware Selector** | **Разрабатываемая система** |
|---|---|---|---|---|---|---|
| Поддержка Luckfox Pico | + | частично | – | – | – | + |
| Выбор пакетов | + | + | частично | – | + | + |
| Автоматическое разрешение зависимостей | + | + | частично | – | + | + |
| Удалённая сборка без локальной среды | – | – | – | – | + | + |
| Веб-интерфейс | – | – | – | – | + | + |
| Библиотека готовых образов | – | – | частично | + | + | + |
| Низкий порог входа | – | – | частично | + | + | + |

Из таблицы 1 видно, что решения, поддерживающие Luckfox Pico и гибкий выбор состава, требуют локальной среды и высокой квалификации, а сервисы с низким порогом входа и удалённой сборкой либо не выполняют сборку (Raspberry Pi Imager), либо ориентированы на другие устройства (OpenWrt Firmware Selector). Готового решения, которое объединяет поддержку Luckfox Pico, визуальный выбор компонентов с автоматическим разрешением зависимостей, удалённую сборку и каталог готовых образов, не выявлено.

Разработка собственной системы обоснована следующим. Во-первых, потребность в большом количестве уникальных образов для Luckfox Pico не покрывается ни штатным образом производителя, ни сервисами для других платформ. Во-вторых, вынесение сборки на сервер позволяет один раз подготовить и проверить среду, устранив зависимость результата от рабочей станции каждого разработчика. В-третьих, автоматическое разрешение зависимостей и визуальный редактор снижают порог входа: пользователю достаточно выбрать нужные компоненты, не изучая устройство SDK. В-четвёртых, каталог готовых образов позволяет повторно использовать однажды созданные конфигурации. Идея онлайн-сборки с выбором пакетов, реализованная в OpenWrt Firmware Selector, является проверенной практикой и переносится в проект применительно к платам Luckfox Pico.

## 1.4 Словарь предметной области

На основе анализа предметной области выделен набор ключевых понятий, используемых при проектировании и реализации системы. Каждое понятие рассмотрено в словаре, приведённом в таблице 2: даны его определение и основные атрибуты, соответствующие модели данных системы. Взаимосвязи между сущностями предметной области показаны на диаграмме классов (см. приложение В).

Таблица 2 – Словарь предметной области

| **Термин** | **Определение** | **Основные атрибуты** |
|---|---|---|
| Пользователь | Зарегистрированное в системе лицо, работающее с каталогами и редактором образов | Идентификатор, имя, фамилия, электронная почта (уникальна), пароль (хранится в виде хэша), роли, заказанные образы |
| Роль | Именованная категория пользователей, определяющая доступные им операции; в системе используются роли USER и ADMIN | Идентификатор, название роли, набор прав |
| Право (Permission) | Элементарное разрешение на выполнение действия, назначаемое роли | Идентификатор, описание действия |
| Образ (Image) | Собираемый или собранный образ операционной системы для платы Luckfox Pico с заданным составом компонентов | Идентификатор, название, описание, дистрибутив, плата, стоимость, статус, путь к артефакту, журнал сборки, версия записи, состав |
| Компонент (Component) | Программный пакет или библиотека, которые могут быть включены в образ | Идентификатор, название, тип, описание, стоимость, версия записи, зависимости |
| Тип компонента | Справочная категория компонента; в каталоге используются типы «Пакет» и «Библиотека» | Идентификатор, название типа |
| Зависимость компонента | Отношение, при котором для работы одного компонента необходим другой; зависимости образуют ориентированный граф, не допускающий циклов | Родительский компонент, зависимый компонент |
| Состав образа | Множество компонентов, включённых в образ, с учётом транзитивных зависимостей | Образ, набор компонентов |
| Заказ сборки | Запрос пользователя на сборку образа с выбранным названием, дистрибутивом, платой и набором компонентов | Название, описание, дистрибутив, плата, список идентификаторов компонентов, заказчик |

| Термин | Определение | Основные атрибуты |
|---|---|---|
| Очередь сборок | Последовательность заказов, ожидающих обработки; заказы выполняются по одному в порядке поступления | Ёмкость очереди (1 000 заказов), порядок обработки FIFO, число рабочих потоков (1) |
| Сборочный воркер | Фоновый поток серверной части, который берёт заказ из очереди и запускает сборочный скрипт | Имя потока, текущий образ, код завершения скрипта |
| Артефакт (.img) | Файл образа, получаемый в результате успешной сборки и доступный для скачивания | Путь к файлу, формат файла, привязка к образу |
| Журнал сборки | Текстовый вывод сборочного скрипта, сохраняемый вместе с образом и используемый для диагностики ошибок | Текст журнала, образ |
| Статус образа | Состояние образа в жизненном цикле сборки: QUEUED, BUILDING, READY, FAILED | Значение статуса, допустимые переходы |
| Дистрибутив | Базовая система, на основе которой формируется образ; в прототипе по умолчанию используется Buildroot | Название |
| Плата (board) | Модель микрокомпьютера Luckfox Pico, для которой собирается образ, например RV1103_Luckfox_Pico | Название конфигурации платы |
| SDK | Официальный пакет разработчика luckfox-pico, содержащий средства сборки загрузчика, ядра и корневой файловой системы | Путь к каталогу SDK, конфигурации плат |
| Режим симуляции | Режим работы сборочного скрипта по умолчанию, при котором вместо кросс-компиляции формируется ext4-файл с манифестом и списком пакетов; получаемый файл не является загрузочным образом | Признак режима (переменная LUCKFOX_REAL_BUILD), искусственная задержка |

## 1.5 Пользователи системы и их роли

В ходе анализа предметной области выделены категории целевой аудитории, потребности которых необходимо учесть при проектировании и реализации системы.

К первичной категории относятся разработчики программного обеспечения для микрокомпьютеров Luckfox, в том числе сотрудники компании, выпускающей оборудование для видеонаблюдения и систем компьютерного зрения. Их основная проблема заключается в необходимости тратить время и место на диске рабочей станции для создания образа. Потребность состоит в

возможности собрать требуемый образ удалённо, не нагружая и не загромождая собственный компьютер, и получить в результате воспроизводимый артефакт.

К вторичной категории относятся энтузиасты и люди, занимающиеся разработкой на микрокомпьютерах в качестве хобби. Основной проблемой для них является сложность создания образа. Потребность состоит в возможности получить нужный образ, не тратя время на изучение зависимостей и установку сложных инструментов.

Исходя из целевой аудитории, определены следующие роли пользователей системы. Взаимодействие ролей с функциями системы показано на диаграмме прецедентов (см. приложение Б).

– Гость (неавторизованный пользователь) – человек, не прошедший аутентификацию. Ему доступен просмотр библиотек образов и компонентов (с поиском и фильтрацией), а также регистрация и вход в систему.

– Клиент (роль USER) – основной пользователь системы. Помимо возможностей гостя, он может создавать образ в редакторе, заказывать его сборку, просматривать свои образы и их статусы в разделе «Мои образы» и скачивать готовые образы. Роль USER назначается при регистрации по умолчанию.

– Администратор (роль ADMIN) – пользователь, ответственный за наполнение каталогов и безопасную работу системы. Он обладает всеми возможностями клиента, а также управляет компонентами (создание, изменение, удаление), изменяет и удаляет образы (при изменении состава образ пересобирается), управляет учётными записями пользователей и их ролями.

### 1.6 Функциональные требования к системе

На основе проведённого анализа сформирован перечень функциональных требований. В перечень включены функции, реализованные в прототипе. Оплата заказа и доставка готового образа по электронной почте рассматривались на этапе анализа как часть сценария заказа, однако в прототипе не реализованы и

заменены выдачей готового файла образа через скачивание авторизованному пользователю.

– система должна обеспечивать регистрацию пользователей по адресу электронной почты и паролю и вход в систему с выдачей токена доступа;

– система должна предоставлять любому посетителю просмотр библиотеки образов и библиотеки компонентов с постраничным выводом, поиском по названию и сортировкой;

– система должна обеспечивать фильтрацию образов по статусу, плате и составу компонентов, а компонентов – по типу и сортировку по цене;

– система должна предоставлять авторизованному пользователю визуальный редактор образа, в котором задаются название, описание, дистрибутив, плата и набор компонентов;

– система должна автоматически дополнять выбранный набор компонентов их транзитивными зависимостями и отклонять заказ при обнаружении цикла в графе зависимостей;

– система должна рассчитывать стоимость образа как сумму стоимостей всех вошедших в него компонентов;

– система должна принимать заказ на сборку образа и выполнять сборку асинхронно, обрабатывая заказы последовательно по одному в порядке поступления;

– система должна поддерживать статусы образа QUEUED (в очереди), BUILDING (сборка), READY (готов) и FAILED (ошибка) и отображать актуальный статус пользователю с автоматическим обновлением;

– система должна сохранять журнал сборки образа и путь к полученному артефакту;

– система должна предоставлять авторизованному пользователю скачивание готового файла образа в формате .img;

– система должна показывать авторизованному пользователю список заказанных им образов в разделе «Мои образы»;

– система должна обеспечивать администратору создание, изменение и удаление компонентов с указанием типа и зависимостей;

– система должна обеспечивать администратору изменение образов с их повторной постановкой в очередь сборки и удаление образов вместе с артефактами;

– система должна запрещать изменение образа, находящегося в очереди или в процессе сборки;

– система должна обеспечивать администратору просмотр списка пользователей, создание, изменение и удаление учётных записей и изменение ролей пользователей;

– система должна поддерживать два режима сборки: режим симуляции, формирующий проверочный ext4-артефакт без кросс-компиляции, и режим реальной сборки с помощью SDK Luckfox.

## 1.7 Нефункциональные требования к системе

Нефункциональные требования описывают не прямую функциональность системы, а критерии, определяющие качество её работы, удобство использования и эксплуатационные характеристики. Требования сформулированы на этапе анализа и уточнены при подготовке методики тестирования.

Требования по производительности:

– время отклика системы для стандартных операций, кроме сборки образов, не должно превышать 5 секунд при одновременной работе до 500 пользователей;

– максимальное время сборки одного образа не должно превышать 24 часа;

– при нагрузочном тестировании доля ошибочных ответов не должна превышать 1 %;

– 95-й процентиль времени чтения (просмотр каталогов) не должен превышать 500 мс;

– 95-й процентиль времени записи не должен превышать 1 000 мс;

– 95-й процентиль времени входа в систему не должен превышать 1 000 мс;

– 95-й процентиль времени скачивания образа размером 18 МБ не должен превышать 3 000 мс.

Требования по надёжности:

– сборки должны выполняться последовательно, одна за другой, поскольку SDK использует единый рабочий каталог и параллельный запуск повреждает результаты;

– операции, затрагивающие несколько таблиц, должны выполняться атомарно в рамках транзакций;

– при одновременном редактировании образов и компонентов должны предотвращаться потерянные обновления с помощью оптимистичной блокировки;

– ошибка сборки одного образа не должна прекращать обработку очереди, а причина ошибки должна сохраняться в журнале сборки.

Требования по безопасности:

– доступ к заказу сборки и скачиванию образов должен быть разрешён только после аутентификации, а операции управления компонентами, образами и пользователями – только администратору;

– аутентификация должна выполняться с использованием токенов JWT без хранения сессии на сервере;

– пароли пользователей должны храниться в базе данных только в виде хэшей BCrypt;

– пароль должен содержать не менее 8 символов, включая цифру, строчную и заглавную буквы и специальный символ;

– система должна разграничивать права доступа в соответствии с ролями «Гость», «Клиент» (USER) и «Администратор» (ADMIN);

– число неудачных попыток входа должно ограничиваться, чтобы затруднить подбор паролей.

Требования к интерфейсу:

– интерфейс должен быть интуитивно понятным и не требовать от пользователя знаний об устройстве SDK и Buildroot;

– интерфейс должен быть адаптивным и корректно отображаться на основных разрешениях экрана 1920×1080 и 375×667;

– состояние сборки образа должно быть видно пользователю в библиотеке и в личном разделе без ручного обновления страницы.

Требования к развёртыванию и эксплуатации:

– система должна разворачиваться средствами Docker Compose в виде трёх сервисов: базы данных, серверной части и клиентской части [18];

– параметры подключения к базе данных, путь к сборочному скрипту, каталог артефактов и режим сборки должны задаваться переменными окружения;

– режим сборки по умолчанию должен быть безопасным для демонстрации (симуляция), а переход к реальной сборке должен выполняться установкой переменной окружения без изменения кода.

# 2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ

## 2.1 Общая архитектура системы и место подсистемы сборки

Программная система Luckfox Image Builder построена по клиент-серверной схеме из трёх независимо развёртываемых частей: одностраничного веб-приложения (SPA) на React [12], серверной части на Spring Boot и реляционной СУБД MySQL. Клиентская часть отвечает только за отображение данных и сбор пользовательского ввода, вся бизнес-логика – проверка состава образа, разрешение зависимостей, расчёт стоимости, управление очередью сборок – выполняется на сервере. Взаимодействие частей осуществляется по протоколу HTTP в формате JSON, аутентификация – по токену JWT, передаваемому в заголовке Authorization [8].

Серверная часть разделена на слои с односторонними зависимостями: контроллеры (приём HTTP-запроса, валидация DTO, формирование ответа), сервисы (бизнес-правила и границы транзакций), репозитории Spring Data JPA (доступ к данным) и сущности JPA (отображение таблиц) [22]. Наружу отдаются только объекты передачи данных (DTO), что исключает рекурсивную сериализацию графа зависимостей компонентов и обращения к ленивым коллекциям за пределами транзакции.

В рамках общей архитектуры выделена подсистема конфигурирования и сборки образов, проектирование и реализация которой составляют предмет данной записки. Подсистема принимает от клиента описание желаемого образа, приводит его к полному непротиворечивому составу, ставит заказ в очередь, выполняет сборку во внешнем процессе и предоставляет готовый файл образа для скачивания. Смежные подсистемы – пользовательский интерфейс, модель данных, аутентификация и администрирование каталогов – используются ею через общие контракты и описаны в записке второго участника проекта.

Состав компонентов подсистемы и их ответственность приведены в таблице 3, общая схема взаимодействия компонентов – на рисунке Г.1, схема развёртывания – на рисунке Г.3 (см. приложение Г).

Таблица 3 – Компоненты подсистемы конфигурирования и сборки образов

| **Компонент** | **Слой** | **Ответственность** |
|---|---|---|
| ImageController | Веб-слой | Эндпоинты заказа, изменения, удаления образа и скачивания артефакта |
| BuildRequest, ImageDto | Контракты | Формат заказа сборки и представление образа со статусом |
| ImageService | Бизнес-логика | Приём конфигурации, разрешение зависимостей, проверка циклов, расчёт стоимости, постановка в очередь после фиксации транзакции |
| AsyncConfig (buildExecutor) | Инфраструктура | Очередь сборок: пул из одного потока и очередь ожидающих заказов |
| BuildService | Бизнес-логика | Выполнение сборки: запуск сценария, сбор журнала, контроль таймаута, определение результата |
| BuildStateService | Бизнес-логика | Короткие транзакции изменения статуса образа и чтения входных данных сборки |
| NotificationService | Бизнес-логика | Уведомление о готовности образа (в прототипе – запись в журнал со ссылкой на скачивание) |
| scripts/luckfox_build.sh | Внешний процесс | Адаптер к сборочному инструментарию: реальная сборка через Luckfox SDK или симуляция |
| Каталог output/images | Файловое хранилище | Хранение готовых файлов образов (*.img) |

При проектировании подсистемы учитывались три свойства предметной области. Во-первых, сборка образа длится от нескольких секунд в режиме симуляции до одного–трёх часов при реальной сборке SDK, поэтому она не может выполняться в потоке обработки HTTP-запроса: заказ должен подтверждаться немедленно, а сборка – продолжаться в фоне. Во-вторых, Luckfox SDK использует единый рабочий каталог (файл выбранной конфигурации платы .BoardConfig.mk и каталог output), поэтому две сборки не могут выполняться одновременно в одной копии SDK. В-третьих, итоговый файл образа имеет размер от десятков до сотен мегабайт, поэтому хранится в файловой системе, а в базе данных фиксируется только путь к нему и журнал сборки.

Из этих свойств следует основное архитектурное решение: запрос на сборку обрабатывается в два этапа. Синхронный этап в транзакции проверяет и

сохраняет заказ, после чего клиент получает ответ 202 Accepted с образом в статусе QUEUED. Асинхронный этап выполняется единственным сборочным потоком, который последовательно извлекает заказы из очереди и запускает внешний сценарий сборки. Клиент узнаёт о результате, периодически запрашивая статус образа.

## 2.2 Сценарии конфигурирования и сборки образов

На диаграмме прецедентов UML, разработанной на этапе анализа (см. рисунок Б.1 приложения Б) [2], [3], подсистеме сборки соответствуют прецеденты клиента «Заказать сбор образа» с включаемыми «Выбрать параметры образа» и «Выбрать основу для образа» и расширяющими «Выбрать доп библиотеки» и «Выбрать доп компоненты», а также прецеденты администратора «Управлять образами» («Добавить образ», «Изменить образ», «Удалить образ»). Включаемые прецеденты «Оплатить операцию» и «Предоставить email адрес для получения образа» относятся к предметной области, но в прототипе не реализованы: готовый образ выдаётся через скачивание. В реализации выбор параметров и компонентов объединён в редакторе образа, заказ дополнен отслеживанием статуса сборки и скачиванием файла .img, а сама сборка выполняется сборочным воркером без участия пользователя. Расширенные описания прецедентов в их реализованном виде приведены в приложении А.

Основной сценарий – заказ сборки. Авторизованный пользователь в редакторе образа задаёт название и описание, выбирает базовый дистрибутив и плату, добавляет компоненты из каталога и нажимает «Заказать образ». Клиентская часть отправляет запрос POST /api/images/build, содержащий только идентификаторы выбранных компонентов. Сервер загружает компоненты, дополняет состав транзитивными зависимостями, проверяет граф на циклы, рассчитывает стоимость, сохраняет образ в статусе QUEUED, связывает его с заказчиком и после фиксации транзакции передаёт задачу в очередь. Бизнес-

процесс заказа и сборки в виде диаграммы деятельности с дорожками «Клиент», «Серверная часть» и «Сборочный воркер» приведён на рисунке Б.2, последовательность вызовов между компонентами – на диаграмме последовательности (см. рисунок Г.2).

Сценарий отслеживания статуса реализован опросом: страницы «Библиотека образов» и «Аккаунт» повторно запрашивают данные каждые 5 с, пока образ не перейдёт в конечное состояние READY или FAILED. Для прототипа такой подход проще двунаправленного канала (WebSocket, Server-Sent Events) и не требует хранения соединений на сервере; при нагрузочном тестировании опрос статуса не создавал заметной нагрузки (см. раздел 4.6).

Сценарий скачивания доступен для образа в статусе READY, у которого существует файл артефакта. Сценарии изменения и удаления образа доступны администратору: изменение заменяет параметры и состав образа, удаляет старый артефакт и повторно ставит образ в очередь; удаление снимает связь с заказчиками, удаляет запись и файл.

Важным результатом проектирования стало согласование контракта между клиентской частью и подсистемой сборки. Клиент передаёт намерение пользователя (выбранные компоненты), а итоговый состав образа определяет сервер. Поэтому предварительная стоимость в редакторе рассчитывается по выбранным компонентам, а окончательная – на сервере с учётом всех зависимостей. Поля запроса на сборку приведены в таблице 4, поля ответа – в таблице 5.

Таблица 4 – Поля запроса на сборку образа (BuildRequest)

| **Поле** | **Тип** | **Обязательность** | **Назначение** |
|---|---|---|---|
| name | строка | да (@NotBlank) | Название образа |
| description | строка | нет | Описание образа |
| distro | строка | да (@NotBlank) | Базовый дистрибутив (например, buildroot) |
| board | строка | нет | Целевая плата; по умолчанию RV1103_Luckfox_Pico |
| componentIds | список чисел | да (@NotEmpty) | Идентификаторы компонентов, выбранных пользователем |

Таблица 5 – Поля представления образа (ImageDto)

| **Поле** | **Назначение** |
|---|---|
| id, name, description | Идентификатор, название и описание образа |
| distro, board | Дистрибутив и плата |
| cost | Стоимость образа с учётом зависимостей |
| state | Статус: QUEUED, BUILDING, READY или FAILED |
| downloadable | Признак доступности скачивания (статус READY и известен путь к артефакту) |
| components | Полный состав образа после разрешения зависимостей (без вложенных зависимостей) |

Путь к файлу и журнал сборки в представление не входят: они являются внутренними данными сервера. Это исключает раскрытие структуры файловой системы и уменьшает объём ответов при просмотре библиотеки.

## 2.3 Модель образа, состав компонентов и разрешение зависимостей

Центральными сущностями подсистемы являются образ (Image) и компонент (Component); диаграмма классов приведена на рисунке В.1 приложения В. Образ хранит параметры сборки (название, описание, дистрибутив, плату), стоимость, статус, путь к артефакту, журнал сборки и номер версии для оптимистичной блокировки. Состав образа хранится в связующей таблице image_component_dependency (отношение «многие ко многим»). Компонент хранит название, тип (пакет или библиотека), описание и стоимость; его зависимости хранятся в таблице dependency как отношение компонента к самому себе.

Множество компонентов с отношением зависимости образует ориентированный граф $G = (V, E)$, где $V$ – компоненты, а ребро $(u, v)$ означает, что компонент $u$ требует компонент $v$. Для корректного образа к составу предъявляются три требования: полнота (в образ входят все компоненты, достижимые из выбранных), ацикличность (граф зависимостей в пределах образа не содержит циклов) и однократность (каждый компонент входит в образ

и в стоимость один раз, даже если его требуют несколько выбранных компонентов).

Полнота обеспечивается построением транзитивного замыкания: множество компонентов образа – это все вершины, достижимые из выбранных. Для его построения выбран обход графа в ширину (BFS) [13]. Алгоритм хранит множество result уже включённых компонентов и очередь queue компонентов, ожидающих обработки. Извлечённый из очереди компонент добавляется в result, если его там ещё нет, а его зависимости ставятся в очередь. Проверка принадлежности множеству обеспечивает однократность и завершение алгоритма даже при наличии циклов. Сложность обхода – O(|V| + |E|) для подграфа, достижимого из выбранных компонентов.

Ацикличность проверяется отдельным обходом в глубину (DFS) [13] по построенному замыканию. Помимо множества посещённых вершин visited поддерживается множество inStack вершин текущего пути. Ребро в вершину из inStack означает цикл, и заказ отклоняется с ошибкой 400 и указанием компонента, участвующего в цикле. Алгоритм разрешения зависимостей и проверки циклов представлен на рисунке 1.

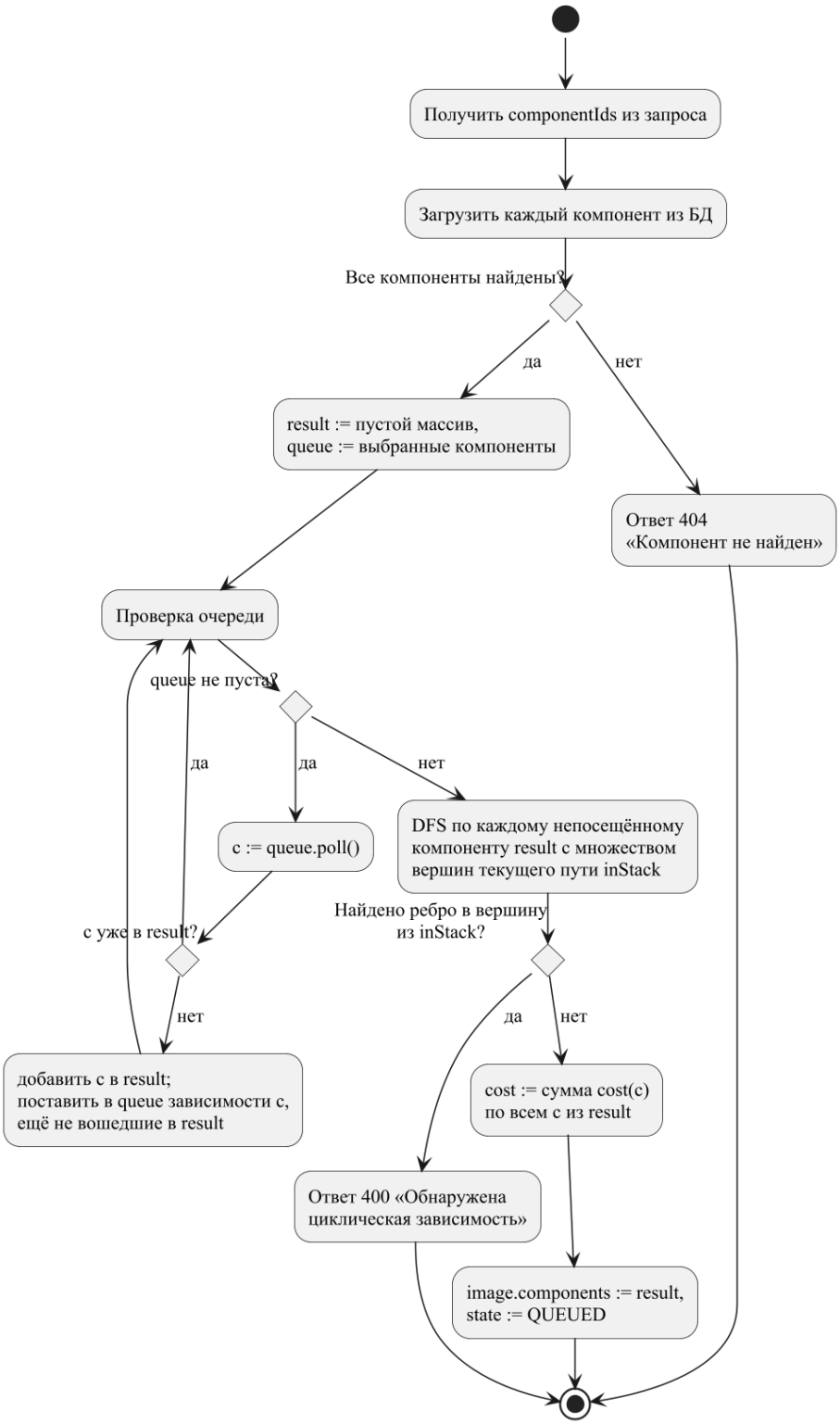



Рисунок 1 – Алгоритм разрешения зависимостей и проверки графа на циклы

Проверка на циклы выполняется на двух уровнях. При изменении компонента администратором ComponentService запрещает зависимость компонента от самого себя и зависимость, которая транзитивно ведёт обратно к изменяемому компоненту, поэтому циклы не должны попадать в каталог.

Проверка при заказе сборки является защитой второго уровня: она гарантирует, что сборка не будет запущена с некорректным составом, даже если данные изменены в обход сервиса, например непосредственно в базе данных.

Стоимость образа рассчитывается как сумма стоимостей всех компонентов замыкания. Благодаря множественной семантике каждый компонент учитывается ровно один раз. Например, при выборе компонента opencv-python (стоимость 80 единиц), который зависит от numpy (30) и python3 (50), причём numpy сам зависит от python3, состав образа – {opencv-python, numpy, python3}, а стоимость – 160 единиц. Этот пример используется в интеграционном тесте INT-29.

Состав и стоимость фиксируются в образе на момент заказа. Последующее изменение каталога (новые зависимости компонента, изменение цены) не меняет уже собранные образы, что обеспечивает воспроизводимость: содержимое артефакта всегда соответствует сохранённому составу. Для применения изменений каталога администратор изменяет образ, и он собирается повторно.

## 2.4 Очередь сборок и жизненный цикл образа

Жизненный цикл образа описан диаграммой состояний UML (см. рисунок В.2 приложения В) [3], а назначение состояний – в таблице 6.

Таблица 6 – Состояния образа

| **Состояние** | **Значение** | **Кто устанавливает** | **Допустимые действия** |
|---|---|---|---|
| QUEUED | Заказ принят и ожидает свободного воркера | ImageService при заказе или изменении | Просмотр; удаление администратором |
| BUILDING | Выполняется сценарий сборки | BuildStateService.markBuilding | Просмотр |
| READY | Сборка успешна, артефакт сохранён | BuildStateService.markReady | Просмотр, скачивание, изменение и удаление администратором |
| FAILED | Сборка завершилась ошибкой; причина – в журнале | BuildStateService.markFailed | Просмотр, изменение и удаление администратором |

Переходы QUEUED → BUILDING → READY или FAILED выполняет только сборочный воркер; переход в QUEUED из READY или FAILED – только изменение образа администратором. Изменение образа в состояниях QUEUED и BUILDING запрещено, поскольку задача уже находится в очереди: повторная постановка привела бы к двум сборкам одного образа. Удаление запрещено только в состоянии BUILDING, когда сценарий записывает файл артефакта. Удаление образа в состоянии QUEUED допустимо: при извлечении задачи воркер не находит образ и завершает её без сборки.

Очередь сборок спроектирована как пул задач Spring с ровно одним рабочим потоком и ограниченной очередью ожидающих заказов [9]. Такая очередь обрабатывает заказы строго последовательно в порядке поступления (FIFO) и гарантирует, что в каждый момент выполняется не более одной сборки. Это требование следует из устройства Luckfox SDK: выбор платы выполняется созданием ссылки .BoardConfig.mk в корне SDK, а результаты сборки записываются в общий каталог output, поэтому параллельные сборки в одной копии SDK повредили бы друг друга.

Рассматривались и альтернативы. Несколько воркеров с изолированными копиями SDK увеличили бы пропускную способность, но каждая копия занимает около 12 ГБ и требует собственной первичной сборки инструментария. Внешний брокер сообщений (например, RabbitMQ) с отдельными сборочными узлами обеспечил бы сохранность очереди и горизонтальное масштабирование, но избыточен для прототипа. Выбранный вариант не требует дополнительной инфраструктуры и соответствует ограничениям SDK; его ограничения – хранение очереди в памяти процесса и последовательная обработка – выявлены при тестировании (дефекты DEF-19 и DEF-24, см. раздел 4.7) и определяют направления развития.

Отдельно спроектирован момент постановки задачи в очередь. Если передать задачу воркеру внутри транзакции заказа, воркер может начать работу раньше фиксации транзакции и не найти ещё не сохранённый образ. Поэтому

задача регистрируется как действие, выполняемое после успешной фиксации транзакции (afterCommit); при откате транзакции сборка не запускается вовсе.

Изменения статуса выполняются короткими отдельными транзакциями вне длительной сборки. Сборка, длящаяся часами, не удерживает соединение с базой данных и не блокирует строки таблицы Image, а каждое изменение статуса сразу становится видимым клиентам, опрашивающим сервер.

## 2.5 Взаимодействие со сборочным инструментарием Luckfox

Серверная часть не обращается к Luckfox SDK напрямую. Взаимодействие выполняется через сценарий-адаптер scripts/luckfox_build.sh, запускаемый как отдельный процесс. Такое решение отделяет Java-приложение от деталей сборочного инструментария: смена версии SDK, способа выбора платы или формата результата затрагивает только сценарий. Контракт адаптера – аргументы командной строки, код возврата, журнал в стандартном выводе и файл результата по заданному пути (таблица 7).

Таблица 7 – Интерфейс сценария luckfox_build.sh

| **Элемент** | **Назначение** |
|---|---|
| --id <число> | Идентификатор образа (для журнала и манифеста) |
| --name <строка> | Название образа |
| --distro <строка> | Базовый дистрибутив |
| --board <строка> | Целевая плата, по подстроке которой выбирается BoardConfig SDK |
| --components <список> | Имена компонентов образа через запятую (отсортированы) |
| --out <путь> | Путь, по которому должен появиться файл образа |
| Код возврата 0 | Успешная сборка (при условии, что файл --out существует) |
| Код возврата, отличный от 0 | Ошибка сборки; образ получает статус FAILED |
| Стандартный вывод и ошибки | Журнал сборки, сохраняемый в образе |

Сценарий поддерживает два режима, выбираемых переменной окружения LUCKFOX_REAL_BUILD. В режиме реальной сборки (значение 1) он находит конфигурацию платы в каталоге project/cfg/BoardConfig_IPC SDK (предпочтительно вариант SD_CARD с Buildroot), создаёт ссылку .BoardConfig.mk, запускает полную сборку build.sh (U-Boot, ядро, корневая

файловая система) и упаковку прошиваемого update.img, после чего копирует результат в выходной файл [5], [6]. Режим требует развёрнутого SDK и кросс-компилятора, а первая сборка длится один–три часа.

Режим симуляции (по умолчанию) предназначен для проверки всего конвейера без многочасовой кросс-компиляции. В нём создаётся настоящий образ файловой системы ext4, содержащий манифест /etc/luckfox-image.json с параметрами образа и составом компонентов, файл /etc/issue и список пакетов /etc/installed-packages.txt. Размер образа – 16 МБ плюс 1 МБ на компонент. Симуляционный артефакт не является загрузочным образом для устройства, но структурно валиден и позволяет проверить содержимое, что используется в интеграционных тестах (см. раздел 4.5).

Настройки подсистемы сборки вынесены в переменные окружения (таблица 8), что позволяет использовать один и тот же артефакт приложения при локальном запуске, на тестовом стенде и в контейнере Docker.

Таблица 8 – Параметры подсистемы сборки

| **Параметр** | **Назначение** | **Значение по умолчанию** |
|---|---|---|
| LUCKFOX_BUILD_SCRIPT | Путь к сценарию-адаптеру | ../scripts/luckfox_build.sh |
| LUCKFOX_OUTPUT_DIR | Каталог готовых образов | ../output/images |
| LUCKFOX_BUILD_TIMEOUT | Максимальная длительность сборки, с | 1800 |
| LUCKFOX_REAL_BUILD | 1 – реальная сборка SDK, иначе симуляция | 0 |
| LUCKFOX_SDK_DIR | Каталог Luckfox SDK | luckfox-pico в корне проекта |
| LUCKFOX_SIM_DELAY | Имитация длительности сборки в симуляции, с | 0 |

Ограничение текущей версии: в реальном режиме выбранный пользователем состав передаётся в сценарий и фиксируется в журнале, но ещё не преобразуется в параметры Buildroot (переменные BR2_PACKAGE_*), поэтому SDK собирает образ со стандартным набором пакетов. Сопоставление «компонент – пакет Buildroot» спроектировано как следующий шаг развития и не требует изменения серверной части: достаточно дополнить сценарий-адаптер.

## 2.6 Хранение, получение и обработка результатов сборки

Результатом сборки являются файл образа и журнал. Файлы образов хранятся в каталоге, заданном параметром LUCKFOX_OUTPUT_DIR, под именем image-<id>.img; при развёртывании в Docker Compose каталог вынесен в именованный том build_output и сохраняется при пересоздании контейнера. Хранение больших двоичных файлов в файловой системе, а не в СУБД, уменьшает объём базы данных и позволяет отдавать файлы потоком без загрузки в память. В таблице Image сохраняются путь к артефакту и полный журнал сборки (тип TEXT), по которому администратор может определить причину ошибки.

Журнал формируется из стандартного вывода и потока ошибок сценария, объединённых в один поток. Каждая строка одновременно добавляется к журналу образа и выводится в журнал приложения с номером образа, что позволяет наблюдать ход длительной сборки по журналу сервера. При ошибке к журналу добавляется код возврата или сообщение о превышении таймаута.

Получение результата выполняется запросом GET /api/images/{id}/download, доступным только авторизованным пользователям. Перед отдачей файла проверяются статус READY, наличие пути к артефакту и существование файла на диске. Ответ имеет тип application/octet-stream и заголовок Content-Disposition с именем файла, образованным от названия образа.

Жизненный цикл файлов согласован с жизненным циклом записей. При изменении образа путь к старому артефакту и журнал очищаются, а сам файл удаляется после фиксации транзакции; при удалении образа файл также удаляется после фиксации. Удаление файла после фиксации исключает ситуацию, когда транзакция откатилась, а файл уже удалён. Ошибка удаления файла только записывается в журнал приложения и не отменяет изменений в базе данных.

Уведомление о готовности образа на этапе анализа предполагало отправку файла по электронной почте. В прототипе NotificationService записывает в журнал сообщение о готовности со ссылкой на скачивание, а пользователь видит изменение статуса в интерфейсе. Отправка писем может быть добавлена в этот сервис без изменения остальной подсистемы.

# 3 РЕАЛИЗАЦИЯ ПРОТОТИПА ПРОГРАММНОЙ СИСТЕМЫ

## 3.1 Реализация управления конфигурацией образа

Серверная часть реализована на Java 17 с использованием Spring Boot 3.2 (модули Web, Data JPA, Security, Validation), Hibernate 6 и MySQL 8 [8], [10], [11]. Подсистема сборки размещена в тех же пакетах, что и остальное приложение, в соответствии со слоями: controller, service, dto, entity, repository и config. Эндпоинты работы с образами сосредоточены в контроллере ImageController (таблица 9).

Таблица 9 – Эндпоинты подсистемы сборки образов

| **Метод и путь** | **Назначение** | **Доступ** | **Ответ** |
|---|---|---|---|
| GET /api/images | Библиотека образов: поиск, фильтры по статусу, плате и компонентам, сортировка, пагинация | Все | 200, страница ImageDto |
| GET /api/images/{id} | Образ и его статус (используется для опроса) | Все | 200 или 404 |
| POST /api/images/build | Заказ сборки | Авторизованный | 202, ImageDto в статусе QUEUED |
| PUT /api/images/{id} | Изменение образа и повторная сборка | ADMIN | 202; 400, если образ в очереди или собирается |
| DELETE /api/images/{id} | Удаление образа и артефакта | ADMIN | 204; 400, если образ собирается |
| GET /api/images/{id}/download | Скачивание файла .img | Авторизованный | 200 (поток файла) или 400 |

Для заказа и изменения образа используется один контракт BuildRequest. Его поля проверяются средствами Jakarta Bean Validation до вызова сервиса: пустое название, пустой дистрибутив или пустой список компонентов приводят к ответу 400 с сообщением по каждому полю, которое формирует глобальный обработчик ошибок GlobalExceptionHandler. Ответы на заказ и изменение имеют код 202 Accepted: он точно отражает семантику операции – запрос принят, но результат (собранный образ) будет получен позже.

Метод ImageService.orderBuild выполняется в одной транзакции. Он находит заказчика по адресу электронной почты из токена, создаёт объект Image, заполняет его общим методом applyAndQueue, сохраняет, добавляет образ в коллекцию заказанных образов пользователя (таблица user_image_order) и регистрирует запуск сборки после фиксации транзакции. Метод update для администратора сначала проверяет состояние образа, затем вызывает тот же applyAndQueue, очищает путь к артефакту и журнал и также регистрирует удаление старого файла и новую сборку после фиксации.

Метод applyAndQueue объединяет общую для заказа и изменения логику: загрузку выбранных компонентов (при отсутствии любого из них – исключение ResourceNotFoundException и ответ 404), разрешение зависимостей, проверку циклов, заполнение полей образа, подстановку платы по умолчанию RV1103_Luckfox_Pico, установку статуса QUEUED и расчёт стоимости. Благодаря общему методу правила формирования состава не могут разойтись между заказом и изменением.

### 3.2 Разрешение зависимостей компонентов и расчёт стоимости образа

Алгоритмы, спроектированные в разделе 2.3, реализованы закрытыми методами ImageService (листинг 1). Для множества result используется LinkedHashSet, который сохраняет порядок добавления и обеспечивает проверку принадлежности за O(1); в качестве очереди используется ArrayDeque. Сущность Component сравнивается по ссылке в пределах сеанса Hibernate, что корректно, так как все компоненты загружаются в одной транзакции и каждый объект в ней существует в единственном экземпляре.

Листинг 1 – Разрешение зависимостей и проверка графа на циклы (ImageService)

```
private Set<Component> resolveDependencies(Set<Component> selected) {
    Set<Component> result = new LinkedHashSet<>();
    Deque<Component> queue = new ArrayDeque<>(selected);
    while (!queue.isEmpty()) {
        Component c = queue.poll();
```

```
            if (!result.add(c)) {
                continue;
            }
            if (c.getDependencies() != null) {
                for (Component dep : c.getDependencies()) {
                    if (!result.contains(dep)) {
                        queue.add(dep);
                    }
                }
            }
        }
        return result;
    }

    private void dfs(Component c, Set<Long> visited, Set<Long> inStack) {
        visited.add(c.getId());
        inStack.add(c.getId());
        if (c.getDependencies() != null) {
            for (Component dep : c.getDependencies()) {
                if (inStack.contains(dep.getId())) {
                    throw new BadRequestException(
                        "Обнаружена циклическая зависимость с участием компонента
'"
                        + c.getName() + "'");
                }
                if (!visited.contains(dep.getId())) {
                    dfs(dep, visited, inStack);
                }
            }
        }
        inStack.remove(c.getId());
    }
```

Коллекция зависимостей компонента объявлена с ленивой загрузкой. Без дополнительных мер обход графа порождал бы отдельный SQL-запрос на каждый компонент (проблема N+1). Параметр hibernate.default_batch_fetch_size = 100, добавленный по итогам нагрузочного тестирования, заставляет Hibernate догружать коллекции пачками одним запросом с условием IN, поэтому число запросов при разрешении зависимостей растёт не с числом компонентов, а с глубиной графа.

Стоимость рассчитывается одной операцией над потоком элементов замыкания: сумма значений cost всех компонентов множества result. Поскольку result является множеством, общий компонент (например, python3, требуемый и opencv-python, и numpy) учитывается один раз. Корректность этого свойства подтверждается модульным тестом

orderResolvesTransitiveDependenciesAndSumsCostOnce, а отклонение циклического графа – тестом cyclicDependencyGraphIsRejected (см. раздел 4.2).

Проверка циклов при редактировании каталога реализована в ComponentService методом dependsOn: перед добавлением зависимости dep к компоненту c выполняется обход в ширину от dep, и если из него достижим c, изменение отклоняется с сообщением «Циклическая зависимость: «…» уже зависит от «…»». Отдельно запрещена зависимость компонента от самого себя. Удаление компонента запрещено, если он входит в какие-либо образы или от него зависят другие компоненты, поэтому удаление не может нарушить целостность состава уже собранных образов.

### 3.3 Асинхронная сборка и управление состояниями образа

Очередь сборок реализована бином buildExecutor типа ThreadPoolTaskExecutor с одним потоком и очередью на 1 000 задач (листинг 2). Метод BuildService.build помечен аннотацией @Async("buildExecutor"), поэтому его вызов через прокси Spring не выполняет сборку, а помещает задачу в очередь этого пула и сразу возвращает управление [9].

Листинг 2 – Конфигурация очереди сборок (AsyncConfig)

```
@Bean(name = "buildExecutor")
public Executor buildExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(1);
    executor.setMaxPoolSize(1);          // строго один воркер
    executor.setQueueCapacity(1000);     // ожидающие заказы
    executor.setThreadNamePrefix("build-worker-");
    executor.initialize();
    return executor;
}
```

Постановка в очередь после фиксации транзакции реализована механизмом TransactionSynchronizationManager (листинг 3). Если синхронизация транзакций активна, действие регистрируется в методе afterCommit и выполняется только после успешного коммита; вне транзакции (например, в

модульных тестах) действие выполняется сразу. Тот же механизм используется для удаления файлов артефактов.

Листинг 3 – Выполнение действия после фиксации транзакции (ImageService)

```
private void afterCommit(Runnable action) {
    if (TransactionSynchronizationManager.isSynchronizationActive()) {
        TransactionSynchronizationManager.registerSynchronization(
            new TransactionSynchronization() {
                @Override
                public void afterCommit() {
                    action.run();
                }
            });
    } else {
        action.run();
    }
}
```

Сборка одного образа выполняется методом BuildService.build в следующем порядке.

– Чтение входных данных: BuildStateService.loadInputs в транзакции только для чтения загружает образ и возвращает неизменяемый снимок BuildInputs (идентификатор, название, дистрибутив, плата и отсортированный список имён компонентов). Снимок нужен потому, что сборка идёт вне транзакции, а ленивая коллекция компонентов вне транзакции недоступна. Если образ удалён, пока ожидал в очереди, задача завершается без сборки;

– Перевод в состояние BUILDING отдельной транзакцией markBuilding;

– Формирование команды bash luckfox_build.sh с аргументами --id, --name, --distro, --board, --components и --out, где выходной файл – image-<id>.img в каталоге артефактов. Аргументы передаются списком, без интерпретации командной оболочкой, что исключает внедрение команд через название образа;

– Запуск процесса ProcessBuilder с объединением потоков вывода и ошибок и построчное чтение вывода: каждая строка добавляется к буферу журнала и в журнал приложения;

– Ожидание завершения процесса не дольше LUCKFOX_BUILD_TIMEOUT (1 800 с по умолчанию); при превышении

процесс принудительно завершается, а образ получает статус FAILED с пометкой о таймауте;

– Анализ результата: код возврата 0 и существующий файл означают успех (markReady сохраняет путь и журнал, затем вызывается NotificationService), иначе markFailed сохраняет журнал и код возврата. Любое исключение также переводит образ в FAILED, поэтому ошибка одной сборки не останавливает обработку очереди.

Операции изменения состояния вынесены в отдельный бин BuildStateService. Аннотации @Transactional и @Async в Spring работают через прокси и не срабатывают при вызове метода того же класса; кроме того, прямая зависимость BuildService от ImageService создала бы циклическую зависимость бинов (ImageService уже зависит от BuildService). Каждый метод BuildStateService – короткая транзакция, которая находит образ, изменяет статус и при наличии сохраняет путь к артефакту и журнал.

Для защиты от потерянных обновлений в сущности Image используется поле версии с аннотацией @Version (оптимистичная блокировка). Конфликт одновременных изменений приводит к исключению ObjectOptimisticLockingFailureException, которое глобальный обработчик преобразует в ответ 409.

### 3.4 Интеграция с Luckfox SDK и реализация симуляционного режима

Сценарий scripts/luckfox_build.sh написан на Bash и запускается с параметрами set -euo pipefail: любая ошибка команды, обращение к неопределённой переменной или сбой в конвейере прерывают сценарий с ненулевым кодом, который серверная часть интерпретирует как FAILED. Разбор аргументов выполнен циклом case; при неизвестном аргументе или отсутствии --out сценарий завершается с кодом 2.

Перед сборкой сценарий очищает переменную PATH от элементов, содержащих пробельные символы. Эта мера появилась при отладке реальной

сборки в WSL: в PATH подмешиваются пути Windows (например, /mnt/c/Program Files/...), а Buildroot отказывается работать при таких элементах. Для ручного запуска реальной сборки с выводом в консоль подготовлен отдельный сценарий scripts/real-build.sh, который дополнительно настраивает таймауты wget: без них загрузка исходных кодов пакетов зависала на недоступных зеркалах, а с таймаутами быстро переходила на резервный источник.

Функция real_build проверяет наличие исполняемого build.sh и исходных кодов SDK, выбирает файл конфигурации платы по подстроке имени (сначала вариант SD_CARD с Buildroot, затем любой подходящий), создаёт ссылку .BoardConfig.mk, выполняет ./build.sh и ./build.sh updateimg, после чего копирует прошиваемый update.img (или, при его отсутствии, самый свежий .img) в выходной файл. Реальная сборка была выполнена в WSL на Ubuntu 24.04: получены артефакты uboot.img, boot.img, rootfs.img и прошиваемый пакет update.img, структурная корректность которых подтверждена. Загрузка полученного образа на физическом устройстве Luckfox Pico не проверялась.

Функция sim_build формирует во временном каталоге дерево корневой файловой системы, записывает манифест, файл /etc/issue и список пакетов, после чего создаёт файл образа заданного размера и форматирует его как ext4 с заполнением содержимым каталога (листинг 4). Если утилита mkfs.ext4 недоступна, в качестве запасного варианта формируется tar-архив. Временный каталог удаляется при выходе из функции. Переменная LUCKFOX_SIM_DELAY добавляет задержку, имитирующую длительность сборки; она использовалась для наглядной демонстрации очереди и в нагрузочных тестах (3 с).

Листинг 4 – Формирование образа в режиме симуляции (luckfox_build.sh, фрагмент)

```
cat >"$rootfs/etc/luckfox-image.json" <<EOF
{
  "imageId": "${IMAGE_ID}",
  "name": "${IMAGE_NAME}",
  "distro": "${DISTRO}",
  "board": "${BOARD}",
  "builtAt": "$(date -u +%Y-%m-%dT%H:%M:%SZ)",
```

```
  "components": [${comp_json}]
}
EOF
echo "$COMPONENTS" | tr ',' '\n' | sed '/^[[:space:]]*$/d' \
    >"$rootfs/etc/installed-packages.txt"
ncomp="$(grep -c . "$rootfs/etc/installed-packages.txt" || echo 0)"
size_mb=$(( 16 + ncomp ))
dd if=/dev/zero of="$OUT" bs=1M count="$size_mb" status=none
mkfs.ext4 -q -F -L luckfox -d "$rootfs" "$OUT"
```

Формирование JSON-массива компонентов выполнено средствами самого Bash, без awk, чтобы сценарий работал в минимальном контейнерном образе. Содержимое симуляционного образа проверяется без монтирования: интеграционный тест читает сигнатуру ext4 (0xEF53) и извлекает манифест утилитой debugfs, сравнивая состав с заказанным.

Для развёртывания в Docker Compose каталог scripts и каталог SDK монтируются в контейнер backend только для чтения, а каталог артефактов – в именованный том. Пути внутри контейнера задаются переменными LUCKFOX_BUILD_SCRIPT, LUCKFOX_OUTPUT_DIR и LUCKFOX_SDK_DIR, поэтому код приложения не содержит путей, зависящих от окружения.

### 3.5 Хранение артефактов и журналов сборки

Каталог артефактов создаётся при первой сборке (Files.createDirectories), а путь к нему нормализуется до абсолютного, поэтому в базе данных хранится однозначный путь независимо от рабочего каталога процесса. Имя файла формируется только из числового идентификатора образа (image-<id>.img), а не из названия, заданного пользователем, что исключает обход каталогов через название образа.

Журнал сборки сохраняется целиком в поле build_log типа TEXT (до 64 КБ в MySQL). Для режима симуляции журнал занимает несколько строк; для реальной сборки SDK, журнал которой значительно длиннее, требуется хранить в базе данных только последние строки, а полный журнал – рядом с артефактом. Это ограничение зафиксировано как направление доработки.

Удаление файлов выполняется методом deleteArtifact после фиксации транзакции: Files.deleteIfExists не завершается ошибкой при отсутствии файла, а исключение ввода-вывода только записывается в журнал. Уведомление о готовности (NotificationService) пишет в журнал приложения сообщение с названием образа, путём к артефакту и адресом для скачивания.

### 3.6 Реализация API заказа сборки и скачивания образов

Эндпоинт скачивания реализован методом ImageController.download. Он проверяет, что образ находится в статусе READY и имеет путь к артефакту, а файл существует на диске; в противном случае возвращается 400 с поясняющим сообщением («Образ ещё не готов к загрузке», «Файл артефакта не найден на сервере»). Файл передаётся объектом FileSystemResource: Spring копирует его в ответ потоком, не загружая в память, поэтому одновременная отдача многих файлов размером 17–19 МБ не увеличивает потребление памяти (см. профиль L-08 в разделе 4.6).

Правила доступа заданы в SecurityConfig. Запросы GET /api/images и GET /api/images/{id} открыты для гостей, так как библиотека образов публична; шаблон пути с одним сегментом после /images не включает /download, поэтому скачивание, как и заказ сборки, требует аутентификации. Изменение и удаление образов дополнительно ограничены ролью ADMIN аннотацией @PreAuthorize("hasRole('ADMIN')").

Со стороны клиентской части заказ выполняется из редактора образа (рисунок 2): пользователь задаёт название, дистрибутив и плату, выбирает компоненты из каталога и нажимает «Заказать образ». Редактор показывает предварительную стоимость только выбранных компонентов (на рисунке – 180 ед. за opencv-python, ffmpeg и openssh) и предупреждает, что транзитивные зависимости добавятся автоматически при сборке.

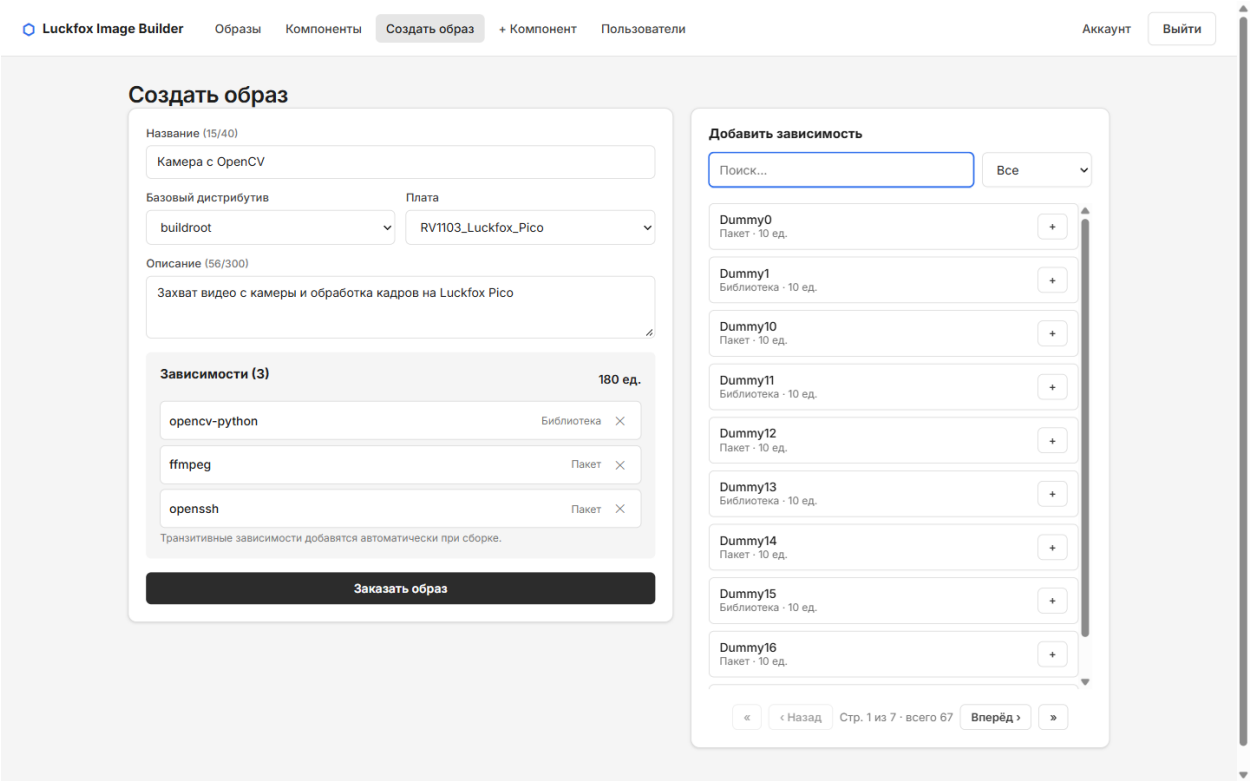



Рисунок 2 – Редактор образа: выбор компонентов перед заказом сборки

После заказа клиент переходит в библиотеку образов, где новый образ отображается со статусом «Сборка» (рисунок 3). Состав образа уже дополнен сервером зависимостями glibc, numpy и python3, а стоимость пересчитана с 180 до 280 ед.; действия «Изменить» и «Удалить» для собираемого образа недоступны. Статусы заказанных пользователем образов отображаются также в разделе «Мои образы» страницы «Аккаунт», где данные обновляются опросом сервера каждые 5 с (рисунок 4); для готового образа доступна кнопка «Скачать .img». Реализация интерфейса описана в записке второго участника проекта; здесь приведены только экраны, через которые пользователь взаимодействует с подсистемой сборки.

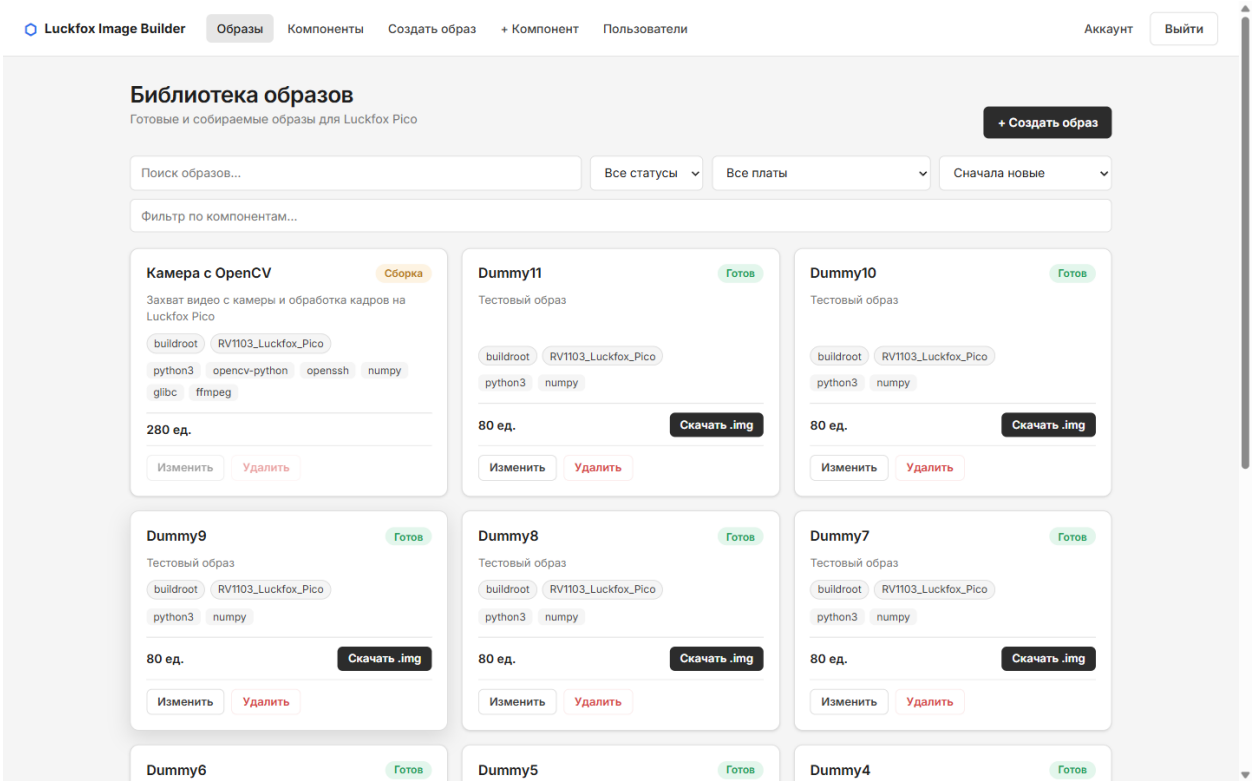



Рисунок 3 – Библиотека образов сразу после заказа: сервер добавил зависимости и пересчитал стоимость

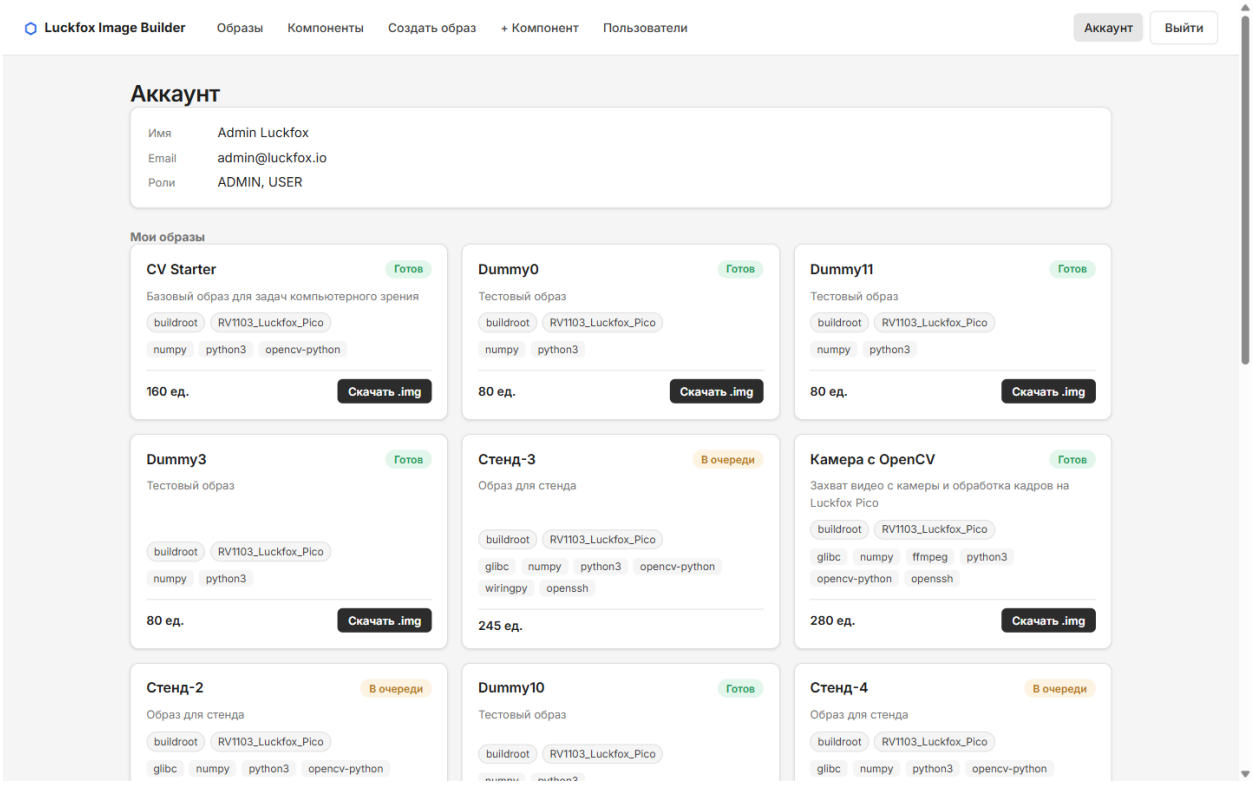



Рисунок 4 – Раздел «Мои образы»: образы в очереди и готовые к скачиванию

Работа эндпоинтов подтверждена интеграционными и сквозными тестами: заказ с разрешением зависимостей (INT-29), сборка до READY (INT-30), проверка содержимого скачанного образа (INT-31), изменение и удаление образа администратором (INT-37, INT-38), заказ и скачивание через интерфейс (E2E-04, E2E-05). Тестирование также выявило дефекты подсистемы: отсутствие проверки владельца при скачивании (DEF-16), потерю имени файла с кириллическим названием в заголовке Content-Disposition (DEF-15) и отображение кнопки скачивания при удалённом с диска файле (DEF-17). Их описание и рекомендации приведены в разделе 4.7.

# 4 ТЕСТИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ

## 4.1 Тестовый стенд, методика и критерии оценки

Тестирование прототипа выполнялось на четырёх уровнях: модульном (изолированная проверка классов серверной части), интеграционном (проверка развёрнутой системы по HTTP с реальной СУБД, файловой системой и внешним процессом сборки), сквозном (сценарии пользователя в браузере) и нагрузочном (поведение под заданной интенсивностью запросов) [23]. Для проверки эффекта оптимизации нагрузочные и интеграционные тесты выполнены дважды – до и после внесения исправлений.

Стенд развёрнут в WSL2 (Ubuntu 24.04) и повторяет топологию Docker Compose: MySQL 8.0 на порту 3306, серверная часть в виде jar-файла на порту 8080 и собранное клиентское приложение, раздаваемое сервером Node.js на порту 5173 и обращающееся к API по абсолютному адресу (с проверкой CORS). Характеристики стенда приведены в таблице 10.

Таблица 10 – Характеристики тестового стенда

| **Параметр** | **Значение** |
|---|---|
| Процессор | AMD Ryzen 7 5800HS, 16 логических ядер, выделенных WSL2 |
| Оперативная память | 7 638 МБ (лимит WSL2) |
| Операционная система | Ubuntu 24.04 LTS в WSL2, ядро 6.6.87.2; хост – Windows 11 Enterprise LTSC 2024 |
| JDK и параметры JVM | OpenJDK 17.0.20; -Xms512m -Xmx1g -XX:+UseG1GC |
| Серверная часть | Spring Boot 3.2.0, Hibernate 6, HikariCP, встроенный Tomcat (200 потоков) |
| СУБД | MySQL 8.0.46, параметры по умолчанию (innodb_buffer_pool_size 128 МБ) |
| Клиентская часть | React 18 + Vite 5, производственная сборка, Node.js 24 |
| Сборка образов | luckfox_build.sh в режиме симуляции, задержка 3 с, один воркер |
| Инструменты | JUnit 5, Mockito, AssertJ; Python 3.12 + requests; Playwright (Chromium); Grafana k6 0.54 |
| Мониторинг | CPU, RSS и потоки процессов (/proc), heap и сборка мусора (jstat), соединения и число SQL-запросов MySQL (SHOW GLOBAL STATUS) с шагом 1 с |

Интеграционные тесты построены по принципу чёрного ящика: каждый тест создаёт собственные данные (уникальные адреса и названия) и проверяет не только HTTP-ответы, но и состояние базы данных (через information_schema и счётчики) и артефакты на диске. Финальные прогоны выполнялись на полностью пересозданной базе данных, наполненной начальными данными (67 компонентов, 15 образов, собранных через тот же конвейер, администратор).

Критерии успеха нагрузочного тестирования определены на основе нефункциональных требований: доля ошибок менее 1 %, 95-й процентиль (p95) времени отклика на чтение менее 500 мс, на запись менее 1 000 мс, на вход менее 1 000 мс, на скачивание образа размером 18 МБ менее 3 000 мс; для стрессовых профилей фиксировалась точка насыщения.

## 4.2 Модульное тестирование

Модульные тесты размещены в каталоге backend/src/test/java с повторением структуры пакетов приложения и выполняются командой mvn test. Использованы JUnit 5 [14], Mockito [15] (расширение MockitoExtension со строгой проверкой заглушек) и AssertJ. Каждый тест проверяет один класс, а его зависимости (репозитории, кодировщик паролей, другие сервисы) заменены заглушками, поэтому тесты не требуют базы данных и выполняются за несколько секунд. Проверки, обращающиеся к развёрнутому API, к модульным не относятся и описаны в разделе 4.3.

Всего разработано 84 теста в 13 классах: сервисы (ImageService, BuildService, BuildStateService, ComponentService, UserService, AuthenticationService, LoginAttemptService, JwtService), DTO и постраничный вывод, валидация запросов, глобальный обработчик ошибок и сущность User. Для подсистемы сборки разработано 29 тестов (таблица 11); полный состав тестов приведён в приложении Ж.

Таблица 11 – Модульные тесты подсистемы сборки

| **Класс** | **Тестов** | **Основные проверки** |
|---|---|---|
| ImageServiceTest | 11 | Транзитивное разрешение зависимостей и однократный учёт стоимости; плата по умолчанию; привязка к заказчику и постановка в очередь; отказ при неизвестном компоненте, пользователе и циклическом графе; запрет изменения в QUEUED и BUILDING; пересборка с удалением старого артефакта; удаление с отвязкой заказчиков и файла |
| BuildServiceTest | 6 | Успешная сборка: READY, путь, журнал, уведомление; передача компонентов сценарию; FAILED при ненулевом коде и при отсутствии файла; принудительное завершение по таймауту; отказ до перехода в BUILDING для удалённого образа |
| BuildStateServiceTest | 4 | Отсортированный снимок входных данных; сохранение артефакта и журнала; сохранение журнала при переходе в BUILDING; ошибка для отсутствующего образа |
| ComponentServiceTest | 8 | Создание с типом и зависимостями; неизвестный тип и зависимость; запрет зависимости от себя и транзитивного цикла; запрет удаления используемого компонента |

Тесты BuildService запускают настоящий процесс: вместо luckfox_build.sh во временном каталоге (@TempDir) создаётся сценарий-заглушка, который печатает строки журнала, создаёт или не создаёт выходной файл, завершается с нужным кодом или «зависает». Это позволяет проверить обработку вывода, кодов возврата и таймаута без SDK; так как тесты требуют bash, они выполняются только в Linux и macOS (аннотация @EnabledOnOs). Постановка в очередь после фиксации транзакции проверяется косвенно: вне транзакции действие afterCommit выполняется сразу, и тест фиксирует вызов buildService.build с идентификатором нового образа.

Результат прогона в WSL (Ubuntu 24.04, JDK 17, Maven): выполнено 84 теста, ошибок и пропусков нет (BUILD SUCCESS). Строки уровня ERROR в выводе прогона являются ожидаемыми сообщениями сценариев неуспешной сборки.

### 4.3 Интеграционное тестирование серверной части

Интеграционные тесты реализованы сценарием на Python (tests/integration/run_integration.py) с библиотекой requests и выполняются против развёрнутого стенда. Тесты сгруппированы по этапам: развёртывание (статические файлы, CORS, схема БД, начальное наполнение), аутентификация и доступ, каталог и пагинация, компоненты, заказ и сборка образа, управление пользователями и отказоустойчивость. Всего 47 API-тестов; результаты по этапам приведены в таблице 12, полный протокол – в приложении Ж.

Таблица 12 – Результаты интеграционного и сквозного тестирования (исходный прогон)

| **Этап** | **Всего** | **Успешно** | **Выявлены отклонения** |
|---|---|---|---|
| 1. Развёртывание | 7 | 5 | 2 |
| 2. Аутентификация и доступ | 11 | 4 | 7 |
| 3. Каталог и пагинация | 5 | 2 | 3 |
| 4. Компоненты (ADMIN) | 5 | 4 | 1 |
| 5. Заказ и сборка образа | 12 | 8 | 4 |
| 6. Управление пользователями | 3 | 2 | 1 |
| 7. Отказоустойчивость | 4 | 2 | 2 |
| 8. Сквозные тесты в браузере | 9 | 7 | 2 |
| Итого | 56 | 34 | 22 |

Основные бизнес-сценарии работают корректно: регистрация и вход, каталоги с поиском, фильтрами, сортировкой и пагинацией, защита графа зависимостей, заказ, сборка и скачивание валидного образа, администрирование, очередь сборок с одним воркером, оптимистичная блокировка при одновременном редактировании (из 10 одновременных изменений компонента принято одно, остальные получили 409) и защита от гонки при одновременной регистрации одного адреса. Большинство отклонений этапа аутентификации имеют общую причину: Spring Security отвечает кодом 403 без тела там, где ожидаются 401, 400 или 503, так как не настроена точка входа аутентификации, а путь /error закрыт для анонимных запросов. Из-за этого клиентское приложение не перенаправляет пользователя на страницу входа при истёкшем токене.

### 4.4 Сквозное тестирование пользовательских сценариев

Сквозные тесты выполнены средствами Playwright [17] в браузере Chromium с хоста Windows против того же стенда. Проверены девять сценариев: открытие главной страницы с перенаправлением в библиотеку и отрисовкой карточек (первая карточка – через 412 мс), регистрация через форму, фильтр по компоненту с сортировкой по цене, заказ сборки в редакторе, скачивание образа из интерфейса, поведение при недействительном токене, поиск пользователя администратором, сообщение при неверном пароле и отсутствие необработанных ошибок JavaScript за весь сценарий.

Ключевой для подсистемы сборки сценарий E2E-04 воспроизводит полный путь пользователя: в редакторе образа задаётся название, в каталоге находится и добавляется компонент ffmpeg, выполняется заказ, после чего в разделе «Мои образы» образ доходит до статуса «Готов», а зависимость glibc оказывается добавленной автоматически. Сценарий E2E-05 нажимает «Скачать .img» и проверяет, что браузер сохраняет файл «E2E образ.img» размером 18 874 368 байт, что соответствует 16 МБ базового образа и 2 МБ на два компонента.

Два сценария выявили отклонения, связанные с упомянутой обработкой ошибок безопасности: при подменённом токене пользователь остаётся на странице аккаунта с сообщением «Request failed with status code 403» вместо перенаправления на вход (DEF-04), а при неверном пароле показывается техническое сообщение вместо «Неверный email или пароль» (DEF-03).

### 4.5 Проверка конвейера сборки и отказоустойчивости

Проверки конвейера сборки входят в этапы 5 и 7 интеграционного тестирования и приведены в таблице 13.

Таблица 13 – Проверки конвейера сборки и отказоустойчивости

| **Тест** | **Проверка** | **Результат** |
|---|---|---|
| INT-07 | Начальное наполнение: 15 старто-вых образов проходят реальную | PASS: 15 из 15 READY |

| Тест | Проверка | Результат |
|---|---|---|
| | очередь сборки | |
| INT-25 | Зависимость компонента от себя и цикл A → B → A | PASS: 400 в обоих случаях |
| INT-29 | Заказ [opencv-python] с транзитивными зависимостями | PASS: 202, QUEUED, состав {numpy, opencv-python, python3}, стоимость 160 |
| INT-30 | Сборка до READY и появление в «Мои образы» | PASS: READY за 2,1 с |
| INT-31 | Содержимое скачанного образа | PASS: ext4 (сигнатура 0xEF53), 19 МБ, манифест с составом |
| INT-36 | Очередь: 6 последовательных и 6 одновременных заказов | PASS: не более одной сборки одновременно, порядок FIFO, 12 сборок за 35,5 с |
| INT-37 | Изменение образа администратором | PASS: пересборка за 3,6 с, новый состав; изменение во время сборки – 400 |
| INT-38 | Удаление образа | PASS: запись, связь и файл удалены |
| INT-40 | Файл артефакта удалён с диска вне приложения | FAIL: кнопка скачивания остаётся, скачивание – 400 (DEF-17) |
| INT-44 | Штатный перезапуск серверной части | PASS: данные сохранены, дублей нет, токен действителен |
| INT-45 | Аварийное завершение (kill -9) во время сборки | FAIL: образы остались в BUILDING и QUEUED и не обрабатываются (DEF-19) |
| INT-46 | Недоступность MySQL | FAIL: ответ 403 через 28,3 с (DEF-20); после запуска СУБД работа восстановилась без перезапуска |

Важно разделять два свойства, проверенных тестами INT-44 и INT-45. Сохранность данных при перезапуске обеспечена: все заказы, образы и артефакты хранятся в MySQL и файловой системе. Восстановление очереди не обеспечено: очередь существует только в памяти процесса, поэтому после аварийного завершения образ, находившийся в сборке, навсегда остаётся в статусе BUILDING, а ожидавшие заказы – в QUEUED, и их нельзя ни изменить, ни (для BUILDING) удалить. Для исправления предложено при запуске приложения переводить образы из BUILDING в FAILED и повторно ставить в очередь образы в статусе QUEUED либо хранить очередь в базе данных.

Проверка артефакта в режиме симуляции подтверждает корректность конвейера (передача состава, формирование файла, сохранение пути, выдача), но не является доказательством загрузки операционной системы на устройстве: для этого требуется реальная сборка SDK и прошивка физического Luckfox Pico.

4.6 Нагрузочное тестирование и анализ производительности

Нагрузочное тестирование выполнено инструментом Grafana k6 [16]. Сценарии описаны на JavaScript (tests/load/scenarios.js); использовались модель закрытой нагрузки (виртуальные пользователи с паузами «на чтение») и модель открытой нагрузки (заданная интенсивность запросов). Операции размечены тегами, что позволило получать метрики отдельно для каждого эндпоинта и для чтения и записи. Разработано 12 профилей (таблица 14).

Таблица 14 – Профили нагрузочного тестирования

| **ID** | **Профиль** | **Нагрузка** | **Результат до оптимизации** |
|---|---|---|---|
| L-01 | Дымовой | 1 пользователь, 1 мин, все пользовательские пути | 0 ошибок, p95 111 мс |
| L-02 | Просмотр каталогов | до 50 гостей | p95 15 мс |
| L-03 | Смешанный (целевой) | 100 пользователей: 70 гостей, 20 пользователей, 8 заказчиков, 2 администратора | 0 ошибок из 22 489, p95 17,5 мс |
| L-04 | Стресс | ступени 50–600 итераций/с | до 755 запросов/с без деградации |
| L-04b | Поиск предела | ступени 1 000–4 000 итераций/с | предел ≈1 300–1 470 запросов/с |
| L-05 | Всплеск | 10 → 400 пользователей за 10 с | без ошибок и роста p95 |
| L-06 | Шторм входов | 10–100 входов/с | на 100/с p95 1 050 мс |
| L-07 | Пакет заказов | 60 одновременных заказов | все 60 собраны, последний ждал 189 с |
| L-08 | Скачивание | 20 потоков, 2 мин | 361 МБ/с, p95 104 мс |
| L-09 | Длительный | 40 пользователей, 15 мин | без утечек и роста задержки |
| L-10a/b | Объёмный | 40 запросов/с на обычной БД и на БД ×100 | p95 9 → 28 мс |

Во всех профилях (около 879 тыс. запросов) не зафиксировано ни одной HTTP-ошибки. Целевой профиль L-03 выполнен с многократным запасом: p95 чтения 17,5 мс, записи менее 60 мс при загрузке машины около 5 %. Длительный профиль L-09 (23 230 запросов, 173 сборки) не выявил утечек памяти (heap стабильно около 200 МБ) и дрейфа задержки.

Для подсистемы сборки наиболее показательны профили L-07 и L-08. В L-07 60 пользователей одновременно заказали сборку: все заказы приняты (p95

ответа на заказ 277 мс), все 60 образов собраны без потерь и сбоев, однако последний заказ ожидал 189 с при длительности одной сборки около 3,2 с – очередь строго последовательна. В смешанном профиле среднее время от заказа до READY составило 12,5 с, то есть ожидание в очереди в 3–4 раза дольше самой сборки. При реальной сборке SDK длительностью 1–3 ч один воркер обеспечит лишь 8–24 образа в сутки, а очередь из 60 заказов будет обрабатываться 2,5–7,5 суток; пользователь при этом не видит свою позицию в очереди (DEF-24). В L-08 отдача файлов потоком обеспечила 361 МБ/с при 20 одновременных скачиваниях без роста потребления памяти.

Узким местом чтения оказалась база данных. В профиле L-04b пропускная способность перестала расти на уровне 1 300–1 470 запросов/с; при большей интенсивности система не отказывала, а накапливала запросы в очередях Tomcat и пула соединений, и медиана задержки выросла до 2–3 с. Мониторинг показал около 12,5 SQL-запроса на один HTTP-запрос (проблема N+1 при загрузке составов образов и зависимостей) и исчерпание пула из 10 соединений. В профиле входа L-06 при 100 входах в секунду медиана выросла в 11 раз (с 80 до 912 мс). Объёмный тест L-10b на базе, увеличенной в 100 раз (20 388 образов, 2 071 компонент), показал рост p95 в 3 раза, а страница из 100 образов порождала 389 SQL-запросов.

### 4.7 Анализ выявленных дефектов и оптимизация системы

По результатам тестирования составлен реестр из 22 дефектов (полный реестр – в приложении Ж): 1 критический, 4 высокой, 9 средней и 8 низкой серьёзности. Критический дефект DEF-07 – ключ подписи JWT, записанный в исходном коде и хранящийся в репозитории: зная его, можно выпустить токен администратора. Дефекты высокой серьёзности: отсутствие проверки владельца при скачивании образа (DEF-16), потеря очереди сборок после аварийного перезапуска (DEF-19), необработанные ошибки разбора JWT (DEF-05) и

отсутствие ограничения попыток входа (DEF-21). К подсистеме сборки относятся DEF-15, DEF-16, DEF-17, DEF-19 и DEF-24.

По итогам нагрузочного тестирования выполнена оптимизация, затрагивающая производительность и устойчивость (таблица 15).

Таблица 15 – Изменения, выполненные по итогам нагрузочного тестирования

| **Изменение** | **Дефекты** |
|---|---|
| Пакетная догрузка ленивых связей (hibernate.default_batch_fetch_size = 100): составы образов, зависимости и типы компонентов загружаются одним запросом IN на страницу | DEF-12, DEF-22 |
| Отключение Open Session In View: соединение с БД не удерживается до конца HTTP-запроса, в том числе на время проверки пароля BCrypt | DEF-21, DEF-22 |
| Загрузка пользователя вместе с ролями одним запросом (@EntityGraph) в фильтре JWT | DEF-22 |
| Соединения пула без autocommit: исключены команды SET autocommit вокруг каждой транзакции | DEF-22 |
| Пул HikariCP увеличен с 10 до 20 соединений; таймаут получения соединения – 3 с с ответом 503 и заголовком Retry-After | DEF-20, DEF-22 |
| Индексы Image(state, cost), Image(cost), Component(name, id), Component(type, cost) | DEF-23 |
| Ограничение неудачных входов: 5 на адрес и 20 на IP за 5 мин, сверх лимита – ответ 429 без проверки BCrypt | DEF-21 |
| 84 модульных теста на сервисы, DTO, валидацию и обработку ошибок | – |

После изменений те же профили нагрузки повторены на том же стенде и на тех же данных. Сравнение ключевых метрик приведено в таблице 16, изменение пропускной способности и задержки на ступенях профиля L-04b – на рисунке 5, время входа по ступеням профиля L-06 – на рисунке 6.

Таблица 16 – Ключевые метрики до и после оптимизации

| **Метрика** | **До** | **После** | **Изменение** |
|---|---|---|---|
| Предел пропускной способности чтения (L-04b), запросов/с | 1 468 | 3 830 | ×2,6 |
| Медиана задержки на ступени 2 000 итераций/с (L-04b), мс | 2 692 | 3,4 | в 790 раз |
| Пропущенные итерации k6 (L-04b), тыс. | 526 | 81 | −85 % |
| p95 стресса до 780 запросов/с (L-04), мс | 15,0 | 6,9 | −54 % |
| SQL-запросов в секунду в L-04 | 4 291 | 1 970 | −54 % |
| CPU mysqld / backend в L-04, % | 97 / 80 | 56 / 56 | −42 % / −30 % |
| p95 целевого профиля (L-03), мс | 17,5 | 5,6 | −68 % |
| p95 входа на 100 входах/с (L-06), мс | 1 050 | 127 | −88 % |
| SQL-запросов на страницу из 100 образов (БД ×100) | 389 | 11 | −97 % |

| Метрика | До | После | Изменение |
|---|---|---|---|
| p95 страницы из 100 образов (БД ×100), мс | 102,8 | 11,8 | −89 % |
| p95 объёмного профиля (L-10b), мс | 27,7 | 29,8 | без улучшения |

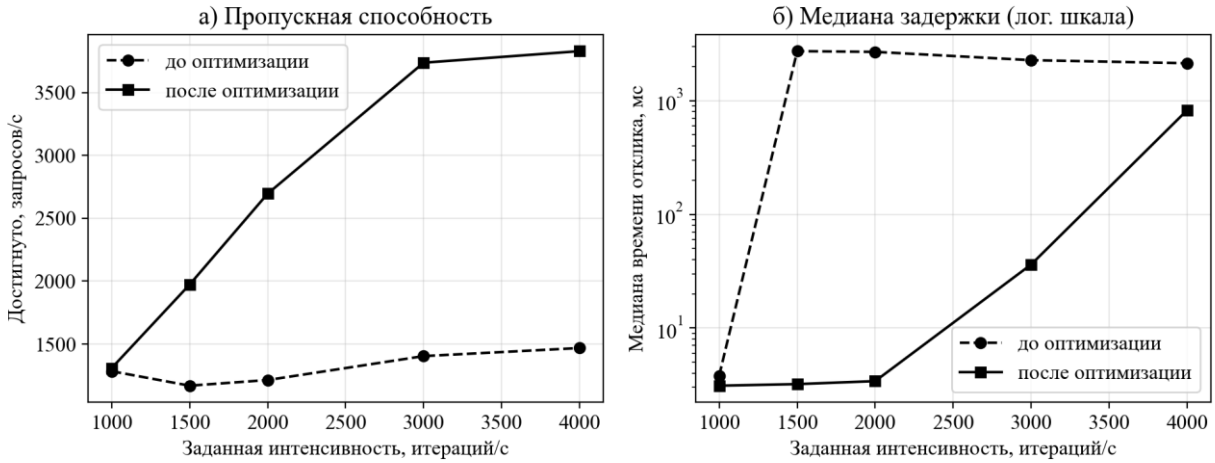



Рисунок 5 – Профиль L-04b до и после оптимизации: а – достигнутая пропускная способность, б – медиана времени отклика

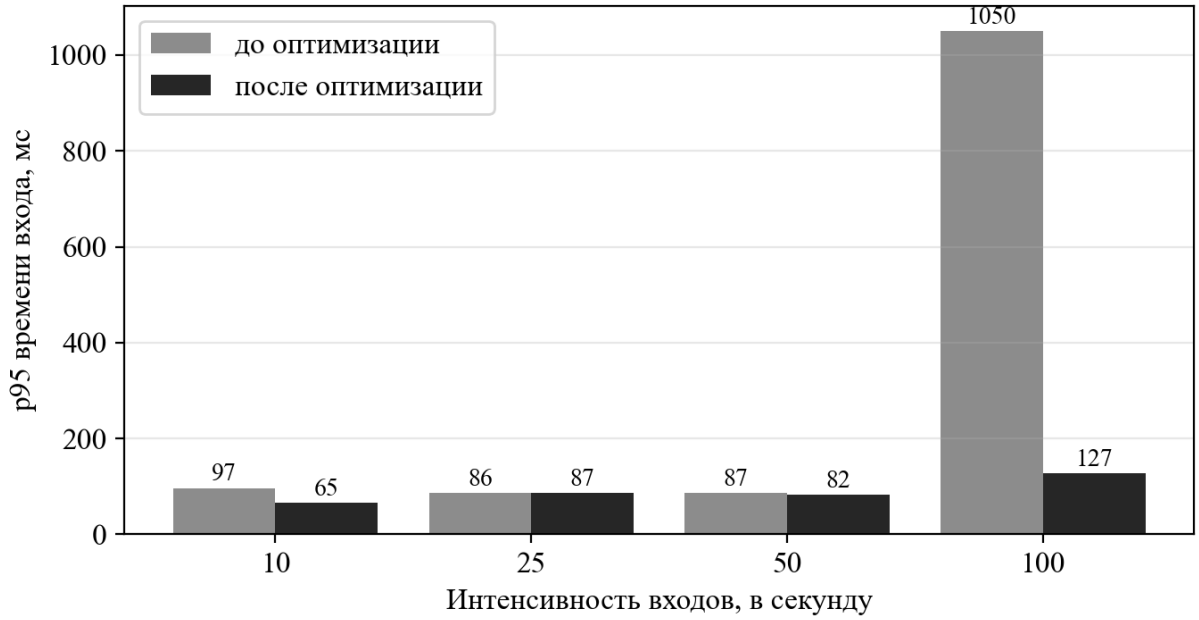



Рисунок 6 – p95 времени входа по ступеням профиля L-06 до и после оптимизации

Главный эффект дало устранение проблемы N+1 и удержания соединения: число SQL-запросов на страницу больше не зависит от её размера (9–11 запросов на любую страницу), на одинаковой нагрузке ресурсов требуется вдвое меньше,

а новый предел пропускной способности определяется процессором машины, а не пулом соединений. Причиной «колена» при входе оказалось не само хеширование BCrypt, а режим Open Session In View: соединение с базой данных удерживалось всё время проверки пароля, и 100 входов в секунду исчерпывали пул из 10 соединений.

Не улучшился p95 объёмного профиля под нагрузкой (27,7 → 29,8 мс, в пределах разброса), хотя число SQL-запросов снизилось на 66 %. На 20 тыс. образов время ответа определяют поиск по подстроке LIKE '%…%' и подсчёт общего числа строк для пагинации – полный проход таблицы, который обычными индексами не ускоряется; поэтому DEF-23 исправлен частично, а дальнейшее улучшение требует полнотекстового индекса и пагинации по ключу.

Регрессий не выявлено. Повторный интеграционный прогон на чистой базе данных дал 30 успешных тестов из 48: результаты INT-23 (число SQL-запросов на страницу) и INT-46 (недоступность БД: теперь 503 за 3,0 с) изменились с FAIL на PASS, новый тест INT-48 (защита от перебора паролей) пройден, остальные результаты совпали с первым прогоном. Ошибок ленивой инициализации после отключения Open Session In View не возникло, поскольку все DTO формируются внутри транзакций сервисов. Итог: исправлены DEF-12, DEF-20, DEF-21 и DEF-22, частично – DEF-23.

Не исправлялись дефекты безопасности и обработки ошибок (DEF-03, DEF-04, DEF-05, DEF-07, DEF-16) и архитектурные ограничения очереди сборок (DEF-19, DEF-24): они выходят за рамки оптимизации производительности. Для них в реестре зафиксированы рекомендации: чтение ключа JWT из переменной окружения, точка входа аутентификации с ответом 401, проверка владельца образа при скачивании, восстановление очереди при запуске, лимит активных заказов на пользователя с отображением позиции в очереди и несколько воркеров с изолированными копиями SDK.

4.8 Оценка результатов и ограничения тестирования

Проведённое тестирование подтвердило работоспособность основных сценариев системы и корректность подсистемы сборки: разрешение зависимостей, защита от циклов, последовательная очередь без потерь заказов, формирование валидного артефакта и его выдача. Целевая нагрузка выполняется с многократным запасом, а после оптимизации предел пропускной способности чтения вырос в 2,6 раза.

При интерпретации результатов необходимо учитывать ограничения:

– сборка образов в нагрузочных и интеграционных тестах выполнялась в режиме симуляции (около 3 с) вместо реальной сборки SDK длительностью 1–3 ч, поэтому оценки очереди для реального режима получены экстраполяцией;

– генератор нагрузки, серверная часть и СУБД работали на одном компьютере и делили процессор, поэтому абсолютные пределы занижены относительно выделенного сервера (в профиле L-04b k6 достиг предела в 3 000 виртуальных пользователей);

– сеть – петлевой интерфейс без сетевых задержек и ограничений пропускной способности;

– MySQL работал с параметрами по умолчанию;

– проверка ext4-артефакта подтверждает корректность конвейера, но не загрузку операционной системы на физическом Luckfox Pico, которая не проверялась;

– автоматизированные тесты клиентской части ограничены сквозными сценариями; модульных тестов компонентов React нет.

# 5 ОРГАНИЗАЦИЯ РАБОТЫ НАД ПРОЕКТОМ

## 5.1 Распределение задач и зон ответственности

Разработка прототипа системы автоматизированной сборки Linux-образов для микрокомпьютеров Luckfox Pico выполнялась командой из двух человек: Поздняковым Н.Э. и Ждановым А.И. Распределение работ строилось по функциональным подсистемам, что позволило вести параллельную разработку серверной и клиентской частей с минимальным числом пересечений в коде.

За Поздняковым Н.Э. закреплены подсистемы конфигурирования и сборки образов: серверная обработка состава образа, разрешение транзитивных зависимостей компонентов и проверка графа зависимостей на циклы, расчёт стоимости образа, очередь сборок (BuildService, AsyncConfig, BuildStateService) с единственным фоновым воркером, интеграция с Luckfox SDK, сценарии luckfox_build.sh и real-build.sh, режим симуляции сборки, хранение и выдача артефактов, а также программный интерфейс заказа и скачивания образов. Помимо этого, в его зону ответственности вошли разработка проверок и тестирование: модульные, интеграционные, сквозные (E2E) и нагрузочные тесты, а также оптимизация обращений к базе данных по результатам измерений.

За Ждановым А.И. закреплены пользовательский интерфейс на React, проектирование и наполнение базы данных, управление пользователями и ролями, аутентификация на основе JWT, каталоги компонентов и образов, административные функции (редактирование компонентов и образов, управление пользователями) и общая инфраструктура проекта (конфигурация Spring Boot, docker-compose, сборка frontend).

Общие для обеих частей решения принимались и фиксировались совместно. К ним относятся формат запроса на сборку BuildRequest и представление образа ImageDto, набор статусов образа (QUEUED, BUILDING, READY, FAILED), состав REST-интерфейса, права доступа ролей USER и ADMIN, а также структура таблиц базы данных. Согласование общих

контрактов до начала реализации исключало расхождения между серверной частью и интерфейсом. Итоговое распределение приведено в таблице 17.

Таблица 17 – Распределение зон ответственности

| **Направление** | **Поздняков Н.Э.** | **Жданов А.И.** | **Совместно** |
|---|---|---|---|
| Конфигурирование образа | Разрешение зависимостей, проверка циклов, расчёт стоимости | Редактор образа и форма выбора компонентов в интерфейсе | Формат BuildRequest и ImageDto |
| Сборка образов | Очередь сборок, скрипты luckfox_build.sh и real-build.sh, режим симуляции, интеграция с SDK | Отображение статусов и автообновление библиотеки образов | Набор статусов образа |
| Артефакты и заказ | API заказа и скачивания, хранение и удаление .img-файлов | Кнопка скачивания и обработка ошибок в интерфейсе | Права доступа к операциям |
| Пользователи и безопасность | Ограничение доступа к API заказа, ограничение попыток входа (по итогам нагрузочного тестирования) | Роли, JWT-аутентификация, управление пользователями | Модель ролей USER и ADMIN |
| База данных | Проверка целостности и оптимизация запросов по результатам тестов | Проектирование схемы, сущности и репозитории | Схема таблиц образов и компонентов |
| Каталоги и администрирование | Проверка правил удаления компонентов и образов | Библиотеки, поиск, фильтры, пагинация, редакторы для администратора | Правила удаления связанных записей |
| Тестирование | Модульные, интеграционные, E2E- и нагрузочные тесты, отчёт | Проверка интерфейса в ходе разработки | Анализ найденных дефектов |
| Документация | Разделы о сборке, тестировании и оптимизации | Разделы об интерфейсе, базе данных и ролях | Введение, анализ предметной области, заключение |

## 5.2 Организация совместной разработки и контроль версий

Для контроля версий использовалась система Git. Общий репозиторий создан на платформе GitHub средствами GitHub Classroom, что обеспечивает преподавателю доступ к истории изменений. Разработка велась в основной ветке main, изменения передавались в общий репозиторий небольшими законченными порциями.

Структура каталогов репозитория отражает разделение подсистем. Каталог backend содержит серверное приложение на Spring Boot: контроллеры, сервисы, репозитории, сущности и объекты передачи данных. Каталог frontend содержит клиентское приложение на React и Vite: страницы, компоненты, обращения к API, контекст аутентификации и хранилище состояния. В каталоге scripts размещены вспомогательные сценарии сборки, включая luckfox_build.sh. Каталог DOCs включает материалы по этапам проекта, диаграммы, описание базы данных, макеты интерфейса, техническую документацию и результаты тестирования. Каталог tests содержит сценарии интеграционных и нагрузочных испытаний и средства формирования отчёта. В корне репозитория расположены файл docker-compose.yml и краткое описание проекта README.md.

Не все файлы, необходимые для работы системы, хранятся в Git. Правила исключения заданы в файле .gitignore. В репозиторий не включаются:

– внешний SDK luckfox-pico (около 12 ГБ), который имеет собственную историю версий и загружается отдельно из репозитория производителя;

– файлы .env с настройками окружения и учётными данными;

– результаты сборки и служебные каталоги (target, node_modules, dist, output);

– сгенерированные образы с расширением .img.

Такой подход сохраняет репозиторий компактным, исключает попадание секретов в общий доступ и не допускает подмену исходных текстов производными файлами.

Сообщения коммитов формулировались кратко и описательно, в повелительном наклонении, например «Add component filter and price sorting to libraries» или «Add unit, integration and load tests; optimize DB access». Первые коммиты этапов проектирования, оформлявшиеся на русском языке, содержат названия этапов («Второй этап + диограммы», «3-4 этапы»). Каждый коммит

объединяет изменения одной логической задачи, что упрощает просмотр истории и поиск причин дефектов.

### 5.3 Календарный план выполнения работ

Работа над проектом была разбита на этапы в соответствии с ходом курса. Фактические сроки определены по истории коммитов репозитория и датам материалов этапов. Календарный план приведён в таблице 18.

Таблица 18 – Календарный план

| **Этап** | **Содержание работ** | **Сроки** | **Результат** |
|---|---|---|---|
| 1. Концепция и прототип | Формулировка идеи, обоснование актуальности, эскиз прототипа системы | Ноябрь 2025 | Презентация этапа 1 (каталог DOCs/этап1) |
| 2. Анализ и проектирование | Анализ предметной области, диаграмма прецедентов, модели BPMN, диаграммы классов и состояний | Декабрь 2025 | Материалы этапа 2, диаграммы |
| 3–4. Интерфейс и данные | Макеты интерфейса, схема навигации, проектирование схемы базы данных | Декабрь 2025 | Макеты интерфейса, схема БД, описание базы данных |
| Реализация | Backend с аутентификацией, очередь сборок и разрешение зависимостей, тулчейн Luckfox, новый frontend, docker-compose | Июнь 2026 | Работающий прототип системы |
| Доработки | Исправление найденных замечаний, пагинация, фильтры и сортировка, администрирование и управление пользователями | Сентябрь 2026 | Расширенная версия прототипа |
| Тестирование и оптимизация | 84 модульных теста, интеграционные и нагрузочные испытания, оптимизация доступа к БД, отчёт о тестировании | Сентябрь 2026 | Отчёт тестирования, наборы тестов |
| Оформление ПЗ | Подготовка пояснительной записки и приложений | Сентябрь 2026 | Пояснительная записка |

Как следует из таблицы, этапы анализа и проектирования были выполнены в срок, в ноябре и декабре 2025 года. Основная реализация системы была зафиксирована в репозитории только в июне 2026 года, а доработки, тестирование и оформление пояснительной записки перенесены на сентябрь 2026 года. Таким образом, фактические сроки реализации сместились относительно первоначального плана; на объём и содержание итоговых

результатов это не повлияло, однако сжало время, отводимое на тестирование и оптимизацию.

### 5.4 Использование инструментов искусственного интеллекта

При выполнении работы применялся инструмент Claude Code – агент на основе больших языковых моделей Anthropic, работающий в среде командной строки с доступом к файлам проекта. Инструмент применялся в следующих сценариях:

– анализ кодовой базы: получение обзора структуры проекта и связей между слоями приложения (контроллер, сервис, репозиторий) перед внесением изменений;

– поиск причин ошибок: разбор сообщений об ошибках и трассировок, выдвижение гипотез о причине дефекта и указание участков кода, требующих проверки;

– подготовка заготовок модульных тестов и сценариев нагрузочного тестирования на k6, которые затем дорабатывались и уточнялись вручную;

– автоматизация прогонов на стенде: запуск наборов тестов и нагрузочных сценариев, сбор и агрегирование результатов измерений;

– подготовка черновиков документации: технической документации проекта и отчёта о тестировании.

Все результаты, полученные с помощью инструмента, проверялись вручную. Корректность кода подтверждалась запуском тестов (команда mvn test, 84 теста из 84 пройдены), а результаты измерений – повторными прогонами сценариев. Предложенные исправления принимались только после просмотра изменений и подтверждения их работоспособности.

Использование инструмента имело ограничения. Проектные решения (состав подсистем, архитектура, структура данных, правила предметной области) принимались разработчиками; инструмент использовался как вспомогательное средство. Предлагаемые инструментом решения и тексты

могли содержать ошибки и неточности: например, неверные предположения о поведении кода или несоответствие описания фактической реализации, поэтому каждое утверждение документации сопоставлялось с исходным кодом. Приёмка результатов без верификации не допускалась; ответственность за содержание работы, включая код, тесты и текст пояснительной записки, несут авторы проекта.

# ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработан прототип распределённой программной системы Luckfox Image Builder для автоматизированной сборки Linux-образов для микрокомпьютеров Luckfox Pico. Система позволяет выбрать готовый образ из библиотеки или сконфигурировать собственный из каталога компонентов, заказать удалённую сборку и получить готовый файл образа без развёртывания сборочной среды на компьютере пользователя.

В рамках личного вклада автора спроектирована и реализована подсистема конфигурирования и сборки образов:

– реализован приём конфигурации образа с проверкой данных и единым контрактом для заказа и изменения образа;

– реализовано разрешение транзитивных зависимостей компонентов обходом графа в ширину, проверка графа на циклы обходом в глубину и расчёт стоимости с однократным учётом каждого компонента;

– реализована очередь сборок с одним воркером, обеспечивающая последовательную обработку заказов в порядке поступления, и постановка задачи в очередь только после фиксации транзакции;

– реализовано управление жизненным циклом образа (QUEUED, BUILDING, READY, FAILED) короткими транзакциями, не удерживающими соединение с базой данных на время сборки;

– разработан сценарий-адаптер к Luckfox SDK с режимами реальной сборки и симуляции, формирующей валидный ext4-образ с манифестом состава, а также решены проблемы реальной сборки в WSL;

– реализованы хранение артефактов и журналов сборки и API скачивания готового образа.

Также разработан и выполнен комплекс проверок: 84 модульных теста, 48 интеграционных тестов API, 9 сквозных сценариев в браузере и 12 профилей

нагрузочного тестирования. Тестирование подтвердило работоспособность основных сценариев и выявило 22 дефекта. По итогам нагрузочного тестирования выполнена оптимизация доступа к базе данных, которая увеличила предел пропускной способности чтения в 2,6 раза (с 1 468 до 3 830 запросов в секунду), снизила p95 целевого профиля с 17,5 до 5,6 мс и время входа под нагрузкой с 1 050 до 127 мс; регрессий повторный прогон не выявил.

Не реализованы или реализованы частично: сопоставление компонентов с пакетами Buildroot при реальной сборке (реальный режим собирает стандартный набор пакетов), доставка образа по электронной почте и оплата (заменены скачиванием файла), восстановление очереди после аварийного перезапуска, проверка владельца при скачивании и загрузка образа на физическом устройстве.

Дальнейшее развитие подсистемы сборки предполагает генерацию конфигурации Buildroot по составу образа, хранение очереди в базе данных с восстановлением при запуске, отображение позиции в очереди и ожидаемого времени, несколько сборочных узлов с изолированными копиями SDK, а также проверку образов на физических устройствах Luckfox Pico.

В процессе работы закреплены знания в области проектирования распределённых программных систем, моделирования на языке UML, разработки на платформе Spring Boot, асинхронной обработки длительных задач, а также методик модульного, интеграционного и нагрузочного тестирования.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017. – 32 с.

2. Фаулер М. UML. Основы. Краткое руководство по стандартному языку объектного моделирования. – 3-е изд. – СПб.: Символ-Плюс, 2004. – 192 с.

3. Буч Г., Рамбо Дж., Якобсон И. Язык UML. Руководство пользователя. – 2-е изд. – М.: ДМК Пресс, 2006. – 496 с.

4. Luckfox Pico Wiki [Электронный ресурс]. – Режим доступа: https://wiki.luckfox.com/Luckfox-Pico/Luckfox-Pico-quick-start (дата обращения: 20.09.2026).

5. LuckfoxTECH/luckfox-pico: Luckfox Pico SDK [Электронный ресурс]. – Режим доступа: https://github.com/LuckfoxTECH/luckfox-pico (дата обращения: 20.09.2026).

6. The Buildroot user manual [Электронный ресурс]. – Режим доступа: https://buildroot.org/downloads/manual/manual.html (дата обращения: 20.09.2026).

7. Yocto Project Documentation [Электронный ресурс]. – Режим доступа: https://docs.yoctoproject.org/ (дата обращения: 21.09.2026).

8. Spring Boot Reference Documentation [Электронный ресурс]. – Режим доступа: https://docs.spring.io/spring-boot/ (дата обращения: 22.09.2026).

9. Spring Framework Reference: Task Execution and Scheduling [Электронный ресурс]. – Режим доступа: https://docs.spring.io/spring-framework/reference/integration/scheduling.html (дата обращения: 22.09.2026).

10. Hibernate ORM User Guide [Электронный ресурс]. – Режим доступа: https://docs.jboss.org/hibernate/orm/6.4/userguide/html_single/Hibernate_User_Guide.html (дата обращения: 28.09.2026).

11. MySQL 8.0 Reference Manual [Электронный ресурс]. – Режим доступа: https://dev.mysql.com/doc/refman/8.0/en/ (дата обращения: 28.09.2026).

12. React Documentation [Электронный ресурс]. – Режим доступа: https://react.dev/ (дата обращения: 22.09.2026).

13. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2013. – 1328 с.

14. JUnit 5 User Guide [Электронный ресурс]. – Режим доступа: https://junit.org/junit5/docs/current/user-guide/ (дата обращения: 29.09.2026).

15. Mockito Documentation [Электронный ресурс]. – Режим доступа: https://site.mockito.org/ (дата обращения: 29.09.2026).

16. Grafana k6 Documentation [Электронный ресурс]. – Режим доступа: https://grafana.com/docs/k6/latest/ (дата обращения: 29.09.2026).

17. Playwright Documentation [Электронный ресурс]. – Режим доступа: https://playwright.dev/docs/intro (дата обращения: 29.09.2026).

18. Docker Compose Documentation [Электронный ресурс]. – Режим доступа: https://docs.docker.com/compose/ (дата обращения: 25.09.2026).

19. Armbian Documentation: Build Framework [Электронный ресурс]. – Режим доступа: https://docs.armbian.com/Developer-Guide_Build-Preparation/ (дата обращения: 21.09.2026).

20. Raspberry Pi Imager [Электронный ресурс]. – Режим доступа: https://www.raspberrypi.com/software/ (дата обращения: 21.09.2026).

21. OpenWrt Firmware Selector [Электронный ресурс]. – Режим доступа: https://firmware-selector.openwrt.org/ (дата обращения: 21.09.2026).

22. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. – СПб.: Питер, 2018. – 352 с.

23. Майерс Г., Баджетт Т., Сандлер К. Искусство тестирования программ. – 3-е изд. – М.: Диалектика, 2012. – 272 с.

ПРИЛОЖЕНИЕ А

РАСШИРЕННЫЕ ОПИСАНИЯ ПРЕЦЕДЕНТОВ

Прецедент: «Заказать сборку образа»

Действующее лицо: пользователь (роль USER или ADMIN);

Предусловие: пользователь авторизован в системе, в каталоге существует хотя бы один компонент;

Основной поток: «Заказать сборку образа». 1. Пользователь выбирает пункт «Создать образ» в главном меню; 2. Система отображает редактор образа с полями «Название», «Базовый дистрибутив», «Плата», «Описание» и каталогом компонентов с поиском и фильтром по типу; 3. Пользователь вводит название образа, выбирает дистрибутив (buildroot, ubuntu или busybox) и плату и при необходимости вводит описание; 4. Пользователь добавляет нужные компоненты из каталога кнопкой «+», система отображает выбранные компоненты в списке «Зависимости» и суммарную стоимость в условных единицах; 5. Пользователь нажимает кнопку «Заказать образ»; 6. Система проверяет данные, находит выбранные компоненты и добавляет к ним все транзитивные зависимости, проверяет граф зависимостей на циклы, рассчитывает стоимость как сумму стоимостей всех компонентов состава, сохраняет образ в статусе «В очереди», привязывает его к пользователю и ставит сборку в очередь; 7. Система возвращает ответ о принятии заказа, редактор очищается, и пользователь перенаправляется в библиотеку образов;

Альтернативный поток 1: «Не заполнены обязательные данные». На шаге 5 название образа не указано или не выбран ни один компонент. Система не отправляет запрос и отображает сообщение «Укажите название образа» или «Добавьте хотя бы один компонент», после чего пользователь возвращается к шагу 3 основного потока;

Альтернативный поток 2: «Компонент не найден или обнаружена циклическая зависимость». На шаге 6 выбранный компонент отсутствует в

каталоге либо в графе зависимостей обнаружен цикл. Система отклоняет заказ и отображает сообщение об ошибке, например «Обнаружена циклическая зависимость с участием компонента '…'», после чего пользователь возвращается к шагу 4 основного потока;

Альтернативный поток 3: «Сборка завершилась ошибкой». После шага 7 фоновая сборка прервана по ошибке или превышено время ожидания. Система переводит образ в статус «Ошибка», сохраняет журнал сборки, файл образа не создаётся и кнопка «Скачать .img» не отображается;

Постусловие: образ сохранён в библиотеке образов и привязан к заказавшему его пользователю; сборка поставлена в очередь и выполняется в фоновом режиме, при этом одновременно собирается только один образ.

Прецедент: «Отследить статус и скачать образ»

Действующее лицо: пользователь (роль USER или ADMIN);

Предусловие: пользователь авторизован в системе, им заказан хотя бы один образ;

Основной поток: «Отследить статус и скачать образ». 1. Пользователь открывает пункт «Образы» главного меню; 2. Система отображает библиотеку образов в виде карточек с названием, статусом, дистрибутивом, платой, составом и стоимостью; 3. Система автоматически обновляет список каждые 5 секунд, статус образа последовательно меняется: «В очереди», «Сборка», «Готов»; 4. Пользователь при необходимости уточняет список с помощью поиска, фильтров по статусу, плате и компонентам и сортировки по цене; 5. Для образа в статусе «Готов» система отображает кнопку «Скачать .img»; 6. Пользователь нажимает кнопку «Скачать .img»; 7. Система проверяет статус образа и наличие файла артефакта и передаёт файл, который сохраняется на устройство пользователя под именем «название образа».img;

Альтернативный поток 1: «Образ не готов к загрузке». На шаге 6 образ не имеет статуса «Готов» либо файл артефакта отсутствует на сервере (например, был удалён). Система отклоняет запрос, и на странице отображается сообщение

«Загрузка недоступна»; при обращении к интерфейсу напрямую сервер возвращает сообщение «Образ ещё не готов к загрузке (статус: …)» или «Файл артефакта не найден на сервере»;

Альтернативный поток 2: «Сборка завершилась ошибкой». На шаге 3 образ получает статус «Ошибка». Кнопка «Скачать .img» не отображается; администратор может изменить состав образа и запустить сборку повторно;

Постусловие: образ в статусе «Готов» передан пользователю в виде файла .img; состояние образа и его артефакт в системе не изменены.

Прецедент: «Изменить образ с пересборкой»

Действующее лицо: администратор (роль ADMIN);

Предусловие: администратор авторизован в системе, образ существует и не находится в статусе «В очереди» или «Сборка»;

Основной поток: «Изменить образ с пересборкой». 1. Администратор открывает библиотеку образов и нажимает кнопку «Изменить» на карточке нужного образа; 2. Система открывает редактор с заголовком «Редактирование образа #N» и подгружает параметры и состав образа, отображая пояснение, что после сохранения образ будет пересобран; 3. Администратор изменяет название, дистрибутив, плату, описание или состав компонентов; 4. Администратор нажимает кнопку «Сохранить и пересобрать»; 5. Система повторно проверяет данные, пересчитывает транзитивные зависимости и стоимость, переводит образ в статус «В очереди», сбрасывает прежний журнал сборки и удаляет ранее созданный файл образа; 6. Система ставит пересборку в очередь, редактор закрывается, и администратор перенаправляется в библиотеку образов, где отслеживает статус образа;

Альтернативный поток 1: «Образ находится в очереди или собирается». На шаге 4 образ имеет статус «В очереди» или «Сборка». Система отклоняет изменение и отображает сообщение «Образ сейчас в очереди или собирается — дождитесь окончания сборки»; кнопка «Изменить» на карточке образа в таких статусах недоступна (подсказка «Дождитесь окончания сборки»);

Альтернативный поток 2: «Некорректные данные». На шаге 4 не указано название образа или не выбран ни один компонент. Система отображает сообщение «Укажите название образа» или «Добавьте хотя бы один компонент» и остаётся в редакторе;

Альтернативный поток 3: «Отмена изменений». На шаге 3 или 4 администратор нажимает кнопку «Отмена». Система очищает редактор и возвращает администратора в библиотеку образов, образ остаётся без изменений;

Постусловие: параметры образа обновлены, предыдущий артефакт удалён, образ находится в очереди на пересборку и после её завершения получает статус «Готов» или «Ошибка».

Прецедент: «Добавить компонент»

Действующее лицо: администратор (роль ADMIN);

Предусловие: администратор авторизован в системе, в системе существует хотя бы один тип компонента;

Основной поток: «Добавить компонент». 1. Администратор выбирает пункт «+ Компонент» главного меню; 2. Система отображает редактор компонентов с полями «Название», «Тип», «Стоимость», «Описание», блоком выбора зависимостей и списком существующих компонентов; 3. Администратор вводит название, выбирает тип, указывает стоимость (неотрицательное число) и описание; 4. Администратор при необходимости находит в блоке зависимостей нужные компоненты и отмечает их; 5. Администратор нажимает кнопку «Создать компонент»; 6. Система проверяет данные, находит тип и зависимости, сохраняет компонент и отображает сообщение «Компонент «…» создан», форма очищается, а список существующих компонентов обновляется;

Альтернативный поток 1: «Не заполнены обязательные поля». На шаге 5 не указано название компонента, тип или стоимость либо стоимость отрицательна. Система отображает сообщение о нарушенном требовании,

например «Название компонента обязательно» или «Стоимость не может быть отрицательной», и остаётся в форме;

Альтернативный поток 2: «Зависимость не найдена». На шаге 6 указанный тип или зависимость отсутствуют в базе данных (например, были удалены другим администратором). Система отображает сообщение «Тип компонента не найден: …» или «Зависимость не найдена: …», компонент не создаётся;

Постусловие: новый компонент сохранён в каталоге, отображается в библиотеке компонентов и доступен для выбора в редакторе образов.

Прецедент: «Удалить компонент»

Действующее лицо: администратор (роль ADMIN);

Предусловие: администратор авторизован в системе, компонент существует в каталоге;

Основной поток: «Удалить компонент». 1. Администратор открывает библиотеку компонентов или редактор компонентов и нажимает кнопку «Удалить» (в редакторе – значок «✕») у нужного компонента; 2. Система запрашивает подтверждение «Удалить компонент «…»?»; 3. Администратор подтверждает действие; 4. Система проверяет, что компонент не входит ни в один образ и что от него не зависят другие компоненты; 5. Система удаляет компонент из каталога и обновляет список;

Альтернативный поток 1: «Компонент входит в образы». На шаге 4 компонент содержится в составе хотя бы одного образа. Удаление запрещено, система отображает сообщение «Компонент «…» входит в образы — сначала измените или удалите эти образы», компонент остаётся в каталоге;

Альтернативный поток 2: «От компонента зависят другие компоненты». На шаге 4 компонент указан как зависимость у другого компонента. Удаление запрещено, система отображает сообщение «От компонента «…» зависят другие компоненты — сначала уберите эту зависимость»;

Альтернативный поток 3: «Отказ от удаления». На шаге 3 администратор отказывается от подтверждения. Система отменяет действие, компонент остаётся без изменений;

Постусловие: компонент удалён из каталога и недоступен для выбора в образах либо, при запрете удаления, остался в каталоге без изменений.

Прецедент: «Зарегистрироваться и войти»

Действующее лицо: посетитель (неавторизованный пользователь);

Предусловие: посетитель не авторизован в системе и не имеет учётной записи;

Основной поток: «Зарегистрироваться и войти». 1. Посетитель нажимает кнопку «Регистрация» в шапке приложения; 2. Система отображает форму регистрации с полями «Имя», «Фамилия», «Email» и «Пароль» и подсказкой о требованиях к паролю (не менее 8 символов, цифра, строчная и заглавная буквы, спецсимвол); 3. Посетитель заполняет поля и нажимает кнопку «Зарегистрироваться»; 4. Система проверяет данные и уникальность адреса электронной почты, сохраняет пользователя с ролью USER, хранит пароль в виде хэша и выдаёт токен доступа; 5. Система выполняет вход, перенаправляет пользователя в библиотеку образов и отображает в меню пункты «Создать образ», «Аккаунт» и «Выйти». Для входа при последующих посещениях: 6. Пользователь нажимает кнопку «Вход», вводит email и пароль и нажимает кнопку «Войти»; 7. Система проверяет учётные данные, выдаёт токен доступа и перенаправляет пользователя на запрошенную ранее страницу или в библиотеку образов;

Альтернативный поток 1: «Данные не прошли проверку». На шаге 4 адрес электронной почты имеет неверный формат либо пароль не соответствует требованиям. Система отображает сообщения проверки (например, «Password must be at least 8 characters long» или «Invalid email format»), регистрация не выполняется, пользователь возвращается к шагу 3;

Альтернативный поток 2: «Адрес уже зарегистрирован». На шаге 4 пользователь с таким адресом уже существует. Система отображает сообщение «Пользователь с таким email уже существует»;

Альтернативный поток 3: «Неверные учётные данные или превышен лимит попыток». На шаге 7 данные не подходят, и попытка входа учитывается как неудачная. После превышения допустимого числа неудачных попыток (по умолчанию 5 для одного адреса почты и 20 для одного адреса клиента за 300 секунд) система отклоняет вход до проверки пароля и отображает сообщение «Слишком много неудачных попыток входа. Повторите через N с»;

Постусловие: учётная запись создана с ролью USER, пользователь авторизован, токен доступа сохранён в браузере, разделы, требующие авторизации, доступны; при выходе токен удаляется.

ПРИЛОЖЕНИЕ Б

ДИАГРАММЫ ПРЕЦЕДЕНТОВ И БИЗНЕС-ПРОЦЕССОВ

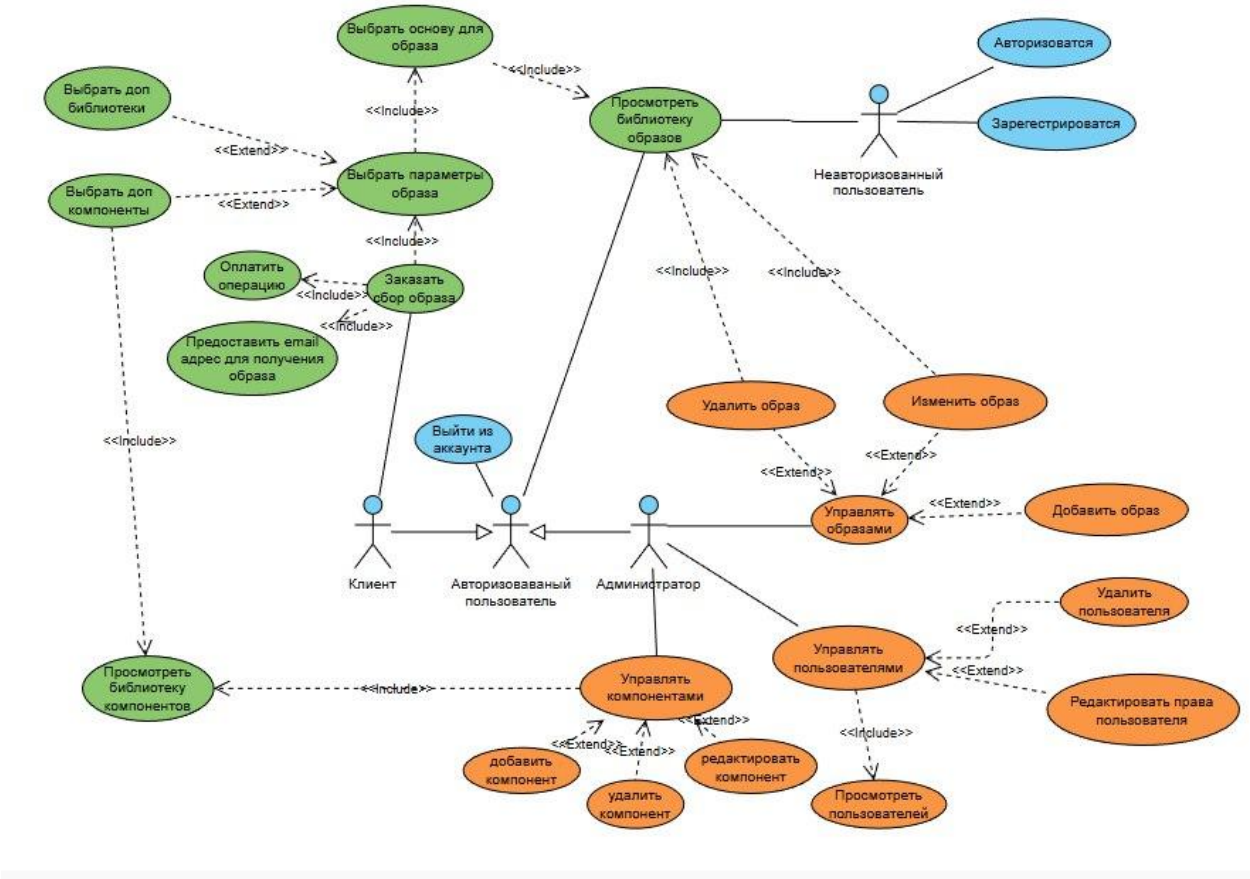



Рисунок Б.1 – Диаграмма прецедентов

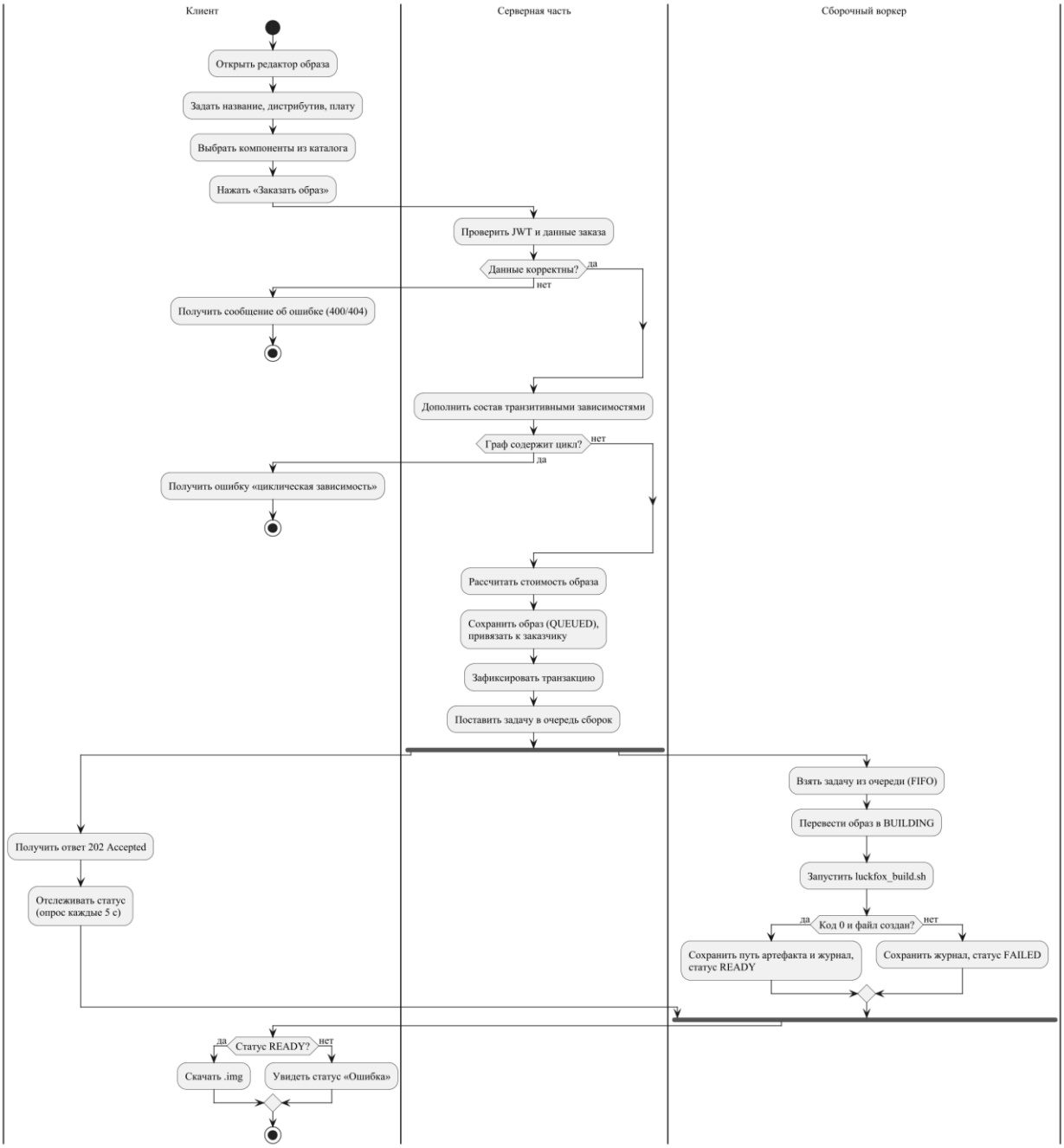



Рисунок Б.2 – Диаграмма деятельности бизнес-процесса «Заказ и сборка образа»

ПРИЛОЖЕНИЕ В

ДИАГРАММЫ КЛАССОВ И СОСТОЯНИЙ

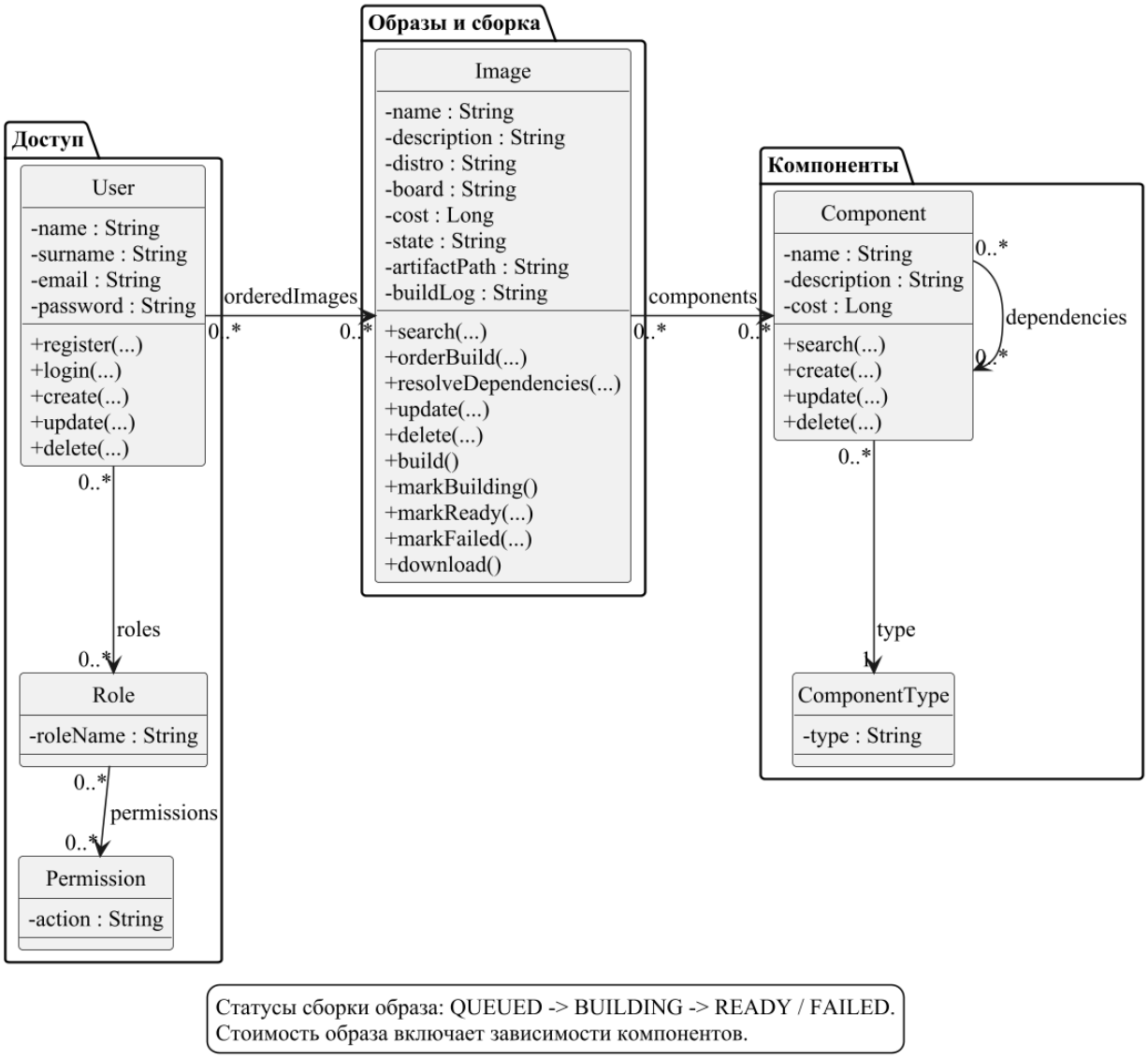



Рисунок В.1 – Диаграмма классов предметной области

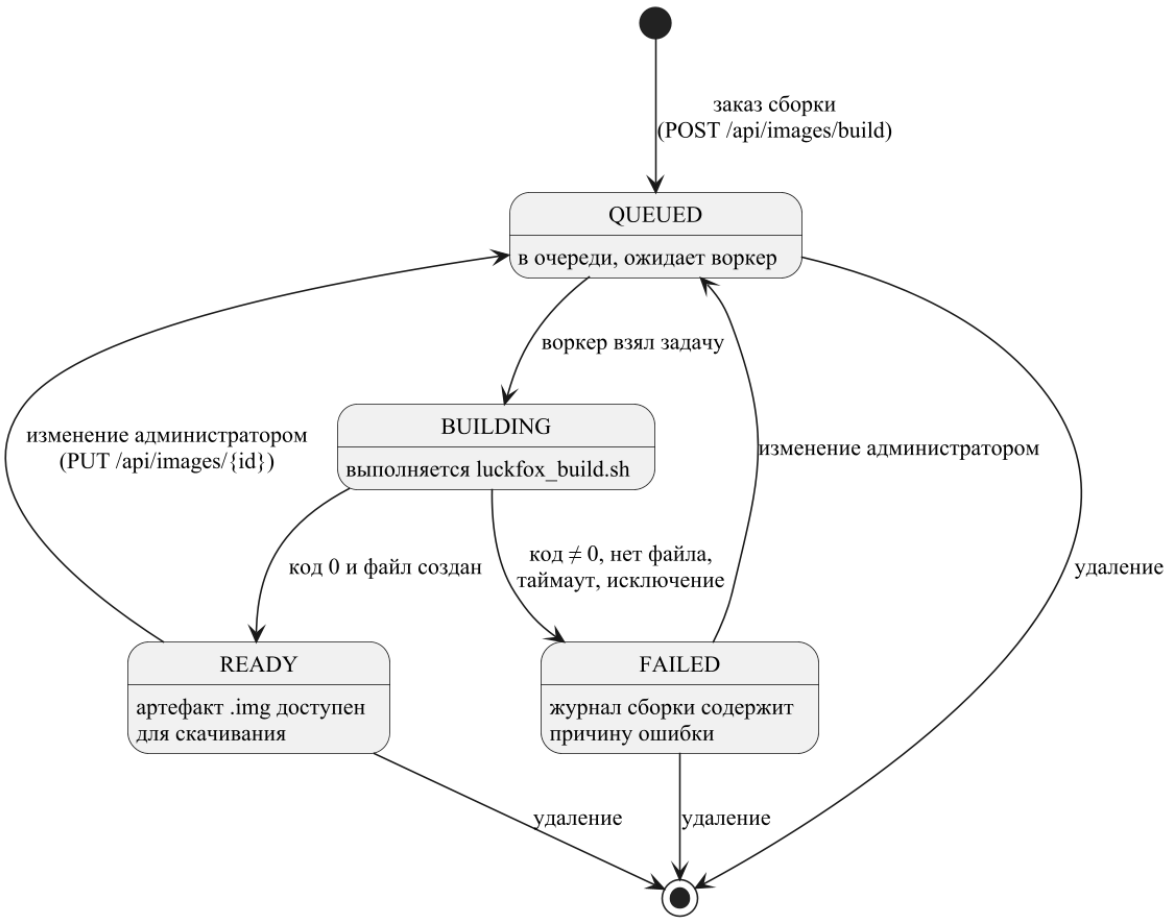



Рисунок В.2 – Диаграмма состояний сущности «Образ»

ПРИЛОЖЕНИЕ Г

СХЕМЫ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТОВ И РАЗВЁРТЫВАНИЯ СИСТЕМЫ

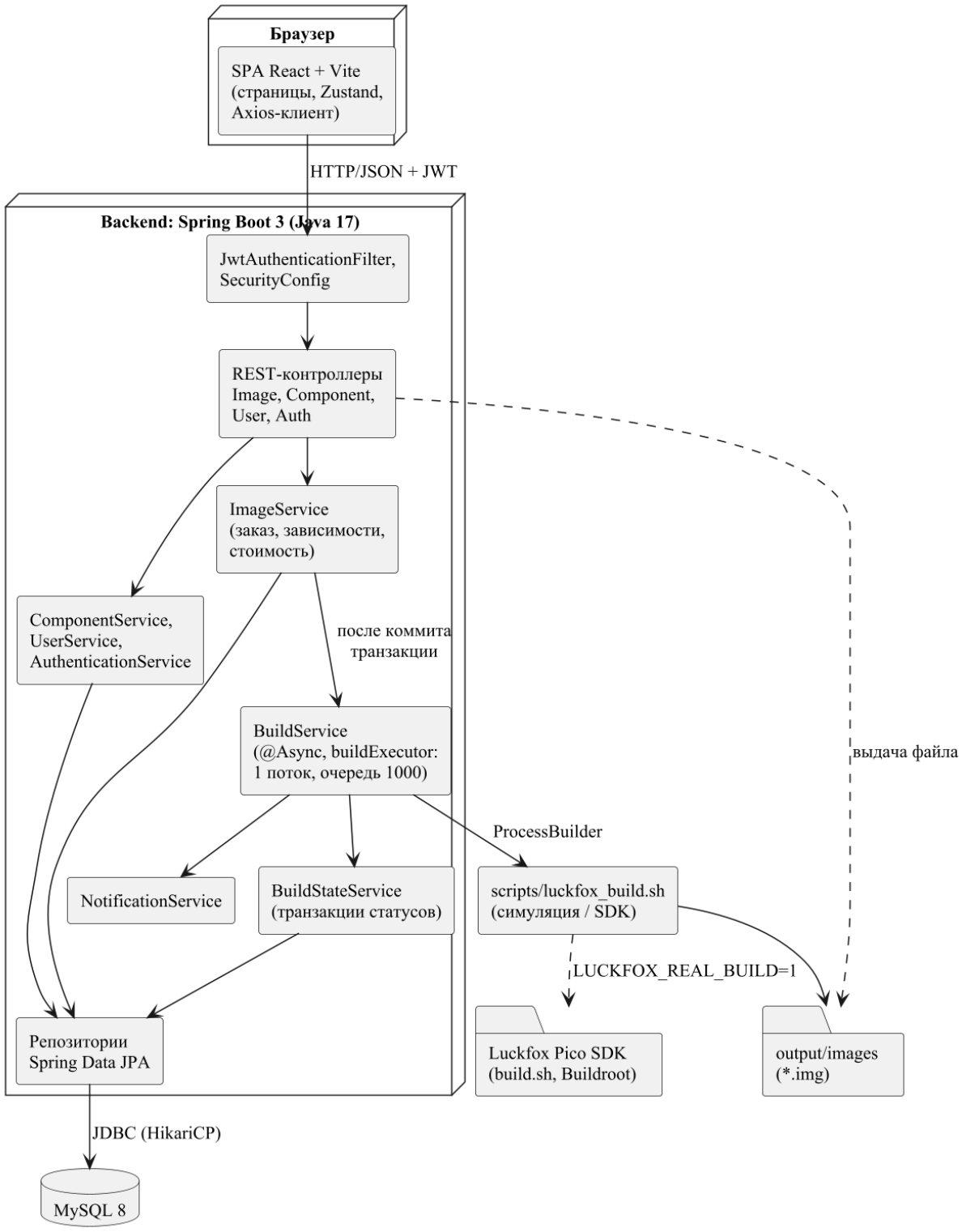



Рисунок Г.1 – Диаграмма компонентов системы

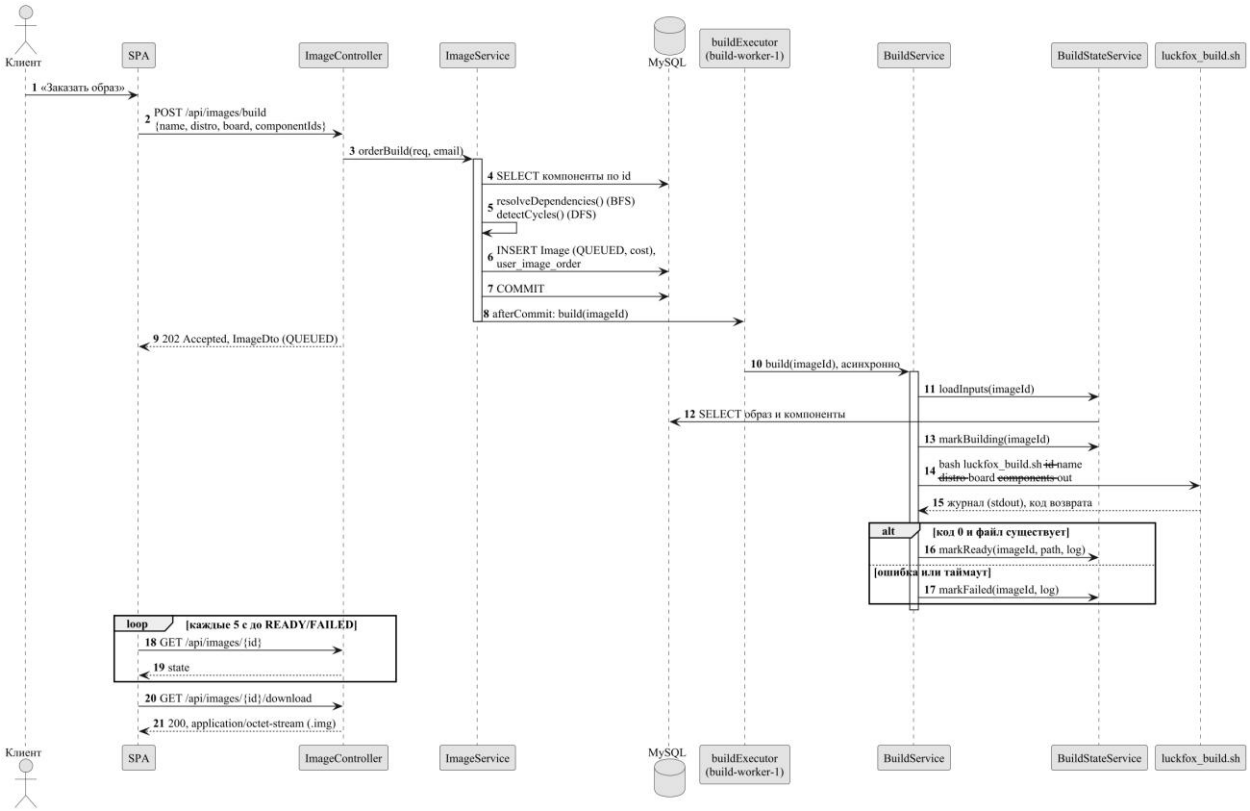



Рисунок Г.2 – Диаграмма последовательности «Заказ, сборка и скачивание образа»

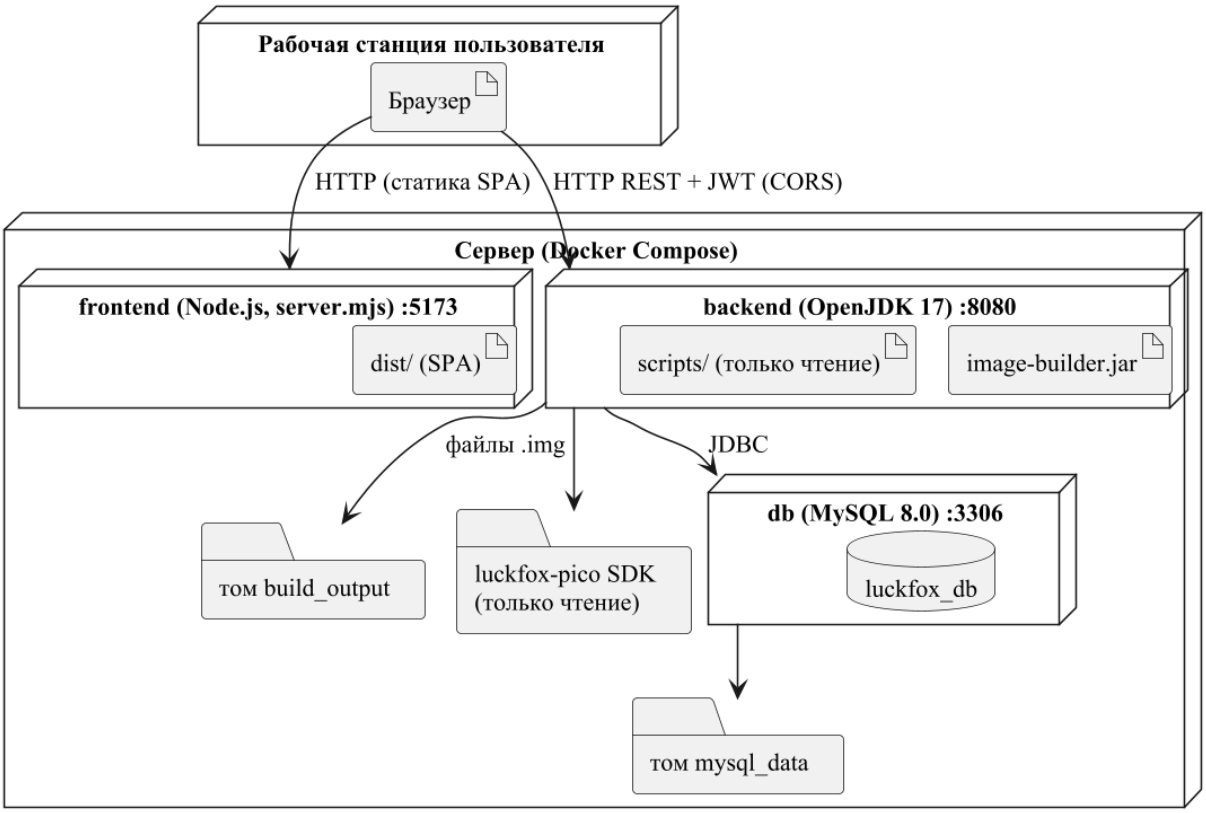



Рисунок Г.3 – Диаграмма развёртывания системы

ПРИЛОЖЕНИЕ Д

СХЕМА БАЗЫ ДАННЫХ И ОПИСАНИЕ ТАБЛИЦ

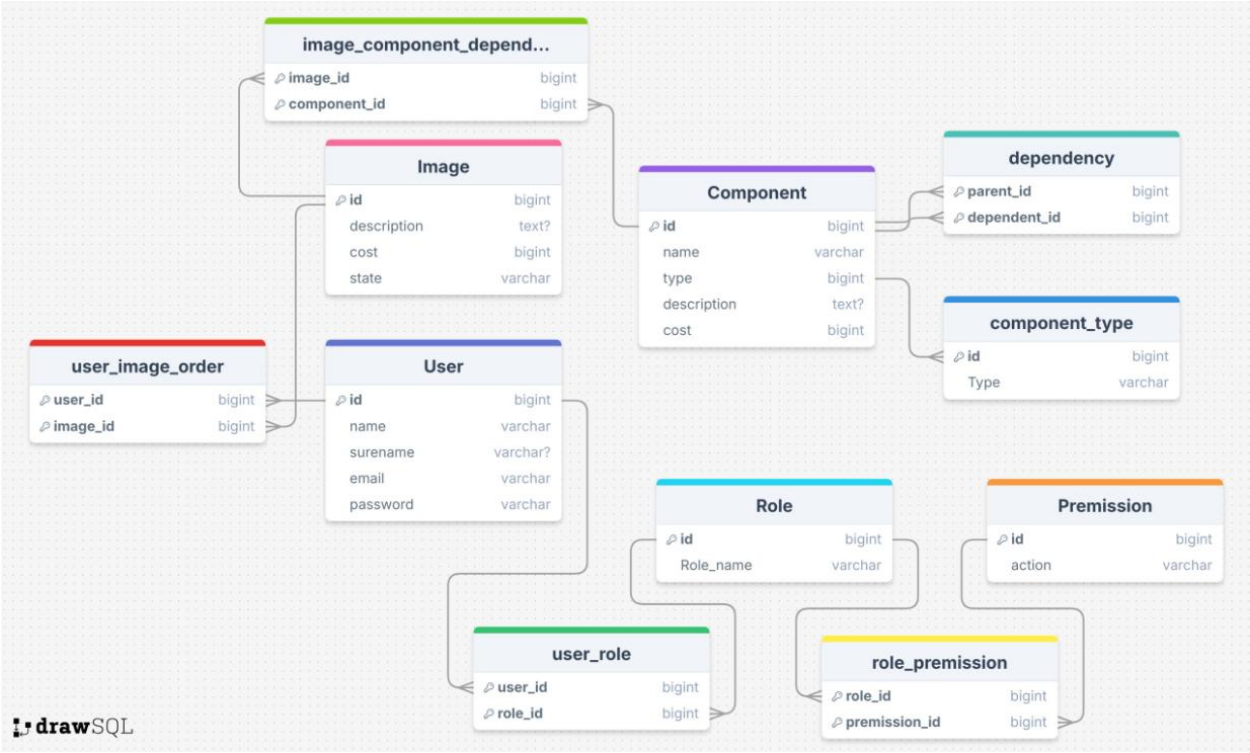



Рисунок Д.1 – ER-диаграмма базы данных (модель этапа проектирования)

На рисунке приведена ER-диаграмма, разработанная на этапе проектирования. В реализации состав таблиц и связей сохранён, а таблицы расширены: в Image добавлены поля name, distro, board, artifact_path, build_log и version, в Component – поле version; фактическая структура всех таблиц приведена ниже.

Рабочую схему базы данных создаёт Hibernate (параметр spring.jpa.hibernate.ddl-auto=update) по JPA-сущностям пакета entity; при первом запуске на пустой MySQL создаются все 11 таблиц, приведённых ниже, а при последующих запусках схема расширяется без удаления данных. Файл DOCs/этап3/java_working.sql является ранней документной моделью и служит только справочным материалом. Имена таблиц и столбцов в приложении указаны так, как они заданы аннотациями @Table, @Column и @JoinTable (в том числе с историческими написаниями Premission и surename); таблица «User» в

запросах заключается в обратные кавычки, так как User – зарезервированное слово MySQL.

Таблица Д.1 – Таблица «User»

| **Поле** | **Тип** | **Ограничения** | **Описание** |
|---|---|---|---|
| id | BIGINT | PK, AUTO_INCREMENT | Идентификатор пользователя |
| name | VARCHAR(255) | NOT NULL | Имя пользователя |
| surename | VARCHAR(255) | NULL | Фамилия пользователя |
| email | VARCHAR(255) | NOT NULL, UNIQUE | Адрес электронной почты, используется как логин |
| password | VARCHAR(255) | NOT NULL | Хеш пароля (BCrypt) |

Таблица Д.2 – Таблица «Role»

| **Поле** | **Тип** | **Ограничения** | **Описание** |
|---|---|---|---|
| id | BIGINT | PK, AUTO_INCREMENT | Идентификатор роли |
| Role_name | VARCHAR(255) | NOT NULL | Название роли (USER, ADMIN) |

Таблица Д.3 – Таблица «Premission»

| **Поле** | **Тип** | **Ограничения** | **Описание** |
|---|---|---|---|
| id | BIGINT | PK, AUTO_INCREMENT | Идентификатор разрешения |
| action | VARCHAR(255) | NOT NULL | Название действия, которое разрешено выполнять |

Таблица Д.4 – Таблица «user_role»

| **Поле** | **Тип** | **Ограничения** | **Описание** |
|---|---|---|---|
| user_id | BIGINT | PK (составной), FK → User.id | Пользователь |
| role_id | BIGINT | PK (составной), FK → Role.id | Роль, назначенная пользователю |

Таблица Д.5 – Таблица «role_premission»

| **Поле** | **Тип** | **Ограничения** | **Описание** |
|---|---|---|---|
| role_id | BIGINT | PK (составной), FK → Role.id | Роль |
| premission_id | BIGINT | PK (составной), FK → Premission.id | Разрешение, входящее в роль |

Таблица Д.6 – Таблица «Image»

| **Поле** | **Тип** | **Ограничения** | **Описание** |
|---|---|---|---|
| id | BIGINT | PK, AUTO_INCREMENT | Идентификатор образа |
| name | VARCHAR(255) | NULL | Название образа |
| description | TEXT | NULL | Описание образа |
| distro | VARCHAR(255) | NULL | Дистрибутив (например, buildroot) |
| board | VARCHAR(255) | NULL | Целевая плата (по умолчанию RV1103_Luckfox_Pico) |
| cost | BIGINT | NOT NULL, индекс idx_image_cost, входит в idx_image_state_cost | Стоимость образа – сумма стоимостей компонентов |
| state | VARCHAR(255) | NOT NULL, входит в idx_image_state_cost | Состояние сборки: CREATED, QUEUED, BUILDING, READY, FAILED |
| artifact_path | VARCHAR(255) | NULL | Путь к собранному файлу образа на сервере |
| build_log | TEXT | NULL | Журнал сборки |
| version | BIGINT | NULL, @Version | Номер версии записи для оптимистичной блокировки |

Таблица Д.7 – Таблица «Component»

| **Поле** | **Тип** | **Ограничения** | **Описание** |
|---|---|---|---|
| id | BIGINT | PK, AUTO_INCREMENT | Идентификатор компонента |
| name | VARCHAR(255) | NOT NULL, входит в idx_component_name | Название компонента |
| type | BIGINT | NOT NULL, FK → component_type.id, входит в idx_component_type_cost | Тип компонента |
| description | TEXT | NULL | Описание компонента |
| cost | BIGINT | NOT NULL, входит в idx_component_type_cost | Стоимость компонента |
| version | BIGINT | NULL, @Version | Номер версии записи для оптимистичной блокировки |

Таблица Д.8 – Таблица «component_type»

| **Поле** | **Тип** | **Ограничения** | **Описание** |
|---|---|---|---|
| id | BIGINT | PK, AUTO_INCREMENT | Идентификатор типа компонента |
| Type | VARCHAR(255) | NOT NULL | Название типа (категория) компонента |

Таблица Д.9 – Таблица «dependency»

| Поле | Тип | Ограничения | Описание |
|---|---|---|---|
| parent_id | BIGINT | PK (составной), FK → Component.id | Компонент, у которого есть зависимость |
| dependent_id | BIGINT | PK (составной), FK → Component.id | Компонент, от которого зависит parent_id |

Таблица Д.10 – Таблица «image_component_dependency»

| Поле | Тип | Ограничения | Описание |
|---|---|---|---|
| image_id | BIGINT | PK (составной), FK → Image.id | Образ |
| component_id | BIGINT | PK (составной), FK → Component.id | Компонент, входящий в состав образа |

Таблица Д.11 – Таблица «user_image_order»

| Поле | Тип | Ограничения | Описание |
|---|---|---|---|
| user_id | BIGINT | PK (составной), FK → User.id | Пользователь, заказавший образ |
| image_id | BIGINT | PK (составной), FK → Image.id | Заказанный образ |

Для ускорения выборок в каталогах определены следующие индексы (заданы в @Table сущностей): idx_image_state_cost по полям (state, cost) – фильтр образов по состоянию с сортировкой по цене; idx_image_cost по полю (cost) – сортировка образов по цене; idx_component_name по полям (name, id) – сортировка и постраничный вывод компонентов по названию; idx_component_type_cost по полям (type, cost) – фильтр компонентов по типу с сортировкой по цене. Кроме того, на поле email таблицы «User» создан уникальный индекс, который гарантирует отсутствие дубликатов адресов при одновременной регистрации. В таблицах «Image» и «Component» предусмотрено поле version (аннотация @Version): при сохранении Hibernate сравнивает номер версии и отклоняет конкурентную запись устаревшей копии, что защищает от потерянных обновлений при одновременном редактировании; в этом случае API возвращает код 409.

ПРИЛОЖЕНИЕ Е

ДИАГРАММА НАВИГАЦИИ И ЭКРАНЫ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

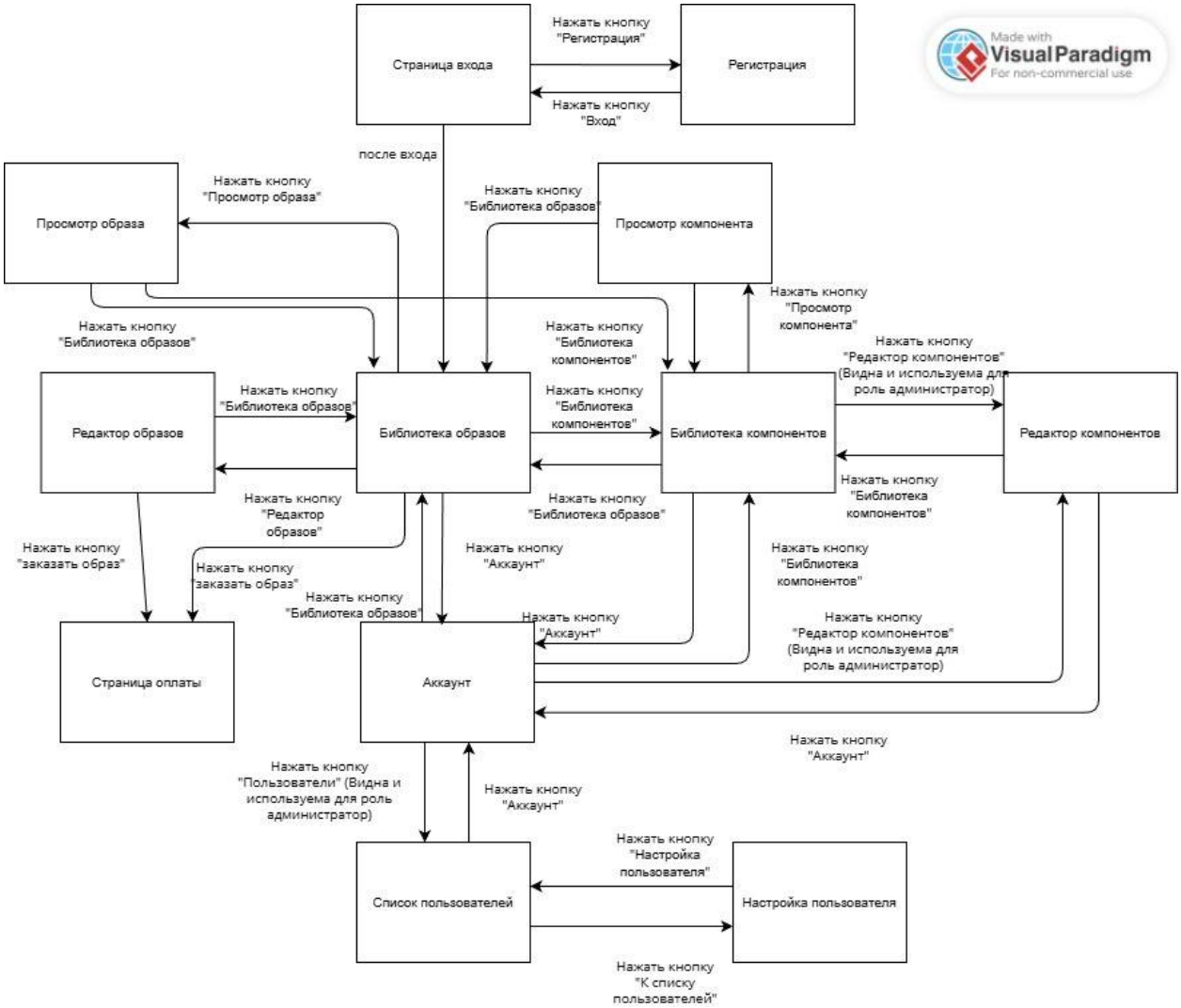



Рисунок Е.1 – Диаграмма навигации

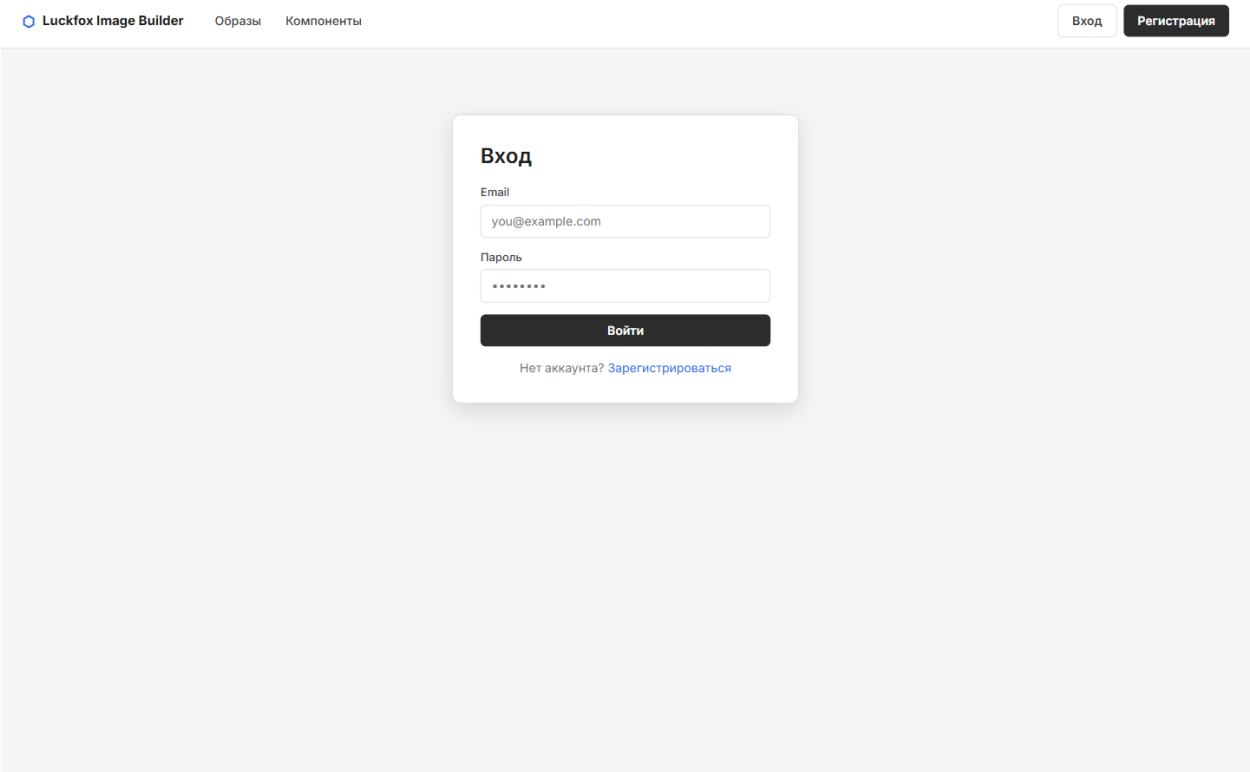



Рисунок Е.2 – Страница входа

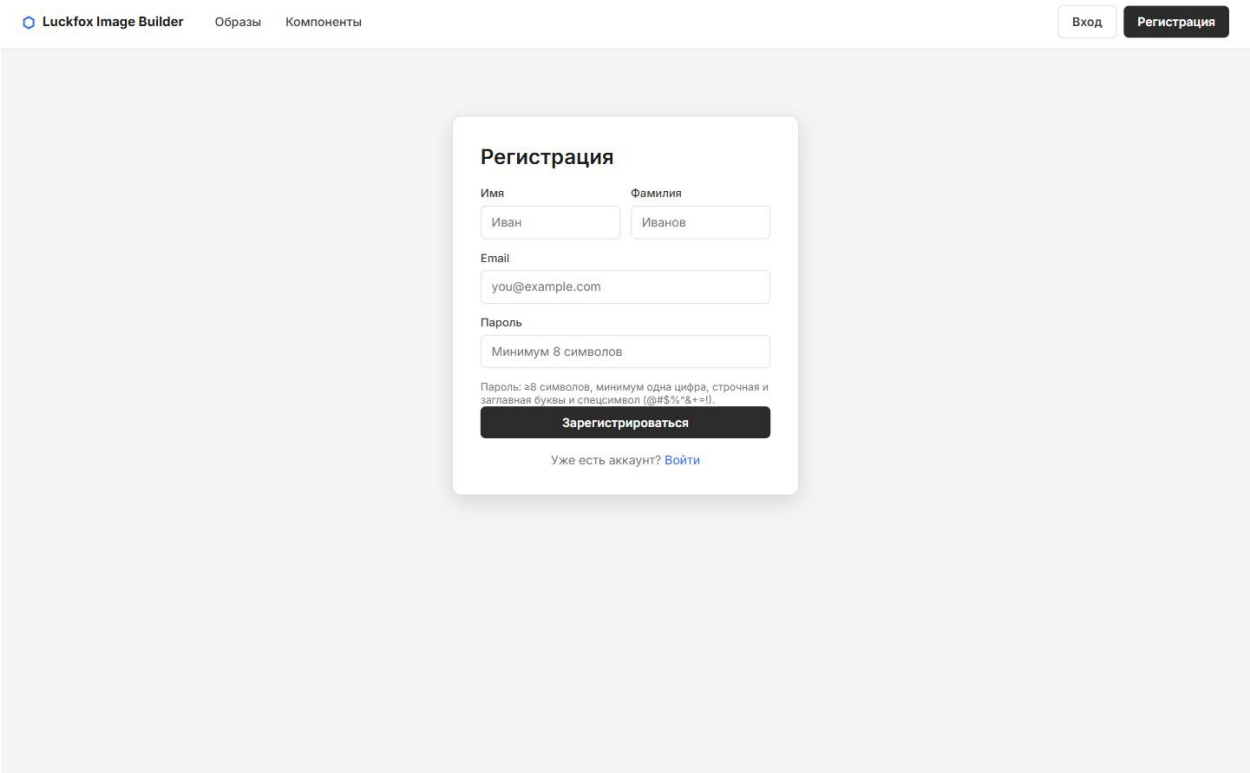



Рисунок Е.3 – Страница регистрации

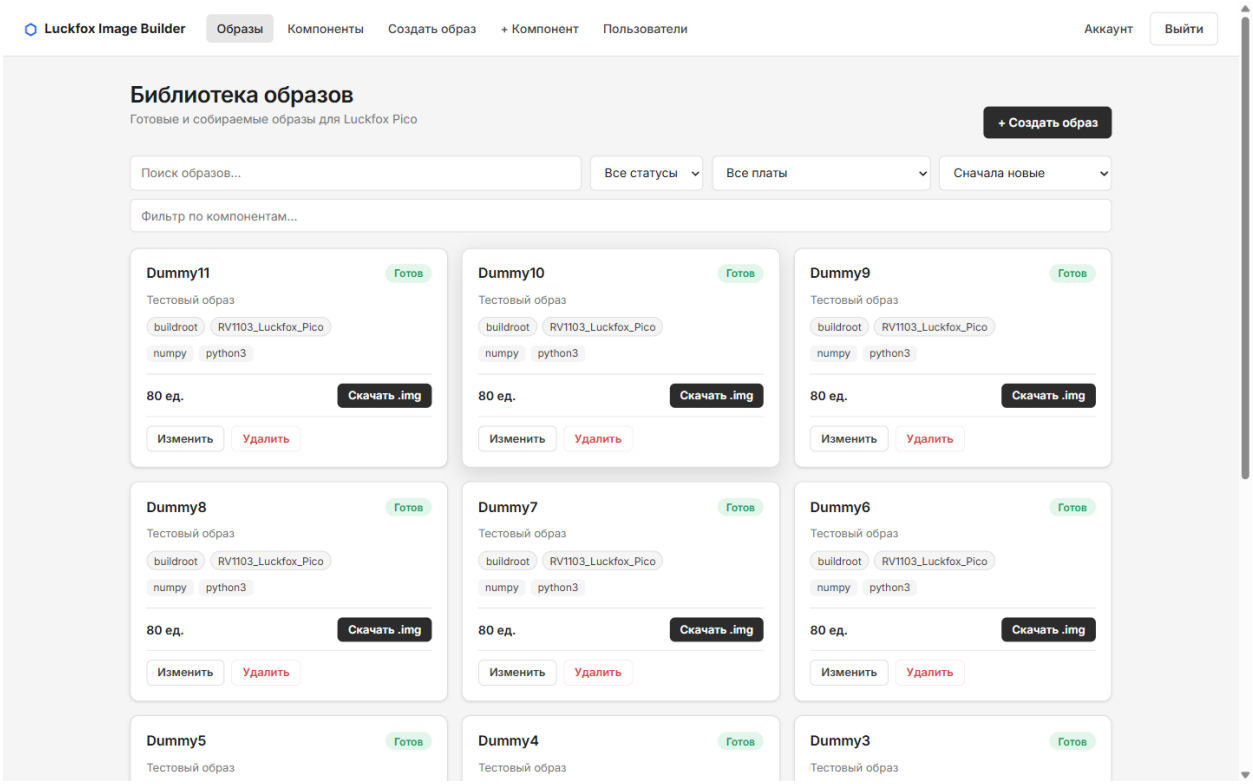



Рисунок Е.4 – Библиотека образов

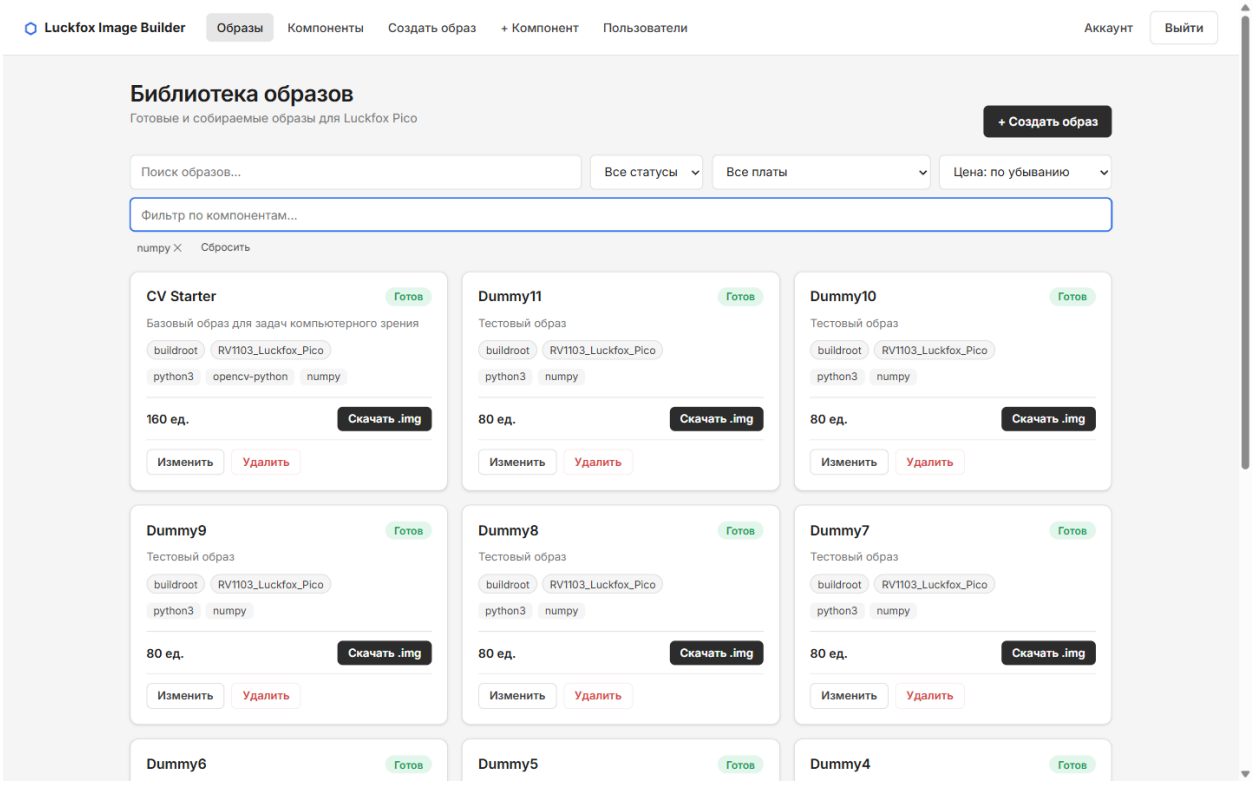



Рисунок Е.5 – Библиотека образов с фильтром по компоненту и сортировкой по цене

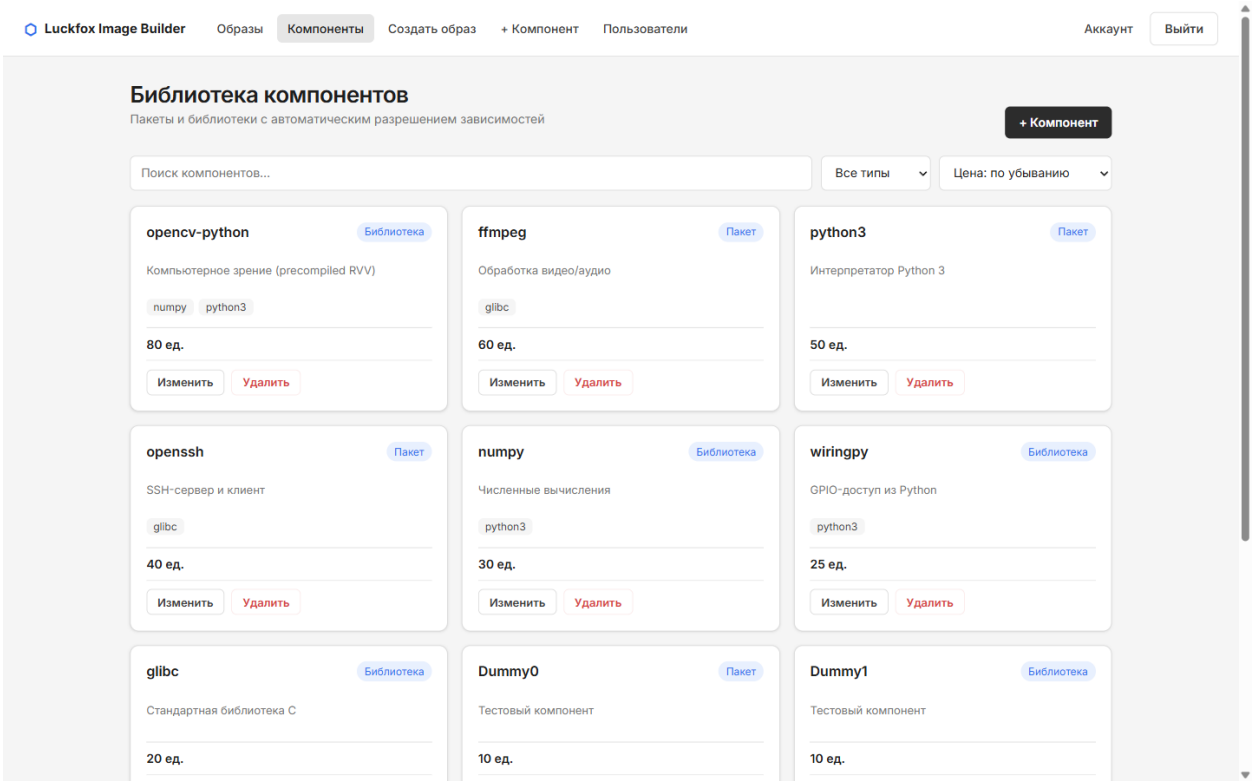



Рисунок Е.6 – Библиотека компонентов

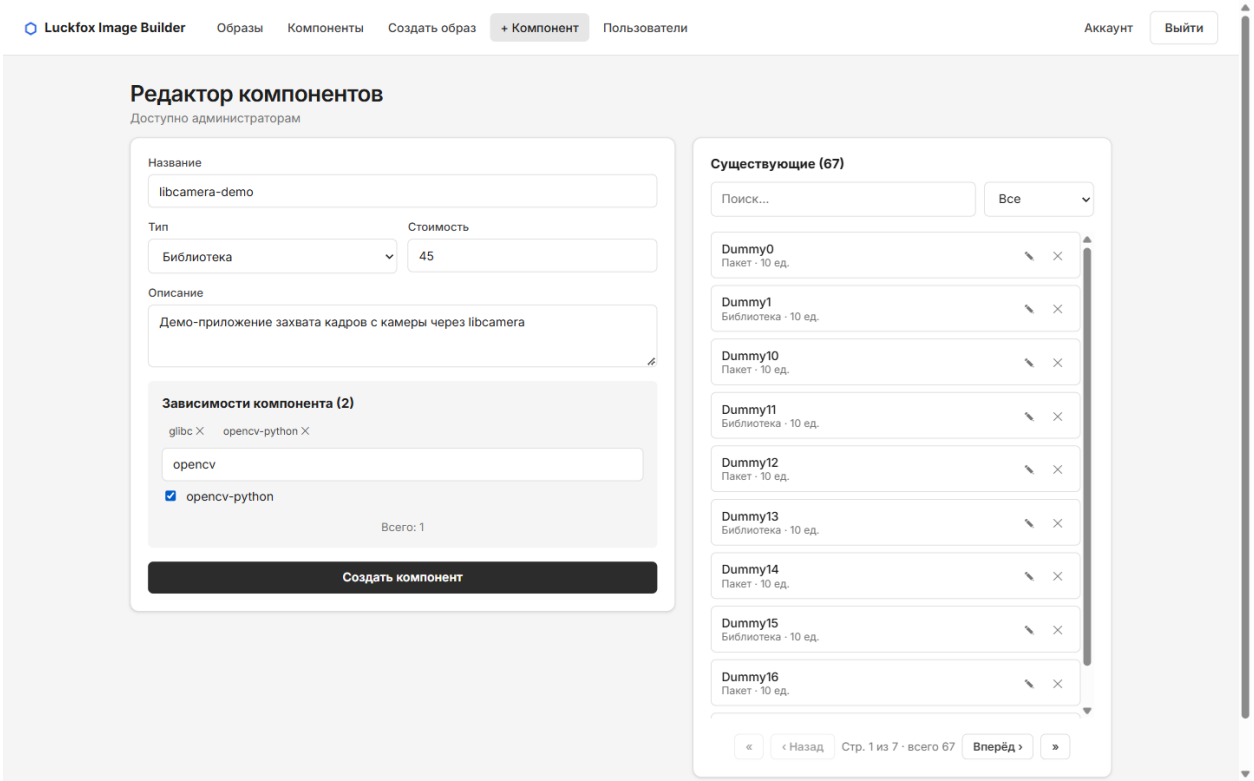



Рисунок Е.7 – Редактор компонентов

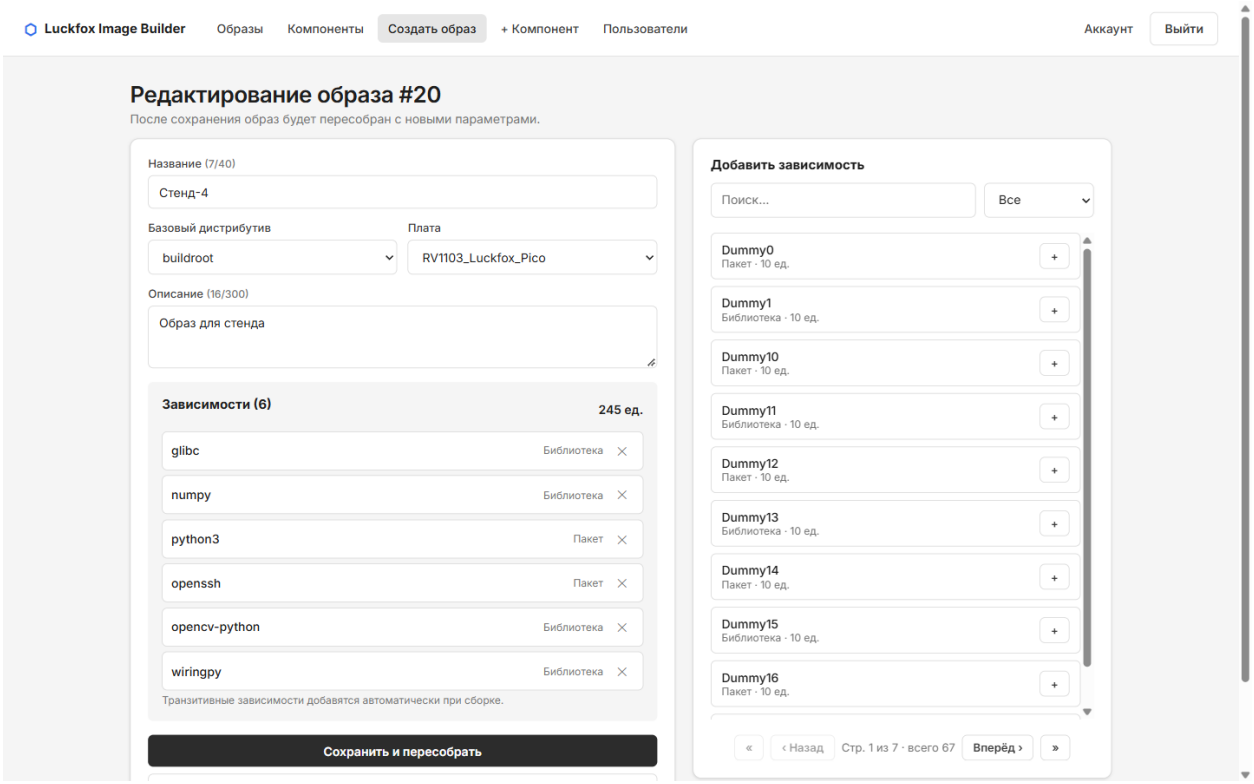



Рисунок Е.8 – Изменение образа администратором с пересборкой

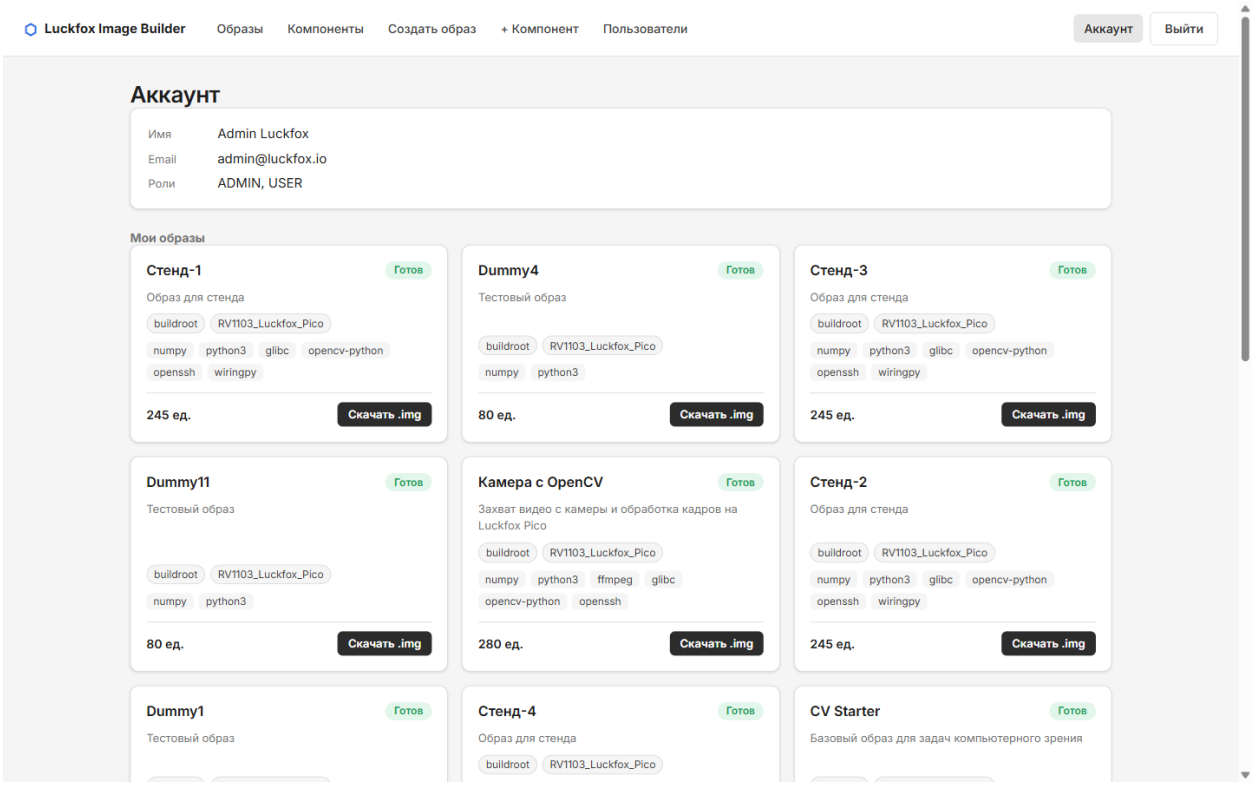



Рисунок Е.9 – Аккаунт и раздел «Мои образы» с готовыми образами

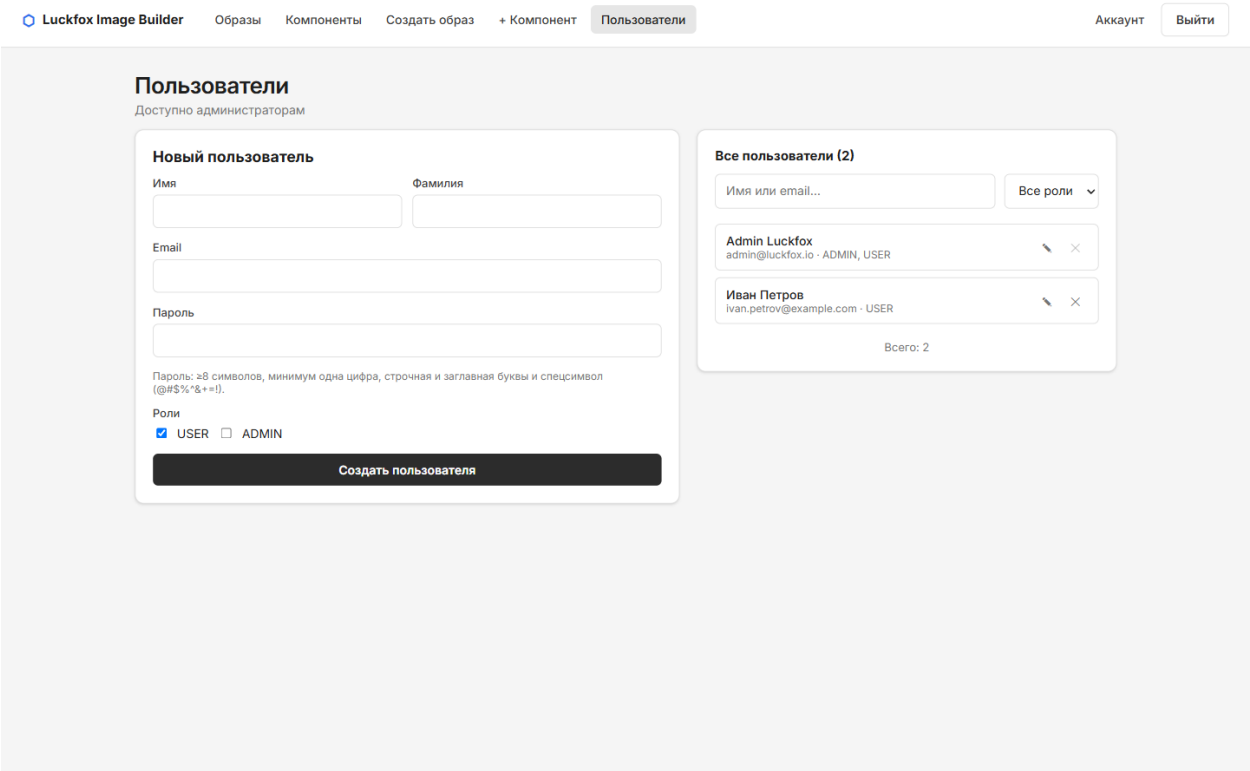



Рисунок Е.10 – Управление пользователями

ПРИЛОЖЕНИЕ Ж

ПРОТОКОЛЫ И СВОДНЫЕ РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

## Ж.1 Модульные тесты

Модульные тесты написаны на JUnit 5 с использованием Mockito и AssertJ и размещены в каталоге backend/src/test/java. Они проверяют сервисы, DTO, валидацию запросов, сущность пользователя и обработчик ошибок без обращения к реальной базе данных. Состав тестовых классов приведён в таблице Ж.1.

Таблица Ж.1 – Состав модульных тестов

| **Класс теста** | **Тестируемый объект** | **Кол-во** | **Проверяемые свойства** |
|---|---|---|---|
| DtoMappingTest | Преобразование сущностей в DTO | 5 | Разворачивание прямых зависимостей компонента, обработка пустого списка зависимостей, признак доступности образа для скачивания, скрытие пути к артефакту и журнала, отсутствие пароля в ответе о пользователе |
| PageResponseTest | Постраничная выдача (PageRequest, PageResponse) | 5 | Ограничение номера и размера страницы, сортировка по цене с сохранением сортировки по умолчанию, замена неизвестной сортировки, перенос данных страницы в ответ |
| RequestValidationTest | Валидация запросов (регистрация, сборка, компонент, пользователь) | 10 | Приём корректной регистрации, отклонение 5 вариантов слабого пароля, некорректный email и пустое имя, обязательность названия, дистрибутива и компонентов, запрет отрицательной цены, обязательность роли |
| UserTest | Сущность User (UserDetails) | 2 | Префикс ROLE_ у полномочий, использование email в качестве имени пользователя и активность учётной записи |
| GlobalExceptionHandlerTest | Глобальный обработчик ошибок | 5 | Ошибки валидации по полям, код 404 с сообщением, код 429 с заголовком Retry-After, код 503 при исчерпании пула соединений, код 409 при конфликте оптимистичной блокировки |
| AuthenticationServiceTest | Сервис регистрации и входа | 6 | Регистрация с хешированием пароля и ролью USER, создание отсутствующей роли, отказ при занятом email, успешный вход |

| Класс теста | Тестируемый объект | Кол-во | Проверяемые свойства |
|---|---|---|---|
| | | | со сбросом счётчика, учёт неудачных попыток, пропуск проверки пароля при блокировке |
| BuildServiceTest | Сервис сборки образов | 6 | Успешная сборка, передача компонентов в сценарий, ненулевой код возврата, отсутствие артефакта, принудительное завершение по таймауту, отсутствие образа |
| BuildStateServiceTest | Сервис состояний сборки | 4 | Чтение входных данных с сортировкой компонентов, фиксация готовности, сохранение журнала при начале сборки, ошибка при отсутствии образа |
| ComponentServiceTest | Сервис компонентов | 8 | Создание с типом и зависимостями, неизвестные тип и зависимость, запрет зависимости от самого себя, транзитивный цикл, запрет удаления используемых компонентов, удаление свободного компонента |
| ImageServiceTest | Сервис образов | 11 | Заказ с транзитивными зависимостями и подсчётом стоимости, целевая плата по умолчанию, привязка к заказчику и запуск сборки, неизвестные компонент и пользователь, циклический граф, изменение, удаление и получение образа |
| JwtServiceTest | Сервис JWT-токенов | 6 | Выпуск токена и проверка его для владельца, отклонение для другого пользователя, дополнительные утверждения, срок жизни, поддельный и просроченный токены |
| LoginAttemptServiceTest | Ограничение частоты попыток входа | 7 | Допуск попыток ниже лимита, блокировка email и IP-адреса с Retry-After, нечувствительность ключа к регистру и пробелам, сброс счётчика после успеха, истечение окна, обработка пустого email |
| UserServiceTest | Сервис управления пользователями | 9 | Создание с хешированием пароля и назначением ролей, отказ без пароля, дубликат email, неизвестная роль, сохранение прежних данных при обновлении, запрет самоблокировки администратора, удаление и получение пользователя |

Всего выполнено 84 модульных теста (сумма по таблице Ж.1), ошибок и пропусков нет: команда mvn test завершилась со статусом BUILD SUCCESS.

Тесты подсистемы сборки образов (заказ, изменение и удаление образов, запуск сборочного сценария, смена состояний, каталог компонентов) приведены отдельно в таблице Ж.2.

Таблица Ж.2 – Модульные тесты подсистемы сборки

| **Тест** | **Проверка** |
|---|---|
| ImageServiceTest.orderResolvesTransitiveDependenciesAndSumsCostOnce | Заказ образа с транзитивными зависимостями: в состав попадают все зависимости, стоимость общих компонентов учитывается один раз (80 + 20 + 30 + 10), состояние QUEUED, скачивание недоступно |
| ImageServiceTest.orderUsesDefaultBoardWhenNotGiven | Если плата не указана, используется RV1103_Luckfox_Pico |
| ImageServiceTest.orderLinksImageToCustomerAndSchedulesBuild | Образ привязывается к заказчику, сборка ставится в очередь |
| ImageServiceTest.orderWithUnknownComponentFails | Заказ с несуществующим компонентом отклоняется (ResourceNotFoundException), сборка не запускается |
| ImageServiceTest.orderByUnknownUserFails | Заказ от неизвестного пользователя отклоняется |
| ImageServiceTest.cyclicDependencyGraphIsRejected | Циклический граф зависимостей отклоняется ошибкой 400, образ не сохраняется |
| ImageServiceTest.updateOfQueuedOrBuildingImageIsRejected | Изменение образа в состоянии QUEUED или BUILDING запрещено |
| ImageServiceTest.updateRequeuesImageAndDropsOldArtifact | Изменение готового образа: пересчёт стоимости, состояние QUEUED, очистка пути и журнала, удаление старого файла, повторный запуск сборки |
| ImageServiceTest.deleteOfBuildingImageIsRejected | Удаление образа в состоянии BUILDING запрещено |
| ImageServiceTest.deleteUnlinksCustomersAndRemovesArtifact | При удалении образа снимаются связи с заказчиками и удаляется файл артефакта |
| ImageServiceTest.getByIdOfMissingImageFails | Запрос несуществующего образа даёт ResourceNotFoundException |
| BuildServiceTest.successfulBuildMarksReadyWithArtifactAndNotifies | Успешная сборка: образ помечается READY с путём к артефакту, отправляется уведомление, отметки FAILED нет |
| BuildServiceTest.componentsArePassedToScript | Сборочному сценарию передаются идентификатор, название, дистрибутив, плата и список компонентов |
| BuildServiceTest.nonZeroExitMarksFailedWithExitCode | Ненулевой код возврата: образ помечается FAILED, в журнале вывод сценария и код возврата, уведомление не отправляется |

| Тест | Проверка |
|---|---|
| BuildServiceTest.zeroExitWithoutArtifactIsFailure | Нулевой код возврата без созданного файла считается сбоем |
| BuildServiceTest.hangingBuildIsKilledByTimeout | Зависшая сборка принудительно завершается по таймауту и помечается FAILED |
| BuildServiceTest.missingImageStopsBeforeBuilding | Если образ не найден, сборка не начинается |
| BuildStateServiceTest.loadInputsReturnsSortedComponentNames | Входные данные сборки: названия компонентов отсортированы, переданы дистрибутив и плата |
| BuildStateServiceTest.markReadyStoresArtifactAndLog | Состояние READY сохраняет путь к артефакту и журнал |
| BuildStateServiceTest.markBuildingKeepsPreviousLog | Начало сборки переводит образ в BUILDING и не стирает прежний журнал |
| BuildStateServiceTest.markFailedOnMissingImageThrows | Перевод в FAILED несуществующего образа вызывает ResourceNotFoundException |
| ComponentServiceTest.createSetsFieldsTypeAndDependencies | Создание компонента: поля, тип и зависимости сохраняются и отражаются в DTO |
| ComponentServiceTest.createWithUnknownTypeFails | Неизвестный тип компонента отклоняется, сохранения нет |
| ComponentServiceTest.createWithUnknownDependencyFails | Неизвестная зависимость отклоняется |
| ComponentServiceTest.componentCannotDependOnItself | Компонент не может зависеть сам от себя (ошибка 400) |
| ComponentServiceTest.transitiveCycleIsRejectedOnUpdate | Обновление, замыкающее транзитивный цикл зависимостей, отклоняется |
| ComponentServiceTest.deleteOfComponentUsedInImagesIsRejected | Нельзя удалить компонент, входящий в образы |
| ComponentServiceTest.deleteOfComponentOthersDependOnIsRejected | Нельзя удалить компонент, от которого зависят другие компоненты |
| ComponentServiceTest.deleteOfUnusedComponentSucceeds | Неиспользуемый компонент удаляется |

### Ж.2 Интеграционные и сквозные тесты

Интеграционное тестирование выполнено методом чёрного ящика по HTTP на прод-подобном стенде (47 API-тестов, скрипт tests/integration/run_integration.py) с дополнением сквозными E2E-тестами в браузере (9 тестов, Playwright). Полный протокол приведён в таблице Ж.3;

результат: 56 тестов, из них 34 PASS и 22 FAIL. Каждое отклонение связано с дефектом из реестра (см. раздел Ж.4).

Таблица Ж.3 – Протокол интеграционного и сквозного тестирования

| ID | Тест | Ожидаемый результат | Фактический результат | Статус | Дефект |
|---|---|---|---|---|---|
| INT-01 | Фронтенд (prod-сборка) отдаёт SPA и ассеты | 200, text/html; бандлы 200 с корректным Content-Type | index-CTuQi8U5.js text/javascript; charset=utf-8 244 КБ; index-CYtgMNis.css text/css; charset=utf-8 8 КБ | PASS | – |
| INT-02 | Прямой переход по клиентскому маршруту (deep link) | 200 и index.html (SPA fallback) | /editor/image → 200, /admin/users → 200, /no/such/route → 200 | PASS | – |
| INT-03 | Отдача несуществующего ассета | 404 для отсутствующего файла со статикой | HTTP 200, Content-Type text/html; charset=utf-8 – браузер получит HTML вместо JS | FAIL | DEF-01 |
| INT-04 | CORS: preflight с origin фронтенда | 200, Access-Control-Allow-Origin = origin | HTTP 200, ACAO=http://localhost:5173 | PASS | – |
| INT-05 | CORS: origin боевого домена | Разрешённые origin задаются конфигурацией (переменной окружения) | HTTP 403 «Invalid CORS request»: фронтенд на другом адресе не сможет обращаться к API, origin без пересборки не изменить | FAIL | DEF-02 |
| INT-06 | Схема БД создана Hibernate на чистой MySQL | Все таблицы есть, уникальный индекс на User.email | 11 таблиц, уникальный индекс email есть | PASS | – |
| INT-07 | Начальное наполнение (DataSeeder) через конвейер сборки | 67 компонентов, 15 образов; все стартовые образы READY и скачиваемы | 67 компонентов, 15/15 образов READY (очередь опустела за 0 с), вход admin OK | PASS | – |
| INT-08 | Регистрация и вход | 200, JWT, профиль с ролью USER | токен выдан, /users/me → it-c28bab-117d46@test.io ['USER'] | PASS | – |
| INT-09 | Валидация регистрации | 400 с понятным сообщением для каждого случая | weak password→400, bad email→400, empty name→400, duplicate→400 | PASS | – |
| INT-10 | Вход с неверным паролем / несуществующим email | 401 Unauthorized с сообщением об ошибке | неверный пароль → HTTP 403 «»; нет пользователя → HTTP 403 | FAIL | DEF-03 |
| INT-11 | Доступ к защищённым ресурсам без токена | 401 (фронтенд перенаправляет на вход только по 401) | /users/me → 403, /images/build → 403 | FAIL | DEF-04 |
| INT-12 | Повреждённый и чужой по подписи токен | 401, без 500 и стектрейсов | коды: {'мусор': 403, 'чужая подпись': 403} | FAIL | DEF-05 |
| INT-13 | Просроченный токен | 401 → фронтенд отправляет на вход | HTTP 403 (ExpiredJwtException не обработано) | FAIL | DEF-05 |
| INT- | Время жизни JWT | 24 часа (так указано в коде/документации) | 4.0 ч | FAIL | DEF- |

| ID | Тест | Ожидаемый результат | Фактический результат | Статус | Дефект |
|---|---|---|---|---|---|
| 14 | | | | | 06 |
| INT-15 | Ключ подписи JWT в исходном коде | Ключ берётся из секрета окружения; такой токен отклоняется | GET /users (ADMIN) с токеном из известного ключа → HTTP 200 | FAIL | DEF-07 |
| INT-16 | Разграничение ролей USER / ADMIN | 403 для всех | GET /users→403, POST /components→403, PUT /images→403, DELETE /images→403, DELETE /users/1→403 | PASS | – |
| INT-17 | Токен удалённого пользователя | 401 | HTTP 403 | FAIL | DEF-08 |
| INT-18 | Гонка: одновременная регистрация одного email | Ровно 1 успех, остальные 400/409, дублей в БД нет, 5xx нет | коды {200: 1, 409: 9, 400: 10}, строк в БД 1 | PASS | – |
| INT-19 | Пагинация: граничные значения | size обрезается до 100, page<0 → 0, пустая страница за пределами | size 1000→100, 0→1, page -1→0, page 999 → пусто | PASS | – |
| INT-20 | Нечисловые параметры (анонимный пользователь) | 400 Bad Request | {'page=abc': 403, 'components=x': 403, '/components/abc': 403} | FAIL | DEF-10 |
| INT-21 | Спецсимволы и инъекции в поиске | Нет 5xx; спецсимволы трактуются буквально (% и _ не шаблоны) | Инъекции нет (параметризованные запросы), но % и _ работают как шаблоны: найдено по 67 записей | FAIL | DEF-11 |
| INT-22 | Комбинация поиска, типа, сортировки и страниц | Без дублей и пропусков, порядок по цене сохраняется между страницами | 30 записей на 5 стр., дублей нет | PASS | – |
| INT-23 | Число SQL-запросов на одну страницу образов | ≤ 10 запросов (без проблемы N+1) | GET /images?size=12 → 23 SQL-запросов; GET /components?size=20 → 31 | FAIL | DEF-12 |
| INT-24 | Создание компонента с зависимостями и чтение | 201/200, зависимости сохранены | HTTP 200, id=68, deps=[1] | PASS | – |
| INT-25 | Защита графа зависимостей | 400 в обоих случаях, граф не меняется | self→400, cycle→400 | PASS | – |
| INT-26 | Удаление используемого и свободного компонента | 400 с сообщением; 204 | used→400 «Компонент «numpy» входит в образы – сначала измени», free→204, затем GET→404 | PASS | – |
| INT-27 | Валидация полей компонента | 400 для некорректных полей (404 допустим для несуществующего типа) | {'empty name': 400, 'negative cost': 400, 'name 300': 409, 'bad type': 404} | FAIL | DEF-13 |
| INT-28 | Параллельное редактирование компонента двумя админами | Без 5xx; итог – одно из отправленных значений | коды {200: 1, 409: 9}, итоговая цена 104: оптимистичная блокировка (@Version) отсекла конкурентные записи | PASS | – |
| INT-29 | Заказ сборки с разрешением транзи- | 202, QUEUED, в составе numpy и python3, цена = сумма компонентов | HTTP 202, state=QUEUED, состав=['numpy', 'opencv-python', 'python3'], cost=160 | PASS | – |

| ID | Тест | Ожидаемый результат | Фактический результат | Статус | Дефект |
|---|---|---|---|---|---|
| | тивных зависимостей | | | | |
| INT-30 | Сборка до READY и появление в «Мои образы» | READY, downloadable=true, образ в /users/me/images | READY за 2.1 с | PASS | – |
| INT-31 | Скачивание артефакта и проверка содержимого | 200, octet-stream, валидный ext4 (магия 0xEF53), размер 16+N МБ, манифест – валидный JSON с составом | ext4 OK, 19 МБ, пакеты ['numpy', 'opencv-python', 'python3'], манифест: IT Образ «Тест» c28bab | PASS | – |
| INT-32 | Имя файла при скачивании (кириллица) | Корректное имя файла (filename*=UTF-8"… по RFC 6266) | HTTP 200, заголовок Content-Disposition отсутствует (значение с не-Latin-1 символами отброшено), файл сохранится как «download» | FAIL | DEF-15 |
| INT-33 | Скачивание чужого образа | 403 (скачивать может владелец или ADMIN) | HTTP 200, получено 19 МБ чужого образа | FAIL | DEF-16 |
| INT-34 | Скачивание без токена и неготового образа | 401; 400 «не готов» | аноним → HTTP 403; неготовый → 400 | FAIL | DEF-04 |
| INT-35 | Валидация заказа | 400/404 без 5xx | {'empty components': 400, 'unknown component': 404, 'empty name': 400} | PASS | – |
| INT-36 | Очередь сборок: один воркер, FIFO | Одновременно BUILDING не более 1; последовательные заказы завершаются в порядке заказа; все READY | max BUILDING = 1, FIFO соблюдён, 12 сборок за 35,5 с (3,0 с на сборку); одновременные заказы идут в порядке фиксации транзакций | PASS | – |
| INT-37 | Изменение образа администратором с пересборкой | 202 → QUEUED → READY; новый состав в артефакте; старый файл заменён | 202, пересобран за 3.6 с, состав ['glibc', 'openssh'], cost 60; PUT во время сборки → 400 | PASS | – |
| INT-38 | Удаление образа администратором | 204; GET → 404; файл удалён с диска; пропал из «Мои образы» | 204, 404, файл удалён, связь с пользователем снята | PASS | – |
| INT-39 | Удаление пользователя, у которого есть образы | 204; образы остаются в библиотеке | 204, образ сохранён | PASS | – |
| INT-40 | Артефакт удалён с диска вне приложения | downloadable=false либо понятная ошибка; без 5xx | downloadable=True, download → HTTP 400 {"error":"Файл артефакта не найден на сервере"} | FAIL | DEF-17 |
| INT-41 | CRUD пользователей администратором | 201/200 на каждом шаге, новые права действуют | создан, повышен до ADMIN, вход с новым паролем, поиск+фильтр, удалён | PASS | – |
| INT-42 | Защита от самоблокировки администратора | 400 в обоих случаях | 400 / 400 | PASS | – |
| INT-43 | Смена email администратором самому себе | Сессия продолжает работать (или выдаётся новый токен) | PUT → 200, затем /users/me со старым токеном → 403 | FAIL | DEF-18 |

| ID | Тест | Ожидаемый результат | Фактический результат | Статус | Дефект |
|---|---|---|---|---|---|
| INT-44 | Перезапуск backend: сохранность данных | Данные сохранены, дублей каталога нет, токен остаётся валидным | компонентов/образов (71, 31) = до рестарта; простой 12 с; токен валиден | PASS | – |
| INT-45 | Падение backend (kill -9) во время сборки | Прерванные сборки возобновляются или помечаются FAILED; их можно пересобрать/удалить | Через 15 с после рестарта статусы {33: BUILDING, 34: QUEUED, 35: QUEUED}; PUT/DELETE зависших: (400, 400), (400, 204), (400, 204) | FAIL | DEF-19 |
| INT-46 | Недоступность БД | Во время сбоя – быстрый 5xx (≤ 5 с) без утечки деталей; после восстановления – 200 без рестарта | во время сбоя HTTP 403 за 28.3 с; восстановление без рестарта за 2.9 с | FAIL | DEF-20 |
| INT-47 | Состояние стенда после всех тестов | API отвечает, очередь пуста | API отвечает | PASS | – |
| E2E-01 | Открытие главной, редирект в библиотеку, отрисовка карточек | Редирект на /library/images, карточки отрисованы < 2 с | /library/images, карточек 12, первая карточка через 412 мс (DOMContentLoaded 50 мс) | PASS | – |
| E2E-02 | Регистрация через форму | Вход выполнен, токен сохранён, переход в библиотеку | после регистрации → /library/images, токен в localStorage: true | PASS | – |
| E2E-03 | Фильтр по компоненту и сортировка по цене | IoT Edge, Media Box (по убыванию цены) | IoT Edge, Media Box | PASS | – |
| E2E-04 | Заказ сборки в редакторе | Образ появляется в «Мои образы», статус доходит до «Готов», зависимость glibc добавлена автоматически | В кабинете: образ «E2E образ», Готов, buildroot, RV1103_Luckfox_Pico, glibc + ffmpeg, 80 ед., кнопка «Скачать .img» | PASS | – |
| E2E-05 | Скачивание .img из интерфейса | Браузер сохраняет файл <название>.img корректного размера | файл «E2E образ.img», 18 874 368 байт (18 МБ = 16 + 2 компонента) | PASS | – |
| E2E-06 | Недействительный токен в браузере | Сессия сбрасывается, редирект на /login | Пользователь остался на /account: «Request failed with status code 403»; API отвечает 403, а клиент перехватывает только 401 | FAIL | DEF-04 |
| E2E-07 | Админ: страница пользователей и поиск | Зарегистрированный в E2E-02 пользователь найден | пользователь найден | PASS | – |
| E2E-08 | Неверный пароль в форме входа | Понятное сообщение «Неверный email или пароль» | показано техническое сообщение «Request failed with status code 403» | FAIL | DEF-03 |
| E2E-09 | JS-ошибки за весь сценарий | Нет необработанных исключений JS | нет | PASS | – |

После оптимизации (индексы, пакетная загрузка связей, отключение Open Session In View, быстрый отказ при недоступной БД, ограничение неудачных

входов) интеграционный набор был выполнен повторно на чистой базе данных. Изменились результаты только двух тестов: INT-23 (число SQL-запросов на страницу образов: 9 вместо 23) и INT-46 (недоступность БД: ответ 503 за 3,0 с вместо 403 через 28,3 с) перешли из FAIL в PASS. Добавлен тест INT-48 «Защита от перебора паролей» – PASS (после 5 неудачных попыток ответ 429 с Retry-After 300 с, другой пользователь входит без ограничений). Итого 30 PASS из 48 API-тестов (было 27 из 47), регрессий нет; изменения сведены в таблицу Ж.4.

Таблица Ж.4 – Изменения результатов после оптимизации

| **ID** | **До** | **После** | **Комментарий** |
|---|---|---|---|
| INT-23 | FAIL | PASS | Страница образов: 9 SQL-запросов (было 23), устранена проблема N+1 (DEF-12) |
| INT-46 | FAIL | PASS | Недоступная БД: ответ 503 за 3,0 с, восстановление без рестарта за 1,0 с (DEF-20) |
| INT-48 | – (новый) | PASS | Защита от перебора паролей: 429 после 5 неудачных попыток, Retry-After 300 с (DEF-21) |

### Ж.3 Нагрузочное тестирование

Нагрузочное тестирование выполнено с помощью k6 (скрипт tests/load/scenarios.js) на стенде WSL2 Ubuntu 24.04, 16 vCPU, 7,5 ГБ, MySQL 8, JDK 17; сборка образов – в режиме симуляции. Профили нагрузки приведены в таблице Ж.5.

Таблица Ж.5 – Профили нагрузочного тестирования

| **ID** | **Профиль** | **Модель нагрузки** | **Критерии** |
|---|---|---|---|
| L-01 | Дымовой (baseline) | 1 VU, 1 мин; все пользовательские пути, каждая 5-я итерация – заказ и скачивание | Ошибки < 1 %; p95 < 300 мс |
| L-02 | Просмотр каталогов (гости) | Рост 0→50 VU за 1 мин, удержание 3 мин, спад 30 с; чтение каталогов с поиском, фильтрами и сортировкой | Ошибки < 1 %; p95 чтения < 500 мс |
| L-03 | Смешанный продакшен-профиль (целевая нагрузка) | 100 одновременных пользователей: гости 70, пользователи 20, заказчики 8, администраторы 2; разгон 1 мин, удержание 5 мин | Ошибки < 1 %; p95 чтения < 500 мс; p95 записи < 1000 мс |
| L-04 | Стресс (поиск предела) | Только чтение, ступени 50, 100, 200, 300, 450, 600 итераций/с по 1 мин, до 1500 VU | Ошибки < 1 %; p95 < 500 мс (нарушение ожидается на верхних ступенях) |

| ID | Профиль | Модель нагрузки | Критерии |
|---|---|---|---|
| L-04b | Стресс до отказа (breakpoint) | Ступени 1000, 1500, 2000, 3000, 4000 итераций/с по 1 мин, до 3000 VU | Фиксация точки насыщения (ошибки ≥ 1 % или p95 ≥ 500 мс) |
| L-05 | Всплеск (spike) | 10 VU, за 10 с рост до 400 VU, удержание 1 мин, возврат к 10 VU | Ошибки < 1 %; p95 < 500 мс; задержка возвращается к исходной |
| L-06 | Шторм авторизаций | Ступени 10, 25, 50, 100 входов/с по 30 с, 50 учётных записей | Ошибки < 1 %; p95 входа < 1000 мс |
| L-07 | Пакет заказов сборки (очередь) | 60 VU одновременно заказывают сборку, опрос статуса каждые 2 с до READY | Ошибки < 1 %; p95 заказа < 1000 мс; все 60 образов READY |
| L-08 | Скачивание артефактов | 20 VU, 2 мин; скачивание готовых образов размером 17–19 МБ | Ошибки < 1 %; p95 < 3000 мс |
| L-09 | Длительная нагрузка (soak) | 40 VU (гости 28, пользователи 8, заказчики 3, администратор 1), 15 мин | Ошибки < 1 %; p95 чтения < 500 мс; нет роста памяти и задержки |
| L-10a | Объём: базовая БД | Чтение каталогов, 40 запросов/с, 3 мин, стартовые данные | Ошибки < 1 %; p95 < 500 мс |
| L-10b | Объём: БД ×100 | То же на БД ×100: 20 000 образов, 2 000 компонентов, 5 000 пользователей | Ошибки < 1 %; p95 < 500 мс |

Сводные результаты прогона до оптимизации приведены в таблице Ж.6. Все 12 профилей (около 879 тыс. запросов) выполнены без HTTP-ошибок; критерии нарушены только в профиле L-04b, целью которого был поиск предела.

Таблица Ж.6 – Сводные результаты нагрузочного тестирования (до оптимизации)

| Профиль | Запросов | RPS | Ошибки, % | p95, мс | p99, мс | SLA |
|---|---|---|---|---|---|---|
| L-01 Дымовой (baseline) | 320 | 5,7 | 0,0 | 111,5 | 194,5 | PASS |
| L-02 Просмотр каталогов (гости) | 7 772 | 30,3 | 0,0 | 15,3 | 17,2 | PASS |
| L-03 Смешанный продакшен-профиль (целевая нагрузка) | 22 489 | 60,3 | 0,0 | 17,5 | 34,4 | PASS |
| L-04 Стресс (поиск предела) | 176 498 | 343,8 | 0,0 | 15,0 | 16,4 | PASS |
| L-04b Стресс до отказа (breakpoint) | 560 700 | 1 294,4 | 0,0 | 2 949,5 | 3 170,0 | FAIL |
| L-05 Всплеск (spike) | 55 548 | 292,9 | 0,0 | 15,4 | 18,0 | PASS |
| L-06 Шторм авторизаций | 9 618 | 42,4 | 0,0 | 987,9 | 1 049,8 | PASS |
| L-07 Пакет заказов сборки (очередь) | 3 059 | 16,1 | 0,0 | 35,7 | 157,3 | PASS |
| L-08 Скачивание артефактов | 2 256 | 19,6 | 0,0 | 103,9 | 234,6 | PASS |
| L-09 Длительная нагрузка (soak) | 23 230 | 26,4 | 0,0 | 18,6 | 73,4 | PASS |
| L-10a Объём: базовая БД | 8 845 | 51,5 | 0,0 | 9,2 | 10,3 | PASS |
| L-10b Объём: БД ×100 | 8 815 | 50,9 | 0,0 | 27,7 | 31,8 | PASS |

Очередь сборок под нагрузкой (таблица Ж.7) работает строго последовательно, по одной сборке за раз, без потерь и сбоев; ожидание в очереди в 3–4 раза дольше самой сборки (около 3,2 с в режиме симуляции). Время до статуса READY отсчитывается от ответа на заказ.

Таблица Ж.7 – Очередь сборок под нагрузкой

| **Профиль** | **Го-тово** | **Сбое в** | **Среднее, с** | **Меди-ана, с** | **p95, с** | **Макси-мум, с** | **Сбо-рок/мин** |
|---|---|---|---|---|---|---|---|
| L-01 Дымовой (baseline) | 6 | 0 | 4,1 | 4,1 | 4,1 | 4,1 | 6,4 |
| L-03 Смешанный продакшен-профиль (целевая нагрузка) | 116 | 0 | 12,5 | 14,1 | 16,1 | 18,1 | 18,6 |
| L-07 Пакет заказов сборки (очередь) | 60 | 0 | 98,1 | 98,8 | 181,1 | 189,5 | 19,0 |
| L-09 Длительная нагрузка (soak) | 173 | 0 | 4,4 | 4,0 | 6,3 | 10,1 | 11,8 |

После оптимизации те же профили были выполнены повторно; ключевые показатели до и после сведены в таблицу Ж.8. Предел пропускной способности чтения вырос с 1 468,1 до 3 829,6 запросов/с, число SQL-запросов на HTTP-запрос снизилось более чем вдвое, HTTP-ошибок по-прежнему нет. Единственные ухудшения – p95 и p99 профиля L-10b (БД ×100) в пределах разброса измерений.

Таблица Ж.8 – Сравнение до и после оптимизации

| **Метрика** | **До** | **После** | **Изменение** |
|---|---|---|---|
| L-04b: Макс. пропускная способность, запросов/с | 1 468,1 | 3 829,6 | +160,9 % |
| L-04b: Медиана на ступени 1 500 ит/с, мс | 2 732,2 | 3,2 | −99,9 % |
| L-04b: Медиана на ступени 2 000 ит/с, мс | 2 692,0 | 3,4 | −99,9 % |
| L-04b: Медиана на ступени 4 000 ит/с, мс | 2 138,2 | 826,7 | −61,3 % |
| L-04b: Ошибки, % | 0 | 0 | без изменений |
| L-04b: SQL-запросов на HTTP-запрос | 12,6 | 5,6 | −55,6 % |
| L-04: p95 за профиль, мс | 15,0 | 6,9 | −54,3 % |
| L-04: p95 на ступени 600 ит/с, мс | 12,4 | 6,3 | −49,2 % |
| L-04: CPU mysqld средн., % | 97,4 | 55,7 | −42,8 % |
| L-04: CPU java средн., % | 80,1 | 55,8 | −30,3 % |
| L-04: SQL-запросов на HTTP-запрос | 12,5 | 5,6 | −55,2 % |
| L-10a: p95, мс (стартовая БД, 40 запросов/с) | 9,2 | 7,9 | −13,7 % |
| L-10b: p95, мс (БД ×100, 40 запросов/с) | 27,7 | 29,8 | +7,6 % |
| L-10b: p99, мс (БД ×100) | 31,8 | 40,5 | +27,1 % |
| L-10b: CPU mysqld средн., % (БД ×100) | 30,9 | 28,3 | −8,4 % |
| L-10b: SQL-запросов на HTTP-запрос (БД ×100) | 17,1 | 5,7 | −66,7 % |

| Метрика | До | После | Изменение |
|---|---|---|---|
| L-03: p95, мс (100 VU, целевой профиль) | 17,5 | 5,6 | −68,2 % |
| L-03: p99, мс | 34,4 | 16,3 | −52,7 % |
| L-03: Ошибки, % | 0 | 0 | без изменений |
| L-06: p95 входа, мс (до 100 входов/с) | 987,9 | 121,5 | −87,7 % |
| L-06: Ошибки, % | 0 | 0 | без изменений |

### Ж.4 Реестр дефектов

По результатам интеграционного и нагрузочного тестирования выявлено 22 дефекта: 1 критический, 4 высоких, 9 средних и 8 низких. Реестр приведён в таблице Ж.9. Статус «Исправлен» означает, что дефект устранён при оптимизации и подтверждён повторным прогоном; остальные дефекты на момент составления отчёта остаются открытыми.

Таблица Ж.9 – Реестр дефектов

| ID | Серьёзность | Описание | Тесты | Статус |
|---|---|---|---|---|
| DEF-07 | Критическая | Секрет JWT захардкожен в JwtService и лежит в Git: любой, кто видел репозиторий, может выпустить токен администратора | INT-15 | Открыт |
| DEF-05 | Высокая | Некорректный/поддельный/просроченный JWT приводит к исключению в JwtAuthenticationFilter (ответ 403/500 вместо 401), в лог пишется стектрейс | INT-12, INT-13 | Открыт |
| DEF-16 | Высокая | Нет проверки владельца при скачивании: любой авторизованный пользователь скачивает чужие образы по id (IDOR) | INT-33 | Открыт |
| DEF-19 | Высокая | Очередь сборок хранится только в памяти: после перезапуска или падения backend образы навсегда остаются в QUEUED/BUILDING и не могут быть изменены или удалены | INT-45 | Открыт |
| DEF-21 | Высокая | Нет ограничения частоты попыток входа: за 4 мин с одного адреса принято 9 618 входов; при 100 попытках/с BCrypt занимает около 8 ядер, вход остальных замедляется в 11 раз | L-06 | Исправлен: 429 после 5 неудач (INT-48) |
| DEF-02 | Средняя | Разрешённый CORS-origin захардкожен (http://localhost:5173) в WebConfig | INT-05 | Открыт |
| DEF-03 | Средняя | Неверный логин/пароль возвращает 403 без тела вместо 401 | INT-10, E2E-08 | Открыт |
| DEF-04 | Средняя | Запрос без токена/с недействительным токеном получает 403 вместо 401 – фронтенд (interceptor по 401) не выкидывает пользователя на страницу входа | INT-11, INT-34, E2E- | Открыт |

| ID | Серьёзность | Описание | Тесты | Статус |
|---|---|---|---|---|
| | | | 06 | |
| DEF-08 | Средняя | Запрос с токеном удалённого пользователя: UsernameNotFoundException в фильтре → 403/500 | INT-17 | Открыт |
| DEF-10 | Средняя | Ошибки разбора параметров у анонимного пользователя возвращают 403 вместо 400 (ошибка уходит на /error, закрытый Spring Security) | INT-20 | Открыт |
| DEF-12 | Средняя | N+1 запросов: список из 12 образов порождает десятки SQL-запросов (ленивые components и dependencies грузятся поштучно) | INT-23 | Исправлен: 389 → 11 SQL (L-10b повторно) |
| DEF-20 | Средняя | При недоступной БД запрос висит ~30 с (connection-timeout Hikari по умолчанию), анонимный клиент получает 403 вместо 503 | INT-46 | Исправлен: 503 за 3 с (INT-46) |
| DEF-22 | Средняя | Чтение ограничено около 1 300 запросов/с из-за БД (около 12,5 SQL на запрос, пул 10 соединений); при превышении запросы копятся до 2–3 с вместо быстрого отказа 503 | L-04, L-04b | Исправлен: предел 1 470 → 3 830 rps |
| DEF-24 | Средняя | Очередь с одним воркером без лимита заказов и без ETA: в L-07 60 заказов ждали до 189 с; при реальной сборке 1–3 ч ожидание составит 60–180 ч | INT-36, L-07 | Открыт |
| DEF-01 | Низкая | server.mjs отдаёт index.html со статусом 200 вместо 404 для отсутствующих /assets/* | INT-03 | Открыт |
| DEF-06 | Низкая | Срок жизни JWT 4 ч вместо заявленных 24 ч (1000*60*24*10 мс) | INT-14 | Открыт |
| DEF-11 | Низкая | Символы % и _ в поиске не экранируются и работают как шаблоны LIKE (поиск «%» возвращает весь каталог) | INT-21 | Открыт |
| DEF-13 | Низкая | Имя компонента длиннее 255 символов даёт 409 с вводящим в заблуждение сообщением «запись используется в других объектах» вместо 400 валидации | INT-27 | Открыт |
| DEF-15 | Низкая | Для образов с кириллическим названием теряется заголовок Content-Disposition: файл сохраняется без имени и расширения (интерфейс не затронут) | INT-32 | Открыт |
| DEF-17 | Низкая | Если файл артефакта пропал с диска, карточка продолжает показывать «Скачать» (downloadable=true), а скачивание даёт 400 | INT-40 | Открыт |
| DEF-18 | Низкая | После смены собственного email старый JWT (sub=старый email) перестаёт работать – админ внезапно получает ошибки вместо перелогина | INT-43 | Открыт |
| DEF-23 | Низкая | Нет индексов для поиска, фильтра по статусу и сортировки по цене; на 20 тыс. образов page=1500 замедлилась в 7,5 раза, LIKE '%…%' – полный проход | L-10b | Частично: индексы добавлены, поиск по подстроке – полный проход |

# ПРИЛОЖЕНИЕ И
# ОСНОВНЫЕ ЛИСТИНГИ ПРОГРАММНОГО КОДА

Листинг И.1 – Класс ImageService (заказ, изменение, удаление образа, разрешение зависимостей)

```
@Service
@RequiredArgsConstructor
public class ImageService {

    private static final Logger log = LoggerFactory.getLogger(ImageService.class);
    private static final String DEFAULT_BOARD = "RV1103_Luckfox_Pico";

    private final ImageRepository imageRepository;
    private final ComponentRepository componentRepository;
    private final UserRepository userRepository;
    private final BuildService buildService;

    /**
     * Страница библиотеки: поиск по названию + фильтры по статусу, плате и компонентам
     * (образ должен содержать все выбранные компоненты, с учётом транзитивных
     * зависимостей). Сортировка — новые сверху или по цене.
     */
    @Transactional(readOnly = true)
    public PageResponse<ImageDto> search(String name, String state, String board, List<Long> com-
ponentIds,
                                         String sort, int page, int size) {
        Specification<Image> spec = (root, query, cb) -> {
            List<Predicate> where = new ArrayList<>();
            if (name != null && !name.isBlank()) {
                where.add(cb.like(cb.lower(root.get("name")), "%" + name.trim().toLowerCase() +
"%"));
            }
            if (state != null && !state.isBlank()) {
                where.add(cb.equal(root.get("state"), state.toUpperCase()));
            }
            if (board != null && !board.isBlank()) {
                where.add(cb.equal(root.get("board"), board));
            }
            if (componentIds != null) {
                // По EXISTS-подзапросу на каждый компонент — без join-дублей в основной выборке.
                for (Long componentId : new LinkedHashSet<>(componentIds)) {
                    Subquery<Long> sq = query.subquery(Long.class);
                    Root<Image> img = sq.from(Image.class);
                    Join<Image, Component> comp = img.join("components");
                    sq.select(img.get("id")).where(
                            cb.equal(img.get("id"), root.get("id")),
                            cb.equal(comp.get("id"), componentId));
                    where.add(cb.exists(sq));
                }
            }
            return cb.and(where.toArray(new Predicate[0]));
        };
        Sort order = PageResponse.sort(sort, Sort.by(Sort.Direction.DESC, "id"));
        return PageResponse.from(
                imageRepository.findAll(spec, PageResponse.request(page, size, order)),
                ImageDto::from);
    }

    @Transactional(readOnly = true)
    public ImageDto getDtoById(Long id) {
        return ImageDto.from(getById(id));
    }

    @Transactional(readOnly = true)
    public Image getById(Long id) {
        return imageRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Образ не найден: " + id));
    }
```

```
    /**
     * Заказ сборки: разрешает транзитивные зависимости выбранных компонентов,
     * проверяет граф на циклы, сохраняет образ в статусе QUEUED, привязывает его
     * к пользователю-заказчику и запускает фоновую сборку.
     */
    @Transactional
    public ImageDto orderBuild(BuildRequest req, String userEmail) {
        User user = userRepository.findByEmail(userEmail)
                .orElseThrow(() -> new ResourceNotFoundException("Пользователь не найден: " + us-
erEmail));

        Image image = new Image();
        applyAndQueue(image, req);
        image = imageRepository.save(image);

        // Привязка образа к пользователю (user_image_order).
        if (user.getOrderedImages() == null) {
            user.setOrderedImages(new HashSet<>());
        }
        user.getOrderedImages().add(image);
        userRepository.save(user);

        scheduleBuild(image.getId());
        return ImageDto.from(image);
    }

    /**
     * Изменение образа администратором: новые параметры и состав, пересчёт
     * зависимостей и повторная постановка в очередь сборки.
     */
    @Transactional
    public ImageDto update(Long id, BuildRequest req) {
        Image image = getById(id);
        if ("QUEUED".equals(image.getState()) || "BUILDING".equals(image.getState())) {
            throw new BadRequestException("Образ сейчас в очереди или собирается — дождитесь
окончания сборки");
        }
        String oldArtifact = image.getArtifactPath();
        applyAndQueue(image, req);
        image.setArtifactPath(null);
        image.setBuildLog(null);
        image = imageRepository.save(image);

        afterCommit(() -> deleteArtifact(oldArtifact));
        scheduleBuild(image.getId());
        return ImageDto.from(image);
    }

    @Transactional
    public void delete(Long id) {
        Image image = getById(id);
        if ("BUILDING".equals(image.getState())) {
            throw new BadRequestException("Образ сейчас собирается — удалить его можно после
окончания сборки");
        }
        // Связь «пользователь заказал образ» принадлежит User — снимаем её до удаления.
        for (User u : userRepository.findByOrderedImagesContains(image)) {
            u.getOrderedImages().remove(image);
        }
        String artifact = image.getArtifactPath();
        imageRepository.delete(image);
        afterCommit(() -> deleteArtifact(artifact));
    }

    /** Общая часть создания и изменения: поля, разрешение зависимостей, стоимость, QUEUED. */
    private void applyAndQueue(Image image, BuildRequest req) {
        Set<Component> selected = new LinkedHashSet<>();
        for (Long id : req.getComponentIds()) {
            selected.add(componentRepository.findById(id)
                    .orElseThrow(() -> new ResourceNotFoundException("Компонент не найден: " +
id)));
        }

        // Транзитивное замыкание зависимостей + проверка на циклы.
        Set<Component> resolved = resolveDependencies(selected);
```

```
        detectCycles(resolved);

        image.setName(req.getName());
        image.setDescription(req.getDescription());
        image.setDistro(req.getDistro());
        image.setBoard(req.getBoard() == null || req.getBoard().isBlank()
                ? DEFAULT_BOARD : req.getBoard());
        image.setState("QUEUED");
        image.setCost(resolved.stream().mapToLong(Component::getCost).sum());
        image.setComponents(resolved);
    }

    /**
     * Фоновую сборку запускаем строго ПОСЛЕ коммита транзакции, иначе
     * async-поток может не найти ещё не сохранённый образ.
     */
    private void scheduleBuild(Long imageId) {
        afterCommit(() -> buildService.build(imageId));
    }

    private void afterCommit(Runnable action) {
        if (TransactionSynchronizationManager.isSynchronizationActive()) {
            TransactionSynchronizationManager.registerSynchronization(new TransactionSynchroniza-
tion() {
                @Override
                public void afterCommit() {
                    action.run();
                }
            });
        } else {
            action.run();
        }
    }

    private void deleteArtifact(String artifactPath) {
        if (artifactPath == null) return;
        try {
            Files.deleteIfExists(Paths.get(artifactPath));
        } catch (IOException e) {
            log.warn("Не удалось удалить артефакт {}: {}", artifactPath, e.getMessage());
        }
    }

    /** BFS по графу зависимостей: возвращает выбранные компоненты + все требуемые ими. */
    private Set<Component> resolveDependencies(Set<Component> selected) {
        Set<Component> result = new LinkedHashSet<>();
        Deque<Component> queue = new ArrayDeque<>(selected);
        while (!queue.isEmpty()) {
            Component c = queue.poll();
            if (!result.add(c)) {
                continue;
            }
            if (c.getDependencies() != null) {
                for (Component dep : c.getDependencies()) {
                    if (!result.contains(dep)) {
                        queue.add(dep);
                    }
                }
            }
        }
        return result;
    }

    /** DFS-детекция циклов в графе зависимостей замыкания. */
    private void detectCycles(Set<Component> components) {
        Set<Long> visited = new HashSet<>();
        Set<Long> inStack = new HashSet<>();
        for (Component c : components) {
            if (!visited.contains(c.getId())) {
                dfs(c, visited, inStack);
            }
        }
    }

    private void dfs(Component c, Set<Long> visited, Set<Long> inStack) {
        visited.add(c.getId());
```

```
            inStack.add(c.getId());
            if (c.getDependencies() != null) {
                for (Component dep : c.getDependencies()) {
                    if (inStack.contains(dep.getId())) {
                        throw new BadRequestException(
                                "Обнаружена циклическая зависимость с участием компонента '" + c.get-
Name() + "'");
                    }
                    if (!visited.contains(dep.getId())) {
                        dfs(dep, visited, inStack);
                    }
                }
            }
            inStack.remove(c.getId());
        }
    }
```

Листинг И.2 – Класс BuildService (выполнение сборки)

```
/**
 * Фоновая оркестрация сборки образа. Вызывает обёртку scripts/luckfox_build.sh,
 * которая выполняет реальную сборку SDK либо симуляцию (см. скрипт).
 */
@Service
@RequiredArgsConstructor
public class BuildService {

    private static final Logger log = LoggerFactory.getLogger(BuildService.class);

    private final BuildStateService stateService;
    private final NotificationService notificationService;

    @Value("${luckfox.build.script:../scripts/luckfox_build.sh}")
    private String scriptPath;

    @Value("${luckfox.build.output-dir:../output/images}")
    private String outputDir;

    @Value("${luckfox.build.timeout-seconds:1800}")
    private long timeoutSeconds;

    @Async("buildExecutor")
    public void build(Long imageId) {
        BuildStateService.BuildInputs in;
        try {
            in = stateService.loadInputs(imageId);
        } catch (Exception e) {
            log.error("Не удалось загрузить данные образа #{}: {}", imageId, e.getMessage());
            return;
        }

        stateService.markBuilding(imageId);
        log.info("Старт сборки образа #{} '{}'", imageId, in.name());

        try {
            Path script = Paths.get(scriptPath).toAbsolutePath().normalize();
            Path outDir = Paths.get(outputDir).toAbsolutePath().normalize();
            Files.createDirectories(outDir);
            Path outFile = outDir.resolve("image-" + imageId + ".img");

            List<String> cmd = new ArrayList<>(List.of(
                    "bash", script.toString(),
                    "--id", String.valueOf(imageId),
                    "--name", safe(in.name()),
                    "--distro", safe(in.distro()),
                    "--board", safe(in.board()),
                    "--components", String.join(",", in.components()),
                    "--out", outFile.toString()
            ));

            ProcessBuilder pb = new ProcessBuilder(cmd);
            pb.redirectErrorStream(true);
            Process process = pb.start();
```

```
            StringBuilder logBuf = new StringBuilder();
            try (BufferedReader reader = new BufferedReader(
                    new InputStreamReader(process.getInputStream(), StandardCharsets.UTF_8))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    logBuf.append(line).append('\n');
                    log.info("[build #{}] {}", imageId, line);
                }
            }

            boolean finished = process.waitFor(timeoutSeconds, java.util.concurrent.TimeUnit.SEC-
ONDS);
            if (!finished) {
                process.destroyForcibly();
                stateService.markFailed(imageId, logBuf + "\nОшибка: превышен таймаут сборки");
                log.error("Сборка образа #{} прервана по таймауту", imageId);
                return;
            }

            int exit = process.exitValue();
            if (exit == 0 && Files.exists(outFile)) {
                stateService.markReady(imageId, outFile.toString(), logBuf.toString());
                log.info("Сборка образа #{} завершена: {}", imageId, outFile);
                notificationService.notifyImageReady(imageId, in.name(), outFile);
            } else {
                stateService.markFailed(imageId, logBuf + "\nКод возврата: " + exit);
                log.error("Сборка образа #{} провалена (код {})", imageId, exit);
            }
        } catch (Exception e) {
            stateService.markFailed(imageId, "Исключение при сборке: " + e.getMessage());
            log.error("Сборка образа #{} упала с исключением", imageId, e);
        }
    }

    private String safe(String s) {
        return s == null ? "" : s;
    }
}
```

Листинг И.3 – Класс BuildStateService (транзакции статусов образа)

```
/**
 * Транзакционные операции над состоянием образа, вызываемые из фонового
 * {@link BuildService}. Вынесено отдельно, чтобы избежать самовызова
 * @Transactional/@Async и циклической зависимости с ImageService.
 */
@Service
@RequiredArgsConstructor
public class BuildStateService {

    private final ImageRepository imageRepository;

    /** Снимок входных данных сборки (читается в транзакции, lazy-коллекция инициализируется). */
    public record BuildInputs(Long id, String name, String distro, String board, List<String>
components) {}

    @Transactional(readOnly = true)
    public BuildInputs loadInputs(Long imageId) {
        Image image = imageRepository.findById(imageId)
                .orElseThrow(() -> new ResourceNotFoundException("Образ не найден: " + imageId));
        List<String> names = image.getComponents() == null ? List.of()
                : image.getComponents().stream().map(Component::getName).sorted().collect(Collec-
tors.toList());
        return new BuildInputs(image.getId(), image.getName(), image.getDistro(), image.get-
Board(), names);
    }

    @Transactional
    public void markBuilding(Long imageId) {
        setState(imageId, "BUILDING", null, null);
    }

    @Transactional
```

```
    public void markReady(Long imageId, String artifactPath, String log) {
        setState(imageId, "READY", artifactPath, log);
    }

    @Transactional
    public void markFailed(Long imageId, String log) {
        setState(imageId, "FAILED", null, log);
    }

    private void setState(Long imageId, String state, String artifactPath, String log) {
        Image image = imageRepository.findById(imageId)
                .orElseThrow(() -> new ResourceNotFoundException("Образ не найден: " + imageId));
        image.setState(state);
        if (artifactPath != null) {
            image.setArtifactPath(artifactPath);
        }
        if (log != null) {
            image.setBuildLog(log);
        }
        imageRepository.save(image);
    }
}
```

## Листинг И.4 – Класс AsyncConfig (очередь сборок)

```
@Configuration
@EnableAsync
public class AsyncConfig {

    /**
     * Очередь сборок: ровно ОДИН воркер. Пока собирается образ одного пользователя,
     * остальные заказы ждут в очереди (FIFO) в статусе QUEUED и берутся по одному.
     *
     * Это обязательно: реальная сборка SDK использует единый рабочий каталог
     * (luckfox-pico/output, .BoardConfig.mk) — параллельные сборки повредили бы друг друга.
     */
    @Bean(name = "buildExecutor")
    public Executor buildExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(1);
        executor.setMaxPoolSize(1);          // строго один воркер — настоящая очередь
        executor.setQueueCapacity(1000);     // ожидающие заказы
        executor.setThreadNamePrefix("build-worker-");
        executor.initialize();
        return executor;
    }
}
```

## Листинг И.5 – Класс ImageController (API образов)

```
@RestController
@RequestMapping("/api/images")
@RequiredArgsConstructor
public class ImageController {

    private final ImageService imageService;

    @GetMapping
    public PageResponse<ImageDto> getAllImages(@RequestParam(required = false) String search,
                                               @RequestParam(required = false) String state,
                                               @RequestParam(required = false) String board,
                                               // components=1,2,3 — образы, содержащие все
перечисленные
                                               @RequestParam(required = false) List<Long> compo-
nents,
                                               @RequestParam(required = false) String sort,
                                               @RequestParam(defaultValue = "0") int page,
                                               @RequestParam(defaultValue = "12") int size) {
        return imageService.search(search, state, board, components, sort, page, size);
    }

    @GetMapping("/{id}")
```

```
    public ImageDto getImageById(@PathVariable Long id) {
        return imageService.getDtoById(id);
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<ImageDto> update(@PathVariable Long id, @Valid @RequestBody BuildRe-
quest request) {
        return ResponseEntity.accepted().body(imageService.update(id, request));
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        imageService.delete(id);
        return ResponseEntity.noContent().build();
    }

    @PostMapping("/build")
    public ResponseEntity<ImageDto> buildImage(@Valid @RequestBody BuildRequest request,
                                               Authentication authentication) {
        return ResponseEntity.accepted().body(imageService.orderBuild(request, authentica-
tion.getName()));
    }

    @GetMapping("/{id}/download")
    public ResponseEntity<Resource> download(@PathVariable Long id) {
        Image image = imageService.getById(id);
        if (!"READY".equalsIgnoreCase(image.getState()) || image.getArtifactPath() == null) {
            throw new BadRequestException("Образ ещё не готов к загрузке (статус: " + im-
age.getState() + ")");
        }
        Path path = Paths.get(image.getArtifactPath());
        if (!Files.exists(path)) {
            throw new BadRequestException("Файл артефакта не найден на сервере");
        }
        String filename = (image.getName() == null ? "image-" + id : image.getName()) + ".img";
        return ResponseEntity.ok()
                .contentType(MediaType.APPLICATION_OCTET_STREAM)
                .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"" + filename +
"\"")
                .body(new FileSystemResource(path));
    }
}
```

## Листинг И.6 – Сценарий scripts/luckfox_build.sh

```
#!/usr/bin/env bash
#
# luckfox_build.sh — обёртка оркестрации сборки Linux-образа для Luckfox Pico.
#
# Вызывается backend-ом (BuildService) для одного заказа на сборку.
# Работает в двух режимах:
#
#   1. Реальная сборка SDK  (LUCKFOX_REAL_BUILD=1):
#        запускает luckfox-pico/build.sh lunch <board> && build.sh,
#        затем копирует получившийся output/image/*.img в выходной файл.
#        Требует развёрнутого SDK (sysdrv/source/...) и кросс-тулчейна.
#
#   2. Симуляция (по умолчанию, для прототипа):
#        собирает реальный .img-артефакт (ext4 rootfs с манифестом и списком
#        выбранных пакетов). Позволяет проверить весь конвейер без многочасовой
#        кросс-компиляции.
#
# Использование:
#   luckfox_build.sh --id <imageId> --name <name> --distro <distro> \
#       --board <board> --components "pkg1,pkg2,..." --out <path/to.img>
#
# Коды возврата: 0 — успех, !=0 — ошибка сборки (backend пометит образ FAILED).

set -euo pipefail

# ---------------------------------------------------------------- параметры ---
IMAGE_ID=""
```

```
IMAGE_NAME="image"
DISTRO="buildroot"
BOARD="RV1103_Luckfox_Pico"
COMPONENTS=""
OUT=""

while [[ $# -gt 0 ]]; do
    case "$1" in
        --id)         IMAGE_ID="$2"; shift 2 ;;
        --name)       IMAGE_NAME="$2"; shift 2 ;;
        --distro)     DISTRO="$2"; shift 2 ;;
        --board)      BOARD="$2"; shift 2 ;;
        --components) COMPONENTS="$2"; shift 2 ;;
        --out)        OUT="$2"; shift 2 ;;
        *) echo "[luckfox_build] неизвестный аргумент: $1" >&2; exit 2 ;;
    esac
done

if [[ -z "$OUT" ]]; then
    echo "[luckfox_build] не задан --out" >&2
    exit 2
fi

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
SDK_DIR="${LUCKFOX_SDK_DIR:-$REPO_ROOT/luckfox-pico}"

# Buildroot (внутри SDK) отказывается собираться, если в $PATH есть элементы с
# пробелами/табами/переводами строк. В WSL в PATH подмешиваются Windows-пути
# (/mnt/c/Program Files/...). Удаляем такие элементы — линуксовой сборке они не нужны.
PATH="$(printf '%s' "$PATH" | tr ':' '\n' | grep -vE '[[:space:]]' | paste -sd:)"
export PATH

mkdir -p "$(dirname "$OUT")"

log() { echo "[luckfox_build] $*"; }

log "Заказ #$IMAGE_ID | образ='$IMAGE_NAME' | дистрибутив=$DISTRO | плата=$BOARD"
log "Компоненты: ${COMPONENTS:-<нет>}"

# ------------------------------------------------------------ реальный режим ---
real_build() {
    log "Режим: РЕАЛЬНАЯ сборка SDK"
    if [[ ! -x "$SDK_DIR/build.sh" ]]; then
        log "ОШИБКА: $SDK_DIR/build.sh не найден/не исполняемый"
        return 1
    fi
    if [[ ! -d "$SDK_DIR/sysdrv/source" ]]; then
        log "ОШИБКА: исходники SDK ($SDK_DIR/sysdrv/source) отсутствуют — разверните SDK."
        return 1
    fi

    pushd "$SDK_DIR" >/dev/null
    log "lunch -> $BOARD"
    # Выбор конфигурации платы по подстроке имени (SD_CARD-Buildroot по умолчанию).
    local cfg
    cfg="$(ls project/cfg/BoardConfig_IPC/BoardConfig-SD_CARD-Buildroot-*"$BOARD"*.mk 2>/dev/null
| head -n1 || true)"
    [[ -z "$cfg" ]] && cfg="$(ls project/cfg/BoardConfig_IPC/BoardConfig-*"$BOARD"*.mk
2>/dev/null | head -n1 || true)"
    if [[ -z "$cfg" ]]; then
        log "ОШИБКА: не найден BoardConfig для '$BOARD'"
        popd >/dev/null
        return 1
    fi
    log "BoardConfig: $cfg"
    ln -rfs "$cfg" "$SDK_DIR/.BoardConfig.mk"

    # Полная сборка: uboot + kernel + rootfs + recovery + прошиваемый update.img.
    ./build.sh
    ./build.sh updateimg || log "updateimg пропущен"
    popd >/dev/null

    # Предпочитаем прошиваемый update.img, иначе — самый свежий .img.
    local img
```

```
    if [[ -f "$SDK_DIR/output/image/update.img" ]]; then
        img="$SDK_DIR/output/image/update.img"
    else
        img="$(ls -t "$SDK_DIR"/output/image/*.img 2>/dev/null | head -n1 || true)"
    fi
    if [[ -z "$img" ]]; then
        log "ОШИБКА: сборка не произвела .img"
        return 1
    fi
    cp "$img" "$OUT"
    log "Готово: $OUT ($(basename "$img"), $(du -h "$OUT" | cut -f1))"
    return 0
}

# ------------------------------------------------------------- симуляция -----
# Собирает настоящий ext4-образ с rootfs-метаданными и списком пакетов.
sim_build() {
    log "Режим: СИМУЛЯЦИЯ (прототип). Для реальной кросс-сборки: LUCKFOX_REAL_BUILD=1"

    # Необязательная задержка — имитирует длительность сборки (для демонстрации очереди).
    if [[ "${LUCKFOX_SIM_DELAY:-0}" -gt 0 ]]; then
        log "Имитация работы: ${LUCKFOX_SIM_DELAY}c"
        sleep "${LUCKFOX_SIM_DELAY}"
    fi

    local workdir
    workdir="$(mktemp -d)"
    trap 'rm -rf "$workdir"' RETURN

    local rootfs="$workdir/rootfs"
    mkdir -p "$rootfs/etc" "$rootfs/usr/lib" "$rootfs/usr/bin" "$rootfs/var/log"

    # JSON-массив компонентов средствами bash (без awk — его может не быть в slim-образе).
    local comp_json="" first=1 IFS_OLD="$IFS"
    IFS=','
    for c in $COMPONENTS; do
        c="${c#"${c%%[![:space:]]*}"}"   # trim leading
        c="${c%"${c##*[![:space:]]}"}"   # trim trailing
        [ -z "$c" ] && continue
        if [ $first -eq 1 ]; then comp_json="\"$c\""; first=0; else
comp_json="$comp_json,\"$c\""; fi
    done
    IFS="$IFS_OLD"

    # Метаданные образа.
    cat >"$rootfs/etc/luckfox-image.json" <<EOF
{
  "imageId": "${IMAGE_ID}",
  "name": "${IMAGE_NAME}",
  "distro": "${DISTRO}",
  "board": "${BOARD}",
  "builtAt": "$(date -u +%Y-%m-%dT%H:%M:%SZ)",
  "components": [${comp_json}]
}
EOF
    echo "Luckfox Pico custom image: ${IMAGE_NAME} (${DISTRO}/${BOARD})" >"$rootfs/etc/issue"
    echo "$COMPONENTS" | tr ',' '\n' | sed '/^[[:space:]]*$/d' >"$rootfs/etc/installed-pack-
ages.txt"

    # Размер образа: база 16 МБ + по 1 МБ на компонент (демонстрация).
    local ncomp size_mb
    ncomp="$(grep -c . "$rootfs/etc/installed-packages.txt" 2>/dev/null || echo 0)"
    size_mb=$(( 16 + ncomp ))

    if command -v mkfs.ext4 >/dev/null 2>&1 && command -v dd >/dev/null 2>&1; then
        dd if=/dev/zero of="$OUT" bs=1M count="$size_mb" status=none
        # -d заполняет ФС содержимым каталога (util-linux/e2fsprogs >= 1.43).
        if mkfs.ext4 -q -F -L luckfox -d "$rootfs" "$OUT" 2>/dev/null; then
            log "Создан ext4-образ с rootfs (${size_mb} МБ)"
        else
            mkfs.ext4 -q -F -L luckfox "$OUT"
            log "Создан ext4-образ (${size_mb} МБ, без -d)"
        fi
    else
        # Фолбэк без e2fsprogs: упакованный tar как .img.
        log "mkfs.ext4 недоступен — собираю tar-артефакт"
```

```
        tar -C "$rootfs" -cf "$OUT" .
    fi

    log "Готово: $OUT ($(du -h "$OUT" | cut -f1))"
    return 0
}

# ---------------------------------------------------------------- запуск -----
if [[ "${LUCKFOX_REAL_BUILD:-0}" == "1" ]]; then
    real_build
else
    sim_build
fi
```